

Forge

A certificate-producing proof-planning layer
above Lean's grind

Merged working draft

Consolidated from nine independent proposals

What this document is. A design for a substantially broader Lean 4 automation system that keeps `grind` as its local saturation engine and adds an outer layer that *changes the proof obligation*: it selects induction principles, generalises accumulators, synthesises auxiliary lemmas, constructs witnesses, and obtains exact algebraic certificates. Every component that can succeed must return something Lean can check.

What this document is not. *Forge* is *not an implemented Lean tactic*. None of the nine contributing efforts had a Lean executable available; no Lean source was compiled by any of them; no comparison against `grind`, `Aesop`, `nlinarith`, `LeanHammer`, or any other Lean tactic was measured. The Python prototypes are research code and are not formally verified. A passing Python check is not a Lean-kernel proof.

Baseline. Lean v4.34.0, released 14 September 2026, with Mathlib revision `1cf325a0cf67aca2b04d76b5380ff6a9e410aefa`.

Contents

1	The objective: more proof construction, not a bigger timeout	4
1.1	Two meanings of “more powerful”	4
1.2	Where new coverage can realistically come from	5
1.3	The deliverable and its evidence levels	5
1.4	What was actually built, and what was not	5
2	A strong baseline, not a straw man	6
2.1	The neighbouring tactics, and the gap between them	6
2.2	Related systems: reuse, do not relabel	7
2.3	A dependency map with integration decisions	8
2.4	A portfolio alone is not the proposal	8
3	Architecture: a scoped obligation graph around a shared fact store	9
3.1	Three data structures that must never be confused	9
3.2	The demand protocol and the worker contract	9
3.3	Three outcomes, not an overloaded Boolean	10
3.4	Snapshots, scope keys, and proof transport	10
3.5	Facts carry explanations, not only truth flags	11
3.6	Event-driven cooperation	11
3.7	An explicit scheduler	11
4	Trust: what exactly is being checked?	12
4.1	Three layers that are not interchangeable	12
4.2	The acceptance judgement	13
4.3	A status lattice: ten outcomes, and what each licenses	13
4.4	Kernel-only and native-checked are different profiles	15
4.5	Two ways to accept a proof the kernel never checked	15
4.6	What a working verification boundary looks like	16
4.7	Diagnostic holes are not accepted proofs	18
4.8	Soundness invariants to assert in code	18
4.9	Adversarial failure modes and their defences	19
4.10	The proof-cost budget	19
4.11	The external certificate protocol	20
5	Structural search: changing the statement that is being proved	20
5.1	Demand-directed theorem use	20
5.2	Demand slicing in a finite Horn fragment	21
5.3	Induction: selecting a scheme and generalising the statement	22
5.4	Invariant synthesis by exact CEGIS	25
5.5	Negative verdicts: when “not found” may become “does not exist”	27
5.6	Witness synthesis: construct terms, not satisfying assignments	30
6	Nonlinear arithmetic: exact certificates, not floating-point proofs	33
6.1	The acceptance language	33
6.2	Search is untrusted; repair, do not round	34
6.3	Building the dictionary	34
6.4	Exact rational quadratic decomposition	35
6.5	Discovered certificates worth recording	37
6.6	A concrete path to general SOS search	37
6.7	Bernstein certificates on rational boxes	38

6.8	Univariate certificates that preserve zeros	40
6.9	Cheap facts for other engines: certified product envelopes	42
6.10	A complete cross-theory proof shape	42
6.11	Domains, casts, and division	42
7	Boolean search, equality exploration, and external engines	42
7.1	A separate lane before a core-engine rewrite	42
7.2	A proof-producing SAT/theory loop	43
7.3	An implemented CDCL(T) with checkable learning	43
7.4	Bounded equational exploration	44
7.5	Higher-order structure and external provers	45
7.6	Solver islands and the limits of theory combination	46
7.7	Candidate generation can be untrusted without being unconstrained	47
8	Lean integration: a conservative bridge before an invasive one	47
8.1	Begin at a stable boundary	47
8.2	The tactic is the interface; it need not be the executor	48
8.3	Three levels of identity	49
8.4	Do not search in <code>Expr</code>	49
8.5	A typed semantic boundary for each worker	50
8.6	Three levels of proof reconstruction	50
8.7	A proposed module layout	51
8.8	A proposed user interface	52
8.9	The deployment pattern	53
8.10	Implementation sequence and acceptance gates	53
8.11	Where the risk actually is	54
9	Executed experiments	54
9.1	Nine runs, reported separately	54
9.2	The runs	55
9.3	Four Horn experiments that must not be merged	60
9.4	Lean status	61
9.5	What the evidence establishes	61
10	The evaluation that would actually decide the question	62
10.1	Freeze the comparison, including its theorem knowledge	62
10.2	Baselines	62
10.3	Two tracks, three input conditions	63
10.4	Corpus design	63
10.5	Metrics	63
10.6	Required ablations	64
10.7	Acceptance gates	64
10.8	A soundness regression suite, run continuously	64
11	Assessment and recommended starting point	65
11.1	What to build first	65
11.2	Where the risk is	65
11.3	What this document has established, and what it has not	65
11.4	The intended end state	66

A	Certificate schemas	66
A.1	The universal rule	66
A.2	Sparse polynomials	66
A.3	Cone and ideal certificates	67
A.4	Bernstein trees	67
A.5	Induction traces	68
A.6	Horn derivations	68
A.7	Lattice traces	68
A.8	SAT and theory proof logs	68
A.9	Invariant records	69
A.10	What the recorded formats actually diverged on	69
A.11	What a production format must add	69
B	Provenance: what came from where	69
B.1	The nine sources	70
B.2	Where each section's material came from	70
B.3	Ideas that survive in exactly one source	72
B.4	Disagreements, and how they were resolved	72
B.5	What the merge changed	73
B.6	Conventions used in this document	74
	References	75

1 The objective: more proof construction, not a bigger timeout

Forge is a *proof-producing automation system with a structural search layer*. It is not a new foundational logic, not a replacement for Lean’s kernel, and not a rewrite of **grind**. It preserves the existing kernel and treats search engines, learned rankings, computer-algebra systems, and numerical solvers as fallible advisers whose output must be checked before it becomes a fact.

The design separates two tasks that are commonly conflated. A *structural planner* chooses a proof shape: introduce a binder, construct a witness, apply a theorem, prove an auxiliary lemma, generalise an accumulator, or use an induction principle. A *local saturation engine* derives consequences within the resulting context. **grind** is an excellent candidate for the second role. The proposed improvement is to supply proof structure and certified facts that local saturation cannot obtain on its own, and to feed useful consequences back rather than restarting every tool from an unchanged goal.

$$\boxed{\text{local consequence search}} \longleftrightarrow \boxed{\text{structural proof search}}$$

Three short examples fix the pattern. A nonlinear worker may prove $0 \leq p(x, y)$, combine it with an existing $p(x, y) \leq 0$, and obtain $p(x, y) = 0$; that equality is then useful to congruence closure even when the final target is an equality between uninterpreted function applications. An induction planner may generalise an accumulator, synthesise its polynomial invariant, and invoke **grind** only on the base and step obligations. A witness worker may construct a parametric solution of a linear system and leave integrality or bound obligations to other workers. In all three cases the benefit comes from *coordination around checked intermediate claims*, not from any single engine becoming stronger.

1.1 Two meanings of “more powerful”

The desired outcome is substantially higher automatic coverage on useful Lean developments: *more goals solved automatically from comparable initial information*, particularly goals that need structural proof steps or new terms. That is not the same as a stronger logic, a change to Lean’s trusted axioms, uniform speed improvement, logical completeness, or success-set inclusion at a fixed total budget. Those are different claims and they should never be argued for interchangeably.

Definition 1.1. For a tactic implementation T , an environment E , a goal G , and a resource allocation B , write $T(E, G; B) \downarrow$ when T returns a proof accepted under the selected trust policy within that allocation.

Forge offers two modes, and an evaluation must report both.

In *compatibility mode* it first runs the stock **grind** invocation, with the same input, options, and solver budget, before running any new machinery. In *fixed-budget mode* it partitions a single budget among workers and may sacrifice some baseline successes in exchange for others.

Proposition 1.2 (Conditional baseline preservation). *Assume the stock run is reproduced deterministically with identical inputs and resources, and that its successful proof is returned immediately. If compatibility mode allocates that full run before spending any additional extension resources, then every success of the stock invocation is also a success of compatibility mode.*

Proof. The first stage is the stock invocation itself. A success exits before any extension can change its state or consume its reserved allocation. $\square$

Proposition 1.2 is a *scheduling* property, not a superiority result at equal cost. Process startup, memory pressure, imports, and global wall-clock limits can invalidate an operational reading of it unless they are controlled. Nor can any finite-budget heuristic promise pointwise dominance over another heuristic at exactly the same cost. A fair experiment reports compatibility mode and fixed-budget mode separately, and never presents the preservation property as an equal-cost performance theorem.

Failure-triggered intervention has recent precedent in research on `grind` itself; the cascade is prior art and is not claimed here as an invention [39].

1.2 Where new coverage can realistically come from

The target is not “everything `grind` cannot prove”. It is a specific collection of mechanisms with a plausible cost/coverage tradeoff:

1. **Proof structure.** Select induction principles, generalise changing state, construct existential witnesses, and establish auxiliary lemmas before using them.
2. **Missing algebraic consequences.** Discover hidden sums of squares, domain-dependent polynomial positivity, and exact zero information that can be shared with other engines.
3. **Directed obligation solving.** Choose theorem applications from the conclusion backward, create their side conditions explicitly, and use successful side conditions to activate guarded forward reasoning.

These are proposed extensions, not assertions that every example requiring one will fail under all existing `grind` configurations or library annotations. Some will already be solved by `grind`, `nlinarith`, `Aesop`, or a suitable library theorem. Only controlled runs on a real corpus can identify genuinely new coverage.

1.3 The deliverable and its evidence levels

This document deliberately keeps four things apart, and the reader is asked to keep them apart too:

1. **Mathematical soundness arguments.** The propositions and theorems below explain why each certificate format implies the statement it claims. These are ordinary mathematics and are independent of any implementation.
2. **Tested Python checking.** Executable prototypes synthesise and re-check certificates in exact rational arithmetic, with negative and mutation tests. This is evidence about programs, not about Lean.
3. **Generated Lean candidate source.** Some workers emit Lean proof scripts. Where a script has not been compiled, it is *candidate source*, and saying so is not a formality.
4. **The proposed production implementation.** The Lean tactic, its reflected checkers, its `grind` adapter, and its measured performance. None of this exists yet.

Remark. A Python check is not a Lean-kernel proof. The prototype checkers are hand-written research code; several share representation and matching routines with the searches they are meant to audit. “Search-free replay” means the numerical and symbolic search libraries are not loaded — it does not mean an independently implemented formal verification.

1.4 What was actually built, and what was not

Across the nine contributing efforts, the following algorithm families were implemented and executed in Python: exact sparse rational polynomial arithmetic; exact quadratic decomposition by symmetric pivoting; finite polynomial-cone search with rational repair; Bernstein box subdivision with checked coverage; square-factor and Sturm certificates for univariate intervals; polynomial recurrence and accumulator-invariant synthesis by exact CEGIS; affine and integer-lattice witness synthesis; demand-directed ground Horn reasoning; a typed equational prover with structural-induction replay; proof-logging CDCL with an integer-difference-logic theory solver; and standard-library-only replay of every stored certificate.

The following were *not* built by anyone: an installed **forge** tactic; a Lean formalisation of any checker; the full obligation scheduler against real Lean snapshots; native **grind** state access; a dependent-type reifier; general quantifier instantiation; higher-order synthesis; external SMT proof reconstruction; a semidefinite backend; and any measured comparison against a Lean tactic.

Section 9 reports what each of the nine runs actually observed. Those runs used different seeds, different generators, and different case sets; their counts are not comparable and must never be pooled into a single accuracy figure.

2 A strong baseline, not a straw man

The Lean reference describes **grind** as cooperating proof-producing engines organised around contradiction, congruence closure, constraint propagation, E-matching, guided case analysis, and theory solvers. That is already a shared-fact architecture, not a sequence of independent tactic calls [1, 2]. A credible successor must reuse those strengths rather than rediscover them.

The baseline must also be *dated*. Lean v4.34.0 was released on 14 September 2026 [8]. Designing against a remembered version of **grind** is the most common way to overstate a contribution. Table 1 records capabilities that the pinned configuration and implementation sources show are already present, and what each implies for this design.

Area	Already present in the inspected source	Consequence for this design
Equality	Congruence reasoning, function-valued congruence, extensionality controls, injectivity support	Do not advertise ordinary e-graphs or function equalities as new.
Arithmetic	Ring and field algebra, linear arithmetic, integer arithmetic, associative/commutative reasoning, order support, model-based theory combination	Add certified inequality reasoning and structure, not a duplicate ring normaliser.
Quantifiers, search	E-matching, term-generation limits, case splits, lookahead	Distinguish goal-directed <i>construction</i> from existing instantiation and splitting.
Bit-vectors	grind => bv_decide encodes relevant equivalence classes and facts learned by theory solvers and E-matching into the SAT problem	Basic cooperation between bit-blasting and grind is existing functionality, not a Forge innovation.
Conditional rewriting	grind discharges side conditions for conditional Sym.simp theorems	The proposal is broader cross-worker obligation scheduling, not the invention of guarded rewriting.
Libraries	Configurable library suggestions and local-definition use	Improve retrieval and obligation-driven use; do not claim retrieval is wholly absent.
Engineering	Saved states, proof-related diagnostics, explicit search limits	Reuse existing machinery behind a versioned adapter.

Table 1: Baseline features from the pinned Lean v4.34.0 configuration and implementation sources [10, 11, 5, 6, 8]. The table is not a completeness claim for any theory.

For reproducible comparison, the pinned configuration source records defaults including nine search-branch splits, five E-matching rounds before a split, generation bounds of eight, and one thousand E-matching instances per branch [10]. Many further limits are exposed. A future benchmark should record the *effective options*, not merely write “default **grind**”, and should include a reasonably tuned baseline as well as the stock one.

2.1 The neighbouring tactics, and the gap between them

grind is the baseline this design builds on, but it is not the only tactic a new capability has to be measured against. Four others bound the space, and naming them is what makes the remaining gap precise rather than rhetorical.

Tactic	What it does	What it does not do
<code>exact?</code> / <code>apply?</code>	Finds an <i>existing</i> library lemma that closes the goal.	Cannot <i>compose</i> a new term. Fails on $(A \rightarrow B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow A \rightarrow C$ unless that exact lemma exists.
<code>tauto</code> / <code>itauto</code>	Decides propositional goals; <code>itauto</code> is intuitionistically complete.	Prop only; needs Mathlib imported; no negative verdicts surfaced as such; nothing for Type .
<code>Aesop</code>	General rule-based proof search.	Heuristic; no non-inhabitation answers. [25]
<code>decide</code> <code>omega</code>	Decidable ground propositions. Linear integer and natural arithmetic.	Nothing polymorphic or data-level. The reference explicitly does not describe it as complete for every integer-linear problem. [19]

The gap this leaves is *higher-order plumbing that has to be constructed rather than retrieved*: currying and uncurrying, projections, composition, distribution laws such as $A \times (B \oplus C) \rightarrow (A \times B) \oplus (A \times C)$, and continuation shuffles such as $((A \rightarrow B) \rightarrow A) \rightarrow (A \rightarrow A)$. These are terms one writes constantly. `exact?` cannot build them, `tauto` covers only the **Prop** shadow of them, and `Aesop` will search for them without ever being able to report that a particular one does not exist.

That last point is the one worth dwelling on, because it is a capability and not merely a diagnostic. §5.5 argues that a tactic which can sometimes say *provably no such term exists* is doing something none of the five above does.

2.2 Related systems: reuse, do not relabel

`Aesop` already provides rule-based AND/OR proof search, distinguishes normalisation from safe and unsafe rules, and uses best-first exploration. Its documented induction examples still perform induction explicitly. It is a source of search infrastructure, not a feature to rediscover [25].

`Duper` supplies proof-producing saturation over supplied facts, with manual premise selection and a configurable portfolio. It is a complementary clause-search worker, especially after a relevance filter has selected a small premise set [26].

Mathlib’s `linarith` separates certificate search from proof reconstruction, and `nlinarith` adds specific nonlinear preprocessing. That is the right architectural precedent, but it is not a general sum-of-squares solver [40]. Foundational checking of numerically discovered SOS certificates has a substantial history; Harrison’s HOL Light work explicitly separates sophisticated search from relatively straightforward verification [56].

There is already a Lean `sos` project implementing proof-producing nonlinear reasoning with rational certificates. Its documented architecture separates reification, search, checking, and verification; named interfaces include `SOS.Engine.solve` and `SOS.Engine.Result.check`, and `sos?` can emit a replayable witness. Its computational polynomial substrate is `Hex.MvPoly` [31, 32, 33, 35]. These are concrete integration targets. The `Forge` contribution is *when* to request such a certificate and *how* to return its proved consequences to structural and saturation workers — not the invention of sum-of-squares certificates.

The historical `lean-egg` project should not become a mandatory new dependency: its repository is marked as no longer developed and directs users towards `grind` [45]. The useful idea is bounded, proof-producing equality search, not allegiance to that package [48, 46, 47].

Remark (A version caveat that matters). The nine contributing efforts observed the Lean `sos` project’s `lean-toolchain` pinning `v4.32.0-rc1`, against a `v4.34.0` baseline [34]. These projects do not automatically form a build-compatible package set on the day a new Lean release appears. Pin the core toolchain, select mutually compatible Mathlib and optional-backend revisions, and

keep their adapters separate. No `lake-manifest.json` is supplied here for a combination that was never built.

2.3 A dependency map with integration decisions

Component	What it provides	Integration decision
Lean core / <code>grind</code>	Shared equality and local-theory reasoning, proof terms, saved states.	Keep as the default local closer; start at the tactic boundary, move to a version-pinned adapter only where measurement justifies it. [1, 12]
Aesop	Best-first AND/OR proof search with indexed rules.	Reuse rule-search ideas and adapters; do not claim AND/OR search is novel. [25]
Mathlib tactics	<code>simp</code> , <code>ring</code> , <code>linarith</code> , <code>nlinarith</code> , <code>positivity</code> , <code>linear_combination</code> , cast normalisation.	Cheap proof reconstruction and preprocessing before expensive search. [43, 40, 42, 44]
Lean <code>sos</code>	Rational nonlinear certificates and a proof-producing tactic.	The production nonlinear backend. Do not replace it with the prototype’s restricted LP dictionary. [31]
Lean <code>lp</code>	Verified linear programming.	Candidate backend for the linear feasibility problems underlying cone and witness search. [36, 37]
Duper	Proof-producing saturation inside Lean.	An optional, separately pinned first-order worker over a small retrieved premise set — never an unchecked external verdict. [26]
<code>lean-auto</code>	Translation and monomorphization infrastructure.	Reuse rather than invent an unverified dependent-to-first-order erasure. [27]
<code>lean-smt</code> / <code>cvc5</code>	SMT search with Lean reconstruction; unsupported steps remain goals.	Optional backend with an explicit residual-obligation interface. [28, 60]
QuerySMT / LeanHammer	Hint-based reconstruction and premise selection.	Prior art for the retrieval and reconstruction lanes. [29, 30]
PBLean	Pseudo-Boolean proof certificates for Lean 4.	Reference point for the Boolean certificate lane. [38]
Z3	Candidate models and quantified search techniques.	Untrusted search assistance; a verdict is not a Lean proof. [61]
<code>egg</code> / CCLemma	Equality saturation and inductive-lemma-search prior art.	Algorithmic references and possible isolated search processes. [48, 50]
Canonical	Dependent type-theory inhabitation and unification.	Integrate by its proof-term interface, not by flattening away the dependencies that make its task meaningful. [54, 55]

2.4 A portfolio alone is not the proposal

Running `first | grind | aesop | nlinarith` is a portfolio. The important difference is that in *Forge* an *unsuccessful* attempt still produces structured, reusable information: which recursive argument changed, which equality would enable progress, which witness template was insufficient,

and which arithmetic side condition remains. That feedback directs the next attempt, and every checked intermediate result returns to the shared proof state.

A benchmark must therefore include a same-workers, same-budget, *non-communicating* portfolio as a baseline. That is the experiment that distinguishes “integration helps” from “we added more solvers” (Section 10).

3 Architecture: a scoped obligation graph around a shared fact store

Status: Specified architecture. The Python prototypes implement several workers and certificate formats; none of them implements this Lean controller.

3.1 Three data structures that must never be confused

A single global collection of “facts” is too ambiguous to be safe. The controller maintains three stores with different admission rules.

The *proved-fact store* contains propositions together with Lean proof expressions valid in a particular local context. Each entry records its source, its assumptions, and the proof dependencies needed to reconstruct it. A worker may consume it as evidence.

The *obligation graph* contains goals waiting for proofs: ordinary goals, side conditions, candidate lemmas, induction cases, witness specifications, and proof-reconstruction holes. An AND node is solved only when all of its required children are solved; an OR node records alternative approaches to the same goal. Failed attempts and exhausted budgets do not become logical facts.

The *search store* contains models, candidate terms, speculative rewrites, usefulness scores, and unsuccessful attempts. These may direct search. They are never accepted as hypotheses by a proof worker. In particular, a conjectural equality may merge nodes in an isolated exploration graph, but it must not merge nodes in the proved equality state.

Invariant 1 (The ledger invariant). Every entry in the proved-fact store carries a proof term and a scope. Nothing enters it that is not accompanied by both. Membership of the proved-fact store is the *only* licence a worker has to use a proposition as a premise.

At the outer layer these stores are organised as an AND/OR search graph over structural choices; at each open leaf a coordination layer exposes certified facts and typed views to specialised workers. **grind** maintains its own efficient internal equality and theory state behind that boundary. The acceptance layer reconstructs a Lean proof of each returned proposition, checks scope and dependencies, and ultimately constructs a proof of the original goal. Numerical success, satisfiability status, an unverified certificate, and a plausible conjecture are never accepted as facts.

3.2 The demand protocol and the worker contract

Workers are addressed through an explicit vocabulary of demands rather than being invoked as opaque tactics. The following is a proposed interface, not a transcription of a released Lean API.

```
Demand :=
| Prove          (goal)
| ProveGuard     (context, proposition)
| FindWitness    (context, binders, specification, grammar)
| Instantiate     (context, theorem, desired_consequence)
| Generalize     (context, recursive_call, blocked_goal)
| FindLemma      (context, left_frontier, right_frontier)
| ProveInduction (context, motive, recursor_candidates)
| Reconstruct    (context, source_goal, external_certificate)

Worker.supports : GoalView -> SupportReport
Worker.step      : Snapshot -> DeltaFacts -> Budget -> WorkerResult

WorkerResult :=
```

```

| Proved      (lean_proof : CheckedProofRef)
              (dependencies) (replay_record)
| Progress    (facts       : Array ScopedFact)
              (obligations : Array ConditionalObligation)
              (continuation : OpaqueToken)
| Candidate   (kind : CertificateKind) (payload : BoundedData)
              (proposed_next_demands)
| Unknown     (reason : FailureReason) (diagnostics) (continuation)

ScopedFact := {
  proposition : Expr,
  proof       : Expr,
  scope       : ScopeId,
  dependencies : Array FactId
}

```

`CheckedProofRef` is a capability issued by the acceptance layer, never by an external process. An external solver can return only data or a candidate. A native Lean tactic worker may return a proof expression, but the coordinator still verifies its type, its local-variable support, its auxiliary declarations, and its allowed axioms before committing it.

A worker returning `Progress` must not pretend that its residual goals are solved. The controller may tentatively construct a parent proof containing metavariables, but overall success is reported only after those metavariables are assigned and the completed expression is checked.

A *conditional obligation* is a directed dependency, not an assertion. Cancelling a in a field creates the obligation $a \neq 0$; until that proof exists, the cancelled equality cannot enter the fact store. The result protocol must keep a conjectured fact visibly different from a proved fact even when the two share the same proposition.

A counterexample belongs in `Candidate` unless its terms, its hypotheses, and the negation of the source conclusion have all been validated in Lean. For an abstraction-based translation, a satisfying assignment may be no more than a model of an overapproximation.

3.3 Three outcomes, not an overloaded Boolean

The tactic’s result type should distinguish *proved*, *refuted with a checked countermodel*, and *unknown*, and the diagnostic attached to `UNKNOWN` should name the first substantive blocked obligation or the exhausted budget. Collapsing these into success/failure destroys exactly the information the next attempt needs, and it invites the most common category error in this area: reading a failed bounded search as a disproof.

Remark. Throughout this document, a search failure is `UNKNOWN`. It is never evidence that the requested theorem is false.

3.4 Snapshots, scope keys, and proof transport

A proposed `GoalSnapshot` contains the environment revision, the local context, local instances, the metavariable context, the target expression, the transparency configuration, branch assumptions, the normalisation version, and worker state. The controller must restore *all* of it on backtracking. Restoring only the visible list of hypotheses is insufficient: a worker may have instantiated metavariables or created local declarations whose identities are used by other expressions.

A persistent fact key contains the expression, its type, universe levels where relevant, the identity of the ordered ring or field instance, a local-context fingerprint, and a scope identifier. Printed text is not a safe key — names printed as x can denote different free variables in sibling goals — and expression pointers are not suitable for caches that outlive an environment. Implicit typeclass arguments matter: two syntactically similar applications of an overloaded operation can mean different things because their instances differ.

Typed views must therefore preserve source semantics, and the traps are concrete:

- $x - y$ over $\mathbb{N}$ is truncated subtraction; over $\mathbb{Z}$ or $\mathbb{R}$ it is ring subtraction. A translator that treats these identically is unsound.
- A `Fin n` expression is not automatically an integer with unrestricted arithmetic.
- A proposition about real division cannot be transported through a denominator without a nonzero or sign guard. Clearing denominators in an inequality requires a sign proof. These become **ProveGuard** obligations; if the guard fails, the rewrite is unavailable, not approximately valid.
- An integer embedding into a field can be injective; a fixed-width bit-vector projection is not. Mixing those relationships inside a generic “numeric cast” cache would be a serious error.

Facts proved using a branch hypothesis can be shared only within that branch or a descendant. To move one upward, discharge the hypothesis and transport an implication, or combine proofs from a complete case split. This applies as strongly to numerical bounds, inferred equalities, and cached certificates as it does to ordinary theorem applications. There is no “most likely branch” rule in the logical layer.

A subtler issue is proof transport. A successful worker may introduce auxiliary theorem declarations and return a proof referring to them; restoring the environment while retaining only the final expression would leave dangling references. A commit must preserve the required declaration closure, after checking it, or reconstruct a self-contained proof term in the parent environment. One safe reuse method abstracts a proof over its exact free-variable dependencies and then instantiates that closed theorem in the destination context. External results must be re-elaborated against the exact destination context rather than transporting free-variable identifiers between unrelated goals.

3.5 Facts carry explanations, not only truth flags

A nonlinear worker may prove $0 < c$ from polynomial constraints. That proof justifies $c \neq 0$, which unlocks a division rewrite; the rewrite’s proof yields an equality; the equality may enable congruence closure; and that may make an E-matching instance useful. The chain should be represented as a proof DAG with explicit edges, not rediscovered by reparsing tactic text.

This is why an arithmetic worker returns individual proved consequences and their assumptions rather than a Boolean “solved” flag. A checked sign or equality is useful well beyond the goal that prompted it.

3.6 Event-driven cooperation

Workers subscribe to deltas rather than repeatedly scanning the entire context. Typical events are a new bound on an arithmetic expression, a new equality joining two term classes, a proved side condition, an instantiated theorem, and a changed target after structural decomposition.

The coordinator normalises a returned fact, checks whether it adds information, stores its proof, and notifies only relevant subscribers. Changing an interval bound invalidates or strengthens a Bernstein search problem; learning a polynomial equality may reduce the dimension of a subsequent positivity certificate; proving an existential witness’s denominator nonzero activates its algebraic replay.

Logical circularity and scheduling cycles must be distinguished. A worker waiting for a fact that another worker might establish is an ordinary dependency cycle; it may call for a different search action. It does not permit both workers to assume their own outputs. Track these dependencies as strongly connected components and either pursue an independent obligation, perform a justified case split, or return a diagnostic explaining the blockage.

3.7 An explicit scheduler

The initial implementation should use deterministic, cost-aware scheduling and must not require a trained model. Each task has a relevance estimate, an expected information gain, a recent cost,

an age, and a resource ceiling. A simple score is

$$s(a) = w_r R(a) + w_g G(a) + w_a A(a) - w_c \hat{C}(a),$$

with the weights fixed in the experiment manifest. This is a heuristic, and the important part is not its shape but two disciplines around it.

First, *reserve a minimum quota for every eligible worker class*, so that a high-scoring stream of cheap actions cannot starve structural search. A cheap task that regenerates itself is the normal failure mode here.

Second, *the estimated benefit of an action must include the cost of the certificate it will produce*, not only the probability that it closes the goal. A worker that reliably succeeds while emitting a proof too large to elaborate is not a good scheduling choice, and a scheduler that scores only success probability will keep choosing it.

```
solve(goal, mode, budgets):
  if mode == compatibility:
    run_exact_stock_grind(goal, budgets.stock)
    if accepted: return proof
    restore_original_goal()
  agenda := initial_structural_and_local_tasks(goal)
  while resources_remain:
    action := choose_with_reserved_worker_quotas(agenda)
    snapshot := save_full_branch_state()
    result := run_bounded(action)
    restore_or_commit_only_checked_changes(snapshot, result)
    if original_goal_has_checked_proof(): return proof
    enqueue_new_obligations_and_relevant_delta_subscribers()
  return Unknown(with_structured_diagnostics)
```

Two accounting rules protect the integrity of what those budgets report, and both are easy to violate by accident.

Deduplication never refunds. A raw proposal consumes its allowance *before* alpha-equivalence deduplication, not after. Otherwise a search that generated a million alpha-variants of one term reports having done twelve units of work, and every downstream resource claim is fiction. Deduplication is a presentation improvement; it is not a discount.

A dropped branch stays visible. When a branch is discarded for a *resource* reason — a queue bound, a depth cap, an exhausted identifier supply — that fact must remain structurally distinguishable from a branch that genuinely closed, all the way up to the top-level result. A search whose truncations are silently absorbed by its own combinators cannot tell **exhausted** from **truncated** (§5.5), and therefore cannot support a negative verdict at all.

Heartbeats alone are insufficient for external numerical code. Maintain separate limits on wall time, memory, instantiated theorems, polynomial monomials, rational bit lengths, certificate size, induction depth, and replay work. External workers should run in bounded child processes without network access. A process timeout returns UNKNOWN and cannot create a theorem.

Finite budgets imply incompleteness. Fair worker quotas do not make unrestricted Lean proof search complete; they make the search policy interpretable and reduce avoidable starvation. A deterministic mode, in which the schedule is a function of the goal and the manifest alone, should be available for reproducible experiments even if an interactive mode is allowed to be adaptive.

4 Trust: what exactly is being checked?

4.1 Three layers that are not interchangeable

It is essential to keep apart the mathematical soundness of a certificate format, the software correctness of a checker, and a proof accepted by Lean. The propositions in this document explain why each certificate implies the statement it claims. Tests provide evidence about the Python

implementations. Neither supplies a Lean theorem. A completed tactic must either reconstruct a Lean proof or use a formally verified checker whose result is established inside Lean.

Proposition 4.1 (Certificate-format soundness is not implementation correctness). *Let $\mathcal{C}$ be a certificate format with a proved soundness statement “ $\text{check}(P, C) = \text{true} \Rightarrow P \text{ holds}$ ”, and let K be a program intended to compute check . Evidence that K accepts only certificates satisfying check is evidence about K , obtained by testing. It does not follow from the soundness statement, and the soundness statement does not follow from it.*

This is laboured deliberately, because the gap is where an automation project most easily deceives itself. The prototypes here replay every stored certificate with the standard library alone, and several run their checkers in a direction algorithmically opposite to their producers — a Bernstein checker that *expands* a claimed basis representation and compares it against an independently computed affine transform shares no conversion formula with the search that proposed it. That is real, and it is still not kernel verification.

4.2 The acceptance judgement

An accepted source theorem must have:

1. no unresolved expression or universe metavariables;
2. no `sorry` dependency anywhere in its transitive closure;
3. no unexpected new axioms relative to the approved environment;
4. every guard and side condition discharged, not merely recorded;
5. a proof of *the exact original goal*, not of a restated one.

The last point deserves emphasis because it is the one a certificate pipeline gets wrong most easily.

Invariant 2 (Certificate binding). A certificate is checked against the problem derived from the *source* — the same variable mapping, types, assumptions, and target. The checker receives the original target and hypotheses externally. It must never accept a problem statement supplied by the certificate itself. A certificate for another polynomial, another box, or a different quantifier scope is not interchangeable with the one required.

A correct polynomial identity about the wrong reification is not a proof of the user’s goal. A certificate must identify the actual hypotheses it used, not merely provide polynomial data that resembles them. The prototypes test this directly: removing a necessary constraint from the problem makes replay fail, and a certificate re-attached to a changed target is rejected.

Remark (A type is not a check). An `Expr` field is not a proof-validation boundary. Calling a structure `CheckedProof` does not make its contents checked, and a reported dependency list is a diagnostic: the acceptance layer must recompute dependencies from the accepted term. The Lean design contract in this collection deliberately names a submitted term `Candidate` for exactly this reason. Likewise, a successful `inferType` or unification inside tactic metaprogramming is part of elaboration, not a replacement for the final kernel boundary — an expression can type-check because an imported axiom states the desired result.

4.3 A status lattice: ten outcomes, and what each licenses

A binary split between “certificate accepted” and “no certificate” under-specifies this system badly. Every real integration lives in the middle, and the middle is where a design silently upgrades a heuristic into evidence. The remedy is to enumerate the outcomes and attach to each one the consequence it *permits* — so that permission is a property of the outcome rather than a decision the consuming code makes locally.

Outcome	Typical producer	Permitted consequence
Preparation refused	reifier, fragment classifier	Retain the goal with the refusal class. Never a negative result about the goal.
Unassessed	budget, policy, inapplicable domain	Retain. Carries no information about the goal at all.
Heuristic status	raw solver sat / unsat / unknown ; a learned score	Guide search and ranking. <i>Never</i> evidence.
Candidate proposed	any untrusted search	Enter the obligation graph as a candidate. Never the fact store.
Countermodel observed	solver model, sampled point	None yet. Decode and replay it before it means anything.
Countermodel replayed	independent evaluator on the sealed problem	Refute the <i>modelled</i> statement for <i>this</i> candidate, subject to the recorded assumptions and the model’s scope.
Validated within bounds	exhaustive finite evaluation	A bounded positive receipt. May rank, and may discharge an explicitly bounded claim. Does <i>not</i> assert an unbounded theorem.
Certificate checked	exact rational replay	The stated mathematical claim holds, given the stated hypotheses. Still Python, not Lean.
Kernel-accepted	synchronous kernel check, options pinned	Proof-grade for the exact original goal in the active environment, under the declared axiom policy.
Unknown / timeout / resource limit	any worker	Preserve the candidate. Never implicitly success, and never implicitly failure.
Internal inconsistency	any invariant violation	Fail the stage <i>closed</i> , invalidate the affected caches and sessions, and preserve the prior result.

Three rules govern movement through this lattice.

Invariant 3 (Evidence is attached to a tuple, not a name). A claim is attached to the exact tuple of domain, inventory, encoding, candidate, hypotheses and policy that produced it. No cache key, diagnostic name, rendered term, hash, model, or solver session may stand in for that tuple unless the substitution has itself been checked.

Invariant 4 (Guidance and authority are separate). Fingerprints, sample signatures, solver scores, learned clauses and unsat cores may direct enumeration without becoming evidence. Evidence requires the independent validation path; a proof-grade claim additionally requires the declared proof authority.

Invariant 5 (Pruning states its approximation polarity). An *over*-approximation of the remaining search space may justify pruning — when even the over-approximation is unsatisfiable, nothing was lost. An *under*-approximation may produce witnesses but may *never* justify pruning. An exact encoding may do either. Any translation that drops a distinction the domain cares about must degrade to *inapplicable* or *unknown*: it must not silently strengthen a precondition, weaken a postcondition, replace a total function by a partial one, equate distinct nominal families, erase overflow, or assume termination.

Invariant 5 is the safety argument for any learned or numerical pruning this design might later acquire, and it is the one most easily lost when a heuristic is described as “just a filter”.

Remark (Caches and fingerprints). Two corollaries of Invariant 3 are worth stating because they are cheap to get wrong. A reusable artefact should be reduced to the minimal *re-checkable input*,

and every reuse must re-run the check — a store holding inputs is a search heuristic and is safe, whereas a store holding verdicts is a cache and is not. And a fingerprint that keys a cached result must include the solver version, invocation profile, parameters, encoding version and objective order; a fingerprint over the problem alone is unsound across a solver upgrade.

4.4 Kernel-only and native-checked are different profiles

The Lean kernel verifies derivations relative to the axioms they use, so a strict automation policy must inspect the transitive axiom dependencies of the final proof and of its auxiliary declarations. A practical whitelist may allow the project’s usual logical axioms — propositional extensionality, quotient soundness, and classical choice where permitted — while rejecting admitted proofs and compiler-result axioms.

The current tactic reference documents that `native_decide` and `bv_decide` rely on native code generation and associated trusted results [19, 20]. Consequently “the SAT solver returns a checked certificate” does not by itself mean “no compiler trust is added”. Forge should place that route behind an opt-in trust mode rather than enabling it under a kernel-only label, and the active profile must be visible in the trace and in the output metadata.

Two further details matter for anyone implementing the audit. Native evaluation in recent Lean versions can introduce a *separate generated axiom per computation*, so a single-name blacklist such as “reject `Lean.trustCompiler`” is not a sufficient policy [21, 22]. And the audit must follow the actual pinned Lean version rather than a remembered one; the set of axioms a given tactic can introduce is a property of the release.

A kernel-only bit-vector route would require replaying a supported certificate checker through a reduction mechanism that introduces no native-result axioms, and measuring the resulting cost. That route is proposed here, not implemented. Lean4Lean is the relevant reference point for what independent kernel-level assurance would mean [24].

4.5 Two ways to accept a proof the kernel never checked

The acceptance judgement above is not self-executing. In the pinned Lean v4.34.0 source there are two concrete paths by which an implementation that believes it is kernel-checking is not. Both were verified directly against the installed toolchain rather than inferred, and neither appears in any of the nine source proposals.

An inherited option can switch the kernel off. `Lean.Kernel.Environment.addDecl` branches on a builtin option:

```
def Kernel.Environment.addDecl (env : Environment) (opts : Options)
  (decl : Declaration) (cancelTk? : Option IO.CancelToken := none)
  : Except Exception Environment :=
  if debug.skipKernelTC.get opts then
    addDeclWithoutChecking env decl
  else
    addDeclCore env ... decl cancelTk?
```

The option’s own registration carries the warning: “*skip kernel type checker. WARNING: setting this option to true may compromise soundness because your proofs will not be checked by the Lean kernel.*” Its default is `false`, but the value used is whichever `Options` the caller passes — so a verifier that inherits ambient options inherits whatever the surrounding file, project, or an enclosing tactic set. A certificate pipeline must *pin the option off* in the options it passes, not merely refrain from setting it.

This is not hypothetical for the Boolean lane of §7. The v4.34.0 documentation for `bv_decide` lists the same option as a supported knob: it “may be set to true to disable actually checking the LRAT proof”, leaving a run that “still runs bitblasting + SAT solving, so this option essentially

trusts the SAT solver”. A benchmark that did not pin the option would be comparing two different trust profiles while reporting one.

The ordinary declaration path is asynchronous. `Lean.addDecl` in `CoreM` goes through `env.addConstAsync`, commits *preliminary* constant information immediately, and — when `Elab.async` is set — defers the actual check to a task:

```
let async <- env.addConstAsync (reportExts := false) name kind ...
async.commitConst async.asyncEnv (some info) ...      -- reported immediately
...
if Elab.async.get (<- getOptions) then
  let t <- BaseIO.mapTask checkAct env.checked        -- checked later
  Core.logSnapshotTask { ... task := t, ... }
```

So the declaration becomes visible before the kernel has accepted it. An acceptance step that adds a declaration and then observes that it is present has observed nothing. The acceptance decision must either call the synchronous `Kernel.Environment.addDecl` directly, or await the environment’s checked task before answering.

Invariant 6 (Synchronous, option-pinned checking). Acceptance calls the synchronous kernel entry point with `debug.skipKernelTC` explicitly set to `false` in the options it passes, and does not report success on the basis of a declaration’s visibility.

A stamp that cannot be forged. The corresponding discipline in the type is that the evidence of checking must be constructible only by the checker. A suitable shape is an opaque stamp returned by the verifier, binding the candidate, the expected type, the environment and toolchain identity, and the options actually used. A fingerprint may be a *view* of that association; it must not be a constructor for it. A `Bool` field named `kernelChecked` that any caller can set to `true` is the same mistake as the `CheckedProof` structure of the previous remark, one layer down.

A workable verification pipeline, then, is: instantiate all assigned metavariables and reject any remaining expression or universe metavariables; check that every free variable belongs to the captured local context and that no local declaration has escaped its scope; abstract the local context into a closed type and value, preserving binder information and dependency order; build a fresh private declaration; call the synchronous kernel checker on a scratch environment with the option pinned; discard that environment unless the result is committed; and delaborate independently for display, optionally re-elaborating the displayed text to establish that what the user is shown is what was checked.

4.6 What a working verification boundary looks like

The preceding subsections say what acceptance must establish. None of the nine proposals built it. A neighbouring project has, for term synthesis rather than for certificates, and its shape is worth recording because several of its decisions are not the obvious ones.

Re-elaborate the original goal text, every time. The candidate is checked by sending Lean a program of the form

```
noncomputable example : (GOAL) := (TERM)
```

where `GOAL` is the user’s original text, parenthesised, re-parsed and re-elaborated on every single candidate. There is no “reuse the already elaborated goal expression” shortcut. That is crude, and it is exactly the invariant of §4: a proof of the *exact original goal* rather than of a restatement, made unforgeable by never holding on to the restatement.

Two modifiers carry weight. `example` rather than `def` makes the check *ephemeral*: it adds no declaration, so neither success nor failure mutates the session. `noncomputable` disables executable-code generation without weakening elaboration, universe checking, safety checking, or the kernel type check — so a candidate is not rejected for a compilation failure that is not a type error.

One modifier is a mistake worth not copying: `autoImplicit` must be *off* at a certificate boundary. With it on, an unbound identifier in the goal — a typo, or a name that has fallen out of scope — is silently generalised into an implicit parameter, so the candidate is verified against a generalised goal rather than the one that was asked.

Committed environment as a positive requirement. The natural formulation of acceptance is “no errors and no `sorry`”. That is not enough, and the gap is easy to miss: a response carrying no error diagnostics, no `sorry` obligations, and *no environment* means the elaborator never committed a checked command at all. The reference implementation checks for a committed environment on three of its four verification paths and omits it on the fourth — which is precisely the “looks well-typed” hole this section warns about, surviving in production code that is otherwise careful.

Invariant 7 (Positive evidence of checking). Acceptance requires a positive signal that the checker ran and committed — not merely the absence of complaints.

A closed rejection vocabulary with a documented precedence. Failure classification should be an enumeration, not a string, and its order should be fixed: transport failure, then a protocol-level fatal response, then an error diagnostic, then a remaining `sorry`. Two consequences follow. Transport failure is *not* a semantic failure — Lean rejecting a term is a healthy outcome that belongs to the semantic classifier, while a dead subprocess belongs to the liveness layer, and conflating them is how a timeout becomes a refutation. And infrastructure degradation must *void* results computed under it: if the channel failed at any point during a batch, a successful-looking result from that batch is converted to a failure when the scope closes, even if no individual step noticed.

The `sorry` check is structural, not textual. A protocol that reports outstanding proof obligations as data lets the checker reject on that data rather than by grepping the source, which catches `sorry` introduced by a tactic or by elaboration error recovery, not only `sorry` the user typed.

Two tiers, because the fast gate cannot afford the slow check. The in-session gate — errors, obligations, committed environment — costs on the order of a hundred to a few hundred milliseconds per candidate. A transitive axiom audit is far more expensive and does not belong in that loop. The workable split is a *fast* per-candidate gate in the session, and a *slow* out-of-session gate that replays each accepted result in a fresh process, in an environment containing only what that result needs, and requires an empty axiom inventory. The fast gate filters; the slow gate is what an assurance claim rests on.

That cost figure has an architectural consequence, not merely an engineering one. If verification dominates, then verifying lazily — best candidate first, stopping at a quota, never forcing the tail — is the design, not an optimisation. It is also why the scheduler of §3 needs a verification budget distinct from its search budget: each failed reconstruction costs real time, and a worker that proposes many cheap candidates can exhaust the checker while a worker that proposes one expensive one never runs.

Make the receipt unforgeable in the type system. §4 observed that naming a structure `CheckedProof` does not make its contents checked. The constructive version of that remark is an

abstract type whose constructor is not exported, produced only by the function that actually calls the checker:

```
-- Opaque receipt that the verification callback accepted a candidate.
-- This records only callback acceptance at this boundary. It is NOT
-- behavioural evidence, a solver certificate, or a kernel proof. A caller
-- needing a stronger claim must retain and replay that authority separately.
opaque Verified (candidate : Type) : Type
```

Three properties make this an enforced boundary rather than a convention: the constructor is module-private, so nothing downstream can mint one; the type parameter is nominal, so no coercion can forge one across representationally equal types; and the documentation deliberately *under-claims*, fixing a ceiling that everything downstream must respect.

Ranking is a capability too. An overlooked hole: the boundary usually governs *admission* and leaves *presentation* ungoverned. But a ranker that may reorder or filter a set of checked results can drop one, duplicate one, or — worse — reassociate a result with another’s evidence. The fix is to hand the ranker opaque, epoch-scoped handles and validate that what comes back is a total, injective map into what went out, checking collection limits, length, index range and duplication, and *reconstructing the output from the original receipts at the proposed indices* rather than from anything the ranker returned. Any seal failure falls back atomically to the original batch in its original order.

The same discipline answers a question §8.4 raises: identity for selection purposes must be a private index, not payload equality, because equal payloads are still different occurrences and only one of them may carry the authority.

4.7 Diagnostic holes are not accepted proofs

The inspected `grind` configuration includes an option concerning admitted branches in trace-oriented diagnostics [10]. That is not a claim that ordinary successful `grind` proofs are unsound. It is a reminder that a coordinator must never mistake a diagnostic artefact for a finished proof: the root declaration’s dependencies must be checked for admitted terms even when an inner tool reports a successful diagnostic operation.

Absence of the token `sorry` in a source file is likewise not evidence that the file compiles, and it is not evidence that the declarations it contains are proved. A benchmark harness should detect an admitted declaration in a tactic’s output and record it as a rejection rather than a success.

4.8 Soundness invariants to assert in code

Invariant 8 (Context validity). Every proved fact is accompanied by a proof whose free variables and local instances belong to its recorded context, or by an explicit abstraction and instantiation that transports it into the consuming context.

Invariant 9 (Candidate quarantine). No candidate theorem, model equation, numerical conjecture, or unsolved proof hole is admitted to the proved-fact state. Exploratory assumptions never serve as proof evidence.

Invariant 10 (Induction discipline). An induction hypothesis is used only with the scope and smaller-argument relationship supplied by its recursor. Generalised parameters may vary; the structural induction index may not be replaced arbitrarily.

Invariant 11 (Fail-closed arithmetic). Numerical tolerances may guide search. Accepted equalities, coefficient signs, partition boundaries, and proof-rule premises are checked exactly.

Invariant 12 (Acyclic justification). A lemma is not available while its own certificate is being checked, and a bundle of lemmas is accepted only in a dependency order in which no entry refers to a later one. Name shadowing of a defining equation or of the induction hypothesis is rejected outright.

Invariant 2 completes the list. Each of these has an executable analogue in at least one prototype, and the corresponding negative tests are part of the recorded evidence.

4.9 Adversarial failure modes and their defences

The most dangerous implementation errors are not weak heuristics. They are mistakes that quietly turn search evidence into a stronger assumption.

Failure mode	Required defence
Self-justifying lemma	Prove candidates in a separate obligation; forbid dependency cycles unless a joint induction principle explicitly justifies them.
Induction hypothesis applied to the wrong argument	Keep the recursion index rigid; let only genuinely generalised binders be instantiated, and record the permitted instantiation set as checked data.
Branch fact leakage	Record split assumptions and scope; combine branches with a proof, never with a global cache insertion.
Floating-point near-equality	Recompute exact rational identities. A small residual is a rejection, not a proof.
Numerical tolerance exploited in a weight	Require the weight to be an exact rational <i>of the right type</i> before comparison; a weight of $(10^{20} + 1)/10^{20}$ must not pass as 1.
Wrong root problem	Bind the certificate to the original reified expression and assumptions, never to an oracle-provided replacement (Invariant 2).
Missing subdivision region	Derive child boxes from a full split tree and check both children; reject a split point at an interval endpoint.
Duplicate keys in a decoded certificate	Reject duplicate JSON object keys at parse time; two parsers disagreeing about which value wins is a real differential hazard.
Vacuous replay success	Fail when a required certificate family is absent from the results directory, and mutate each decoded certificate to confirm the checker actually rejects it.
Erased dependent type or cast	Preserve typed denotation proofs and explicit transport obligations.
Numerical field proof used for integer division	Use a source-semantics bridge, or reject the fragment.
Opaque model element used as a witness	Reify an actual source term with the correct scope; otherwise reject the proposal.
Incomplete external reconstruction	Keep every hole as an unsolved obligation. No success until all are checked.
Stale state after metavariable assignment	Invalidate affected caches, or restore a consistent snapshot containing the old identities.

4.10 The proof-cost budget

Search can produce a valid but impractically large certificate, and proof size is a resource distinct from search time. Track certificate term count, polynomial support size, rewrite-trace length, proof-DAG size, final proof-term size, replay time, and axiom-audit time.

Share repeated polynomial normalisations and repeated nonnegativity proofs. Emit a proof DAG topologically as local lemmas rather than re-expanding shared subterms. Prefer sparse certificates and small rational coefficients when several valid certificates are available — but never alter a certificate approximately in order to reduce its size, and never delete a dependency because

it looks unused in a heuristic trace while remaining inside the actual proof term. Minimise a replay script only by rerunning and rechecking the minimised version.

When a proof is rejected for size or replay budget, return a useful UNKNOWN diagnostic and preserve the candidate certificate as a debugging artefact. Do not accept a partial proof, and do not silently raise the trust level in order to finish the goal.

4.11 The external certificate protocol

Exchange structured data, never executable code. A problem request carries a schema version, registered variable identifiers, exact rational coefficients, normalised relation kinds, input bounds, and a resource envelope. A returned certificate may reference only registered expressions and hypotheses; the Lean side reconstructs the original semantics from its own registry.

The parser must reject malformed dimensions, duplicate or non-canonical monomials where canonical form is required, negative exponents, oversized integers, unknown references, invalid tree splits, and unsupported certificate variants. It must not use `eval`, run a returned script, load arbitrary shared libraries, or trust a theorem name supplied by the external solver. Limits should be enforced *before* expensive polynomial expansion as well as after it, and search processes should be isolated from the proof-checking process.

The packaged decoders demonstrate bounded rational parsing with numerator and denominator bit-length caps, arity, term-count, total-degree, tree-depth and file-size limits, and duplicate-key rejection. They are research tooling, not hardened services for hostile uploads. A denial of service is not a soundness failure, but it still makes a tactic unusable.

5 Structural search: changing the statement that is being proved

This is the capability that most clearly exceeds local saturation. A saturation engine is very effective when useful ground terms and useful lemmas are already present. A much wider class of goals requires *making one of those things exist*.

5.1 Demand-directed theorem use

Trigger-based instantiation works forward, from terms present in the equality state. The documented E-matching machinery is a baseline, not something to discard [6]. `Forge` adds a complementary backward direction.

Maintain an index over the *conclusions* of approved theorems — initially a discrimination tree keyed by the target’s head symbol and normalised type structure. An entry records the theorem constant, its universe and type parameters, its conclusion head, a discrimination key, the shapes of its premises, and a cheap estimate of elaboration cost. Retrieve a bounded candidate list, instantiate universes, and unify the theorem conclusion with the target. Successful unification creates premises as AND obligations and instantiation choices as OR alternatives, with explicit dependencies so that the conclusion cannot become available before its premises are proved.

For a theorem of shape $\forall x, P(x) \rightarrow Q(f(x))$ and a goal $Q(f(u))$, backward application immediately proposes the obligation $P(u)$ and introduces useful terms even when no suitable trigger was present in the initial forward state. For a goal $Q(t)$ with arbitrary t it does *not* justify inventing an inverse of f : the system must unify t with an application already known to be equal, synthesise a candidate u and prove $f(u) = t$, or decline.

Remark. This is not a claim that E-matching could never reproduce the same argument with different patterns and annotations. What a conclusion index removes is the need for a human to have chosen those patterns in advance. Conclusion-selected backward patterns already exist in `grind` in some form [2]; the proposed addition is that a retrieved theorem’s unproved premises become persistent obligations that other workers can be scheduled against.

Premise retrieval should combine the target with currently *blocked* obligations, not just a bag of symbols from the original goal. A theorem resolving a denominator guard can be more useful than one mentioning the final function symbol. Start with local hypotheses, the equations of

mentioned definitions, relevant extensionality lemmas, and a small retrieved set; expand only if those fail.

Solving for the missing term

Backward matching does not determine every parameter, and the honest responses to that are limited. In $\forall x y, R(x, y) \rightarrow P(x)$, a demand for $P(a)$ leaves y unknown. Forge must not silently choose an arbitrary inhabitant. It can enumerate appropriately typed terms from the local context, ask a witness engine, exploit equalities, or leave the application suspended. Every selected instantiation is checked by Lean, and typeclass synthesis and inhabitation obligations are scheduled explicitly rather than assuming an arbitrary type has an element.

A sharper technique applies when matching stalls at an *interpreted* subterm. Rather than failing, emit an equality constraint between that subterm and the corresponding source expression, and solve for the missing binder.

```
matchDemand(theorem, goal):
  freshen theorem parameters in a child metavariable context
  unify rigid heads and directly matching arguments
  collect the remaining argument equations
  dispatch recognized affine equations to witness synthesis
  enumerate small typed terms only for the still-free parameters
  apply the theorem with a complete candidate assignment
  retain every generated side condition as an obligation
  accept only after all obligations and the final proof replay
```

For hypotheses $\forall k \in \mathbb{Z}, P(2k + 1)$ and $n \bmod 2 = 1$ with goal $P(n)$, the controller synthesises $k = n/2$ from the arithmetic shape and proves $2(n/2) + 1 = n$ using the oddness assumption, then transports along that checked equality. For a conclusion containing $f(t + 1)$ against a goal involving $f(s)$, the candidate $t = s - 1$ comes from solving $t + 1 = s$ over $\mathbb{Z}$ — and over $\mathbb{N}$ the same surface subtraction has different semantics and requires a guard.

Two disciplines make this safe. An arithmetic identity used to match arguments must appear in the proof as rewriting or transport; it cannot be smuggled into the unifier as an unverified definitional equality. And several triggers or repeated binders produce a *joint* constraint system: solving each occurrence independently loses the equality relating two occurrences of the same variable. Cache instances by theorem identity, universe and type instantiation, the complete substitution, and the valid branch context. A proof of $P(x)$ under $h : x = 0$ is not a proof of the same displayed goal under $h : x = 1$.

5.2 Demand slicing in a finite Horn fragment

Status: Implemented and tested in Python by four independent efforts, in both propositional and first-order ground variants.

The prototypes isolate the mechanism in a fragment small enough to prove things about: finite ground definite Horn programs. Backward relevance closure selects the rules that could contribute to the target; forward chaining then runs only on that slice.

Proposition 5.1 (Slicing preserves derivability). *For a finite ground definite Horn program, restricting forward chaining to the backward conclusion-dependency closure of a target preserves whether that target belongs to the least model.*

Proof. A derivation of the target is a finite tree of rule applications. Starting from its root, every rule used and every premise of every such rule is included by the backward closure, so every derivation of the target is retained. Conversely every rule of the restricted program belongs to the original program, so any restricted derivation is valid in the original. Forward fixed-point evaluation computes derivability in both finite programs. Cycles without a supporting fact create no derivation in either. $\square$

Termination and non-circularity are handled by *tabling*, not by a depth-first convention that treats “currently being visited” as success. Each rule instance carries a counter of unresolved premises; input facts initialise the agenda; proving a premise decrements the counters of its dependents; a rule fires only when every premise is proved, and the proof node is stored at that moment. Rules $P \rightarrow Q$ and $Q \rightarrow P$ with no base fact prove neither.

Proposition 5.2 (Relative completeness of the finite core). *For a fixed finite set of ground candidate rule instances, if the dependency agenda is exhausted without a resource cutoff, every fact in their least forward closure is eventually proved. In particular, a backward-relevant closure containing a ground derivation of the target will discover a proof.*

Proof. Induct on the height of a finite derivation. Height-zero input facts enter the agenda initially. Once all smaller-height premises of an instance have entered, its unresolved counter reaches zero and its conclusion is enqueued. Tabling ensures that repeated discoveries do not obstruct propagation. $\square$

The certificate is a topologically ordered DAG: every premise index is strictly smaller than its conclusion’s index; the substitution domain must be exactly the head’s variables; and the head instantiation and premise alignment are checked. A dependency cycle in the *search* graph may be legitimate, but the final proof certificate cannot justify itself through that cycle.

Two honesty constraints govern how this is measured, and Section 9 keeps four separate Horn experiments apart for exactly this reason. First, the cost of *building* the conclusion index over all input rules is part of the demand-directed method’s cost; none of the prototypes obtains a sublinear end-to-end runtime merely by activating fewer rules. Second, proof minimisation reduces even a full-closure derivation to the same small proof, so *proof size alone conceals the search-work difference* — the interesting quantity is rule firings, not the length of the answer.

Remark. None of these Horn engines is an implementation or emulation of **grind**’s E-matching. They compare two deliberately small search policies against each other, and one of them is a fair semi-naive forward-chaining baseline rather than a straw man.

5.3 Induction: selecting a scheme and generalising the statement

Status: The list and natural-number fragments, changing-argument detection, bounded right-hand-side synthesis, structural induction, and trace replay are implemented. General dependent induction and e-graph-guided frontier discovery are specified only.

The induction planner starts from the recursive applications visible in the target and in relevant hypotheses. For each function it records which argument positions are destructured by pattern matching, which are passed to recursive calls as proper subterms, and which non-structural arguments change. Candidates are ranked by how many blocked recursive calls they expose after one constructor split. A variable is not selected merely because its type has an induction principle, and blind induction on every natural variable is not acceptable.

A case split on an inductive value does not automatically supply an induction hypothesis; confusing the two is a design error. Each candidate plan carries an explicit Lean motive, an induction principle, the generalised variables, and the recursive hypotheses available in each case. User-specified schemes take priority.

Generalisation follows a *dependency closure*, and this is where a naive implementation goes wrong. In a context containing $x : A$, $y : B(x)$, and $h : C(x, y)$, induction on x requires a motive of the form

$$M(x) := \forall y : B(x), C(x, y) \rightarrow P(x, y),$$

not a proposition containing the old fixed y at every new value of x . If a generalised variable occurs in the type of another binder, that binder must be generalised or explicitly transported as well. A textual substitution that ignores dependent hypotheses is not a correct generalisation algorithm, and a well-typed-looking first-order erasure is not a substitute for building the motive through Lean’s elaborator.

Generalisation is an *OR alternative*, not an irreversible preprocessing step: too much generality can make a useful theorem false or unnecessarily difficult. Candidate generalisation sets are ordered by size and capped.

The accumulator example, including the obstruction

Write A for append, R for reverse, and RA for accumulator reverse:

$$\begin{aligned} A([], y) &= y, & A(h :: t, y) &= h :: A(t, y), \\ R([]) &= [], & R(h :: t) &= A(R(t), [h]), \\ RA([], a) &= a, & RA(h :: t, a) &= RA(t, h :: a). \end{aligned}$$

The requested theorem is $RA(x, []) = R(x)$. Inducting on x without generalisation gives an induction hypothesis about $RA(t, [])$, whereas the recursive call in the step case is $RA(t, [h])$. *Increasing a rewriting budget cannot make this particular hypothesis say something stronger.* This is the shape of obstruction the planner exists to detect.

The changing-argument detector observes that argument 0 is destructed structurally while argument 1 changes from a to $h :: a$. It replaces the fixed empty accumulator with a fresh parameter and asks for a right-hand side in a typed grammar

$$L ::= [] \mid x \mid a \mid R(L) \mid A(L, L),$$

enumerated in increasing syntax-tree size. The successful candidate is

$$\forall x a, \quad RA(x, a) = A(R(x), a). \quad (1)$$

The base case is immediate. The step case uses the generalised hypothesis $RA(t, h :: a) = A(R(t), h :: a)$ against

$$A(R(h :: t), a) = A(A(R(t), [h]), a) = A(R(t), A([h], a)) = A(R(t), h :: a),$$

the middle step being append associativity. Instantiating (1) at $a = []$ and using append's right identity proves the original theorem.

A second family makes the same point over $\mathbb{N}$. For

$$A(0, a) = a, \quad A(n + 1, a) = A(n, a + 2n + 1),$$

the theorem $A(n, a) = a + n^2$ needs a generalised before induction; the step then reduces to $a + 2n + 1 + n^2 = a + (n + 1)^2$. The automation problem is choosing the generalisation, not performing the final ring calculation.

A worked cross-engine plan: an accumulator *inequality*

The examples above are equations. The mechanism matters at least as much for inequalities, where the induction planner and the arithmetic worker each supply something the other cannot. Let

$$\text{energyAcc}([], a) = a, \quad \text{energyAcc}(x :: xs, a) = \text{energyAcc}(xs, a + (x - 1)^2),$$

and take the goal

$$\forall xs a, \quad a \leq \text{energyAcc}(xs, a). \quad (2)$$

The planner identifies xs as structural, detects that the accumulator changes in the recursive call, and generalises a . The base case is reflexivity. In the step, the generalised induction hypothesis gives

$$a + (x - 1)^2 \leq \text{energyAcc}(xs, a + (x - 1)^2),$$

leaving the demand $a \leq a + (x - 1)^2$. The ordered-algebra worker proves $(x - 1)^2 \geq 0$, the linear worker adds a , and transitivity closes the step. *Four* components contribute and none of them solves the problem alone.

If the update were an expanded polynomial such as $q(x, y) = x^2 + 6xy + 9y^2$, the same plan would call the exact quadratic decomposition of §6.4 to discover $q = (x + 3y)^2$. If the update were nonnegative only under a loop invariant, that invariant would have to be proved and threaded through the induction motive — an arithmetic certificate cannot supply an invariant about recursive calls.

One of the source packages contains an ordinary Lean proof of (2) as an uncompiled integration target. It is *not* claimed that a planner generated it; its purpose is to make the intended constructor sequence concrete enough to validate once a pinned project exists.

Remark (Sampling rejects; it never proves). Two prototypes filter grammar candidates against finite sample environments — one uses all 49 pairs of lists of length at most two over a two-element alphabet. Samples *only reject*. The surviving candidate is accepted solely because it then passes symbolic induction and independent trace replay.

What a replay checker must restrict

The miniature checkers make the induction discipline a first-class, checked datum, and this is the part a production implementation most needs to inherit.

There are separate syntactic constructors for universal pattern variables and for *rigid* free variables. In an induction step the recursion index — a list tail, a natural predecessor — is rigid; only genuinely generalised parameters remain instantiable in the induction hypothesis. Treating the tail as an ordinary rewrite metavariable would let an induction hypothesis rewrite the goal at an unjustified larger argument. In the base case the induction hypothesis is not available at all.

The strongest formulation attaches to each rule an explicit *flexible* variable set — the variables at which that rule, notably the induction hypothesis, may be instantiated — and requires each replayed step to supply exactly that substitution set with correct sorts, rejecting missing or illegal instantiations.

Every rewrite certificate contains a rule identifier and a subterm path. The checker retrieves the rule from the defining equations, from an earlier checked lemma, or from the current induction hypothesis; matches the left-hand side; checks repeated-variable identity and sorts; replaces the selected subterm; and continues. The two replayed sides must end syntactically identical. No search and no normalisation happen during replay.

Proposition 5.3 (Soundness of the list certificate format). *Assume that previously installed equations are valid for all finite lists. If the checker accepts an induction certificate, its universally quantified equation holds for all finite lists under the supplied definitions.*

Proof. Each defining rewrite preserves the interpreted value. Each installed lemma instance preserves it by assumption, and congruence permits replacement inside a typed term context. An accepted base trace therefore proves the empty-list case. In the step trace the only additional rule is the desired statement for the rigid tail, universally quantified over its generalised parameters, and that rule is valid under the structural induction hypothesis. The accepted step trace therefore proves the constructor case, and structural induction establishes the universal equation. $\square$

This justifies a small format. It is not a machine-checked Lean proof of the Python implementation, and the negative tests reduce rather than remove the risk of a checker bug.

Lemma discovery and non-circularity

A public theorem-installation operation first checks both induction cases, and *a lemma is not available while its own certificate is being checked* — which rejects a forged proof that cites its own

conclusion. A bundle of lemmas is accepted only in a dependency order in which no entry refers to a later one, and shadowing a defining equation or the induction-hypothesis name is rejected outright.

Beyond generalisation, a demand-driven lemma explorer works from the residual of a failed step. Retain the normalised left and right frontiers and the rewrites that produced them. Bounded equality saturation can expose alternative forms a one-way normaliser would have discarded — for the accumulator example, unfolding `append` in reverse exposes $h :: a$ as $A([h], a)$, and the remaining frontier then suggests $A(A(u, v), w) = A(u, A(v, w))$.

```
find_lemma(left_frontier, right_frontier):
    expand each frontier using a bounded set of proved equalities
    identify pairs separated by a small structural difference
    anti-unify shared subterms, preserving repeated-variable identity
    abstract a well-typed parameter telescope
    reject trivial, cyclic, or irrelevant candidates
    filter candidates with executable counterexamples where available
    prove surviving candidates in a separate obligation
    return only checked lemmas to the main equality state
```

Typed anti-unification starts in a restricted fragment: fully applied, homogeneous first-order terms with compatible types. Memoise pairs of subterms so that anti-unifying $f(a, a)$ and $f(b, b)$ gives $f(z, z)$ and not $f(z_1, z_2)$. In a dependent fragment, an abstraction that changes a term's type needs a typed telescope and possibly transport proofs; failing to find one is an ordinary search failure.

Three further candidate sources are worth implementing: generalise a changing non-structural argument; propose a distribution or homomorphism lemma such as $R(A(xs, ys)) = A(R(ys), R(xs))$ when nested calls obstruct normalisation; and synthesise a stronger invariant containing the measure, bound, or relation the step requires. Useful templates for the last are

$$f(s, a) = H(s, a), \quad \mu(f(s, a)) = \mu(a) + J(s), \quad R(f(s, a), s, a).$$

Each candidate becomes its own obligation, never a rewrite rule merely because it helped simplify another candidate. Quotient syntactically different candidates only by *already proved* equalities; otherwise the quotient can discard the correct expression or conceal an unproved assumption.

This goal-directed exploration is closely related to CCLemma's use of e-graphs to focus inductive lemma discovery on useful proof frontiers, and to the rippling tradition; the proposal adapts that work to a proof-producing controller rather than originating it [50, 51, 52].

Impose separate limits on term size, saturation nodes, candidate count, induction depth, and lemma-dependency depth, and use term fingerprints and alpha-normalised telescopes to avoid reproving the same lemma under new binder names. Bounded enumeration is exponential in its grammar size; the production advantage must come from recursive-call analysis, relevance filtering, typed indexing, and frontier selection, not from assuming that unrestricted enumeration will be cheap.

5.4 Invariant synthesis by exact CEGIS

Status: Implemented and tested in Python by six independent efforts, in univariate, bivariate, multivariate, accumulator and state-machine forms.

The additive fragment and its certificate

Consider

$$T(0) = t_0, \quad T(n+1) = T(n) + f(n), \quad n \in \mathbb{N}, \quad (3)$$

with $t_0 \in \mathbb{Q}$ and $f \in \mathbb{Q}[X]$. Search for $q \in \mathbb{Q}[X]$ with

$$q(0) = t_0, \quad q(X+1) - q(X) = f(X). \quad (4)$$

The certificate is the exact coefficient list of q . Its checker computes the shift $q(X + 1)$ by polynomial arithmetic and compares exact coefficients; it does not sample the step relation and does not trust an interpolator.

Proposition 5.4 (Recurrence certificate). *If (4) holds then $T(n) = q(n)$ for every $n \in \mathbb{N}$.*

Proof. The base equality is the first condition. If $T(n) = q(n)$ then (3) and the second condition give $T(n + 1) = q(n) + f(n) = q(n + 1)$. Apply induction on n . $\square$

The multivariate form carries parameters. For $S(0, \mathbf{a}) = I(\mathbf{a})$ and $S(n + 1, \mathbf{a}) = S(n, \mathbf{a}) + P(n, \mathbf{a})$, search for $F(n, \mathbf{a}) = \sum_{|\alpha| \leq D} c_\alpha(n, \mathbf{a})^\alpha$, substitute into $F(0, \mathbf{a}) = I(\mathbf{a})$ and $F(n + 1, \mathbf{a}) - F(n, \mathbf{a}) = P(n, \mathbf{a})$, and equate coefficients to obtain a rational linear system in the c_α .

For the accumulator shape $\text{go}(0, a) = a$, $\text{go}(n + 1, a) = \text{go}(n, a + r(n))$, the ansatz is $F(n, a) = P(n) + aQ(n)$ with obligations $F(0, a) = a$ and $F(n + 1, a) = F(n, a + r(n))$. One implementation joins the two systems by introducing a formal tag variable t , encoding the base equation in the coefficients of t^0 and the step equation in those of t^1 , so that a single exact row reduction solves both simultaneously; t does not appear in the final invariant.

Counterexamples direct the coefficient search

For degree bound d , write $q(X) = \sum_{i=0}^d c_i X^i$, maintain exact sample constraints $q(n) = T(n)$, and solve the rational linear system with free coefficients set to zero. Check the candidate's base and step identities. If the step residual

$$r(X) = q(X + 1) - q(X) - f(X)$$

is nonzero, choose a natural n with $r(n) \neq 0$ and add *both* $q(n) = T(n)$ and $q(n + 1) = T(n + 1)$ to the sample set. At least one is a new constraint: if both were already satisfied by the candidate, their difference would force $r(n) = 0$. A nonzero degree- k polynomial cannot vanish at all of $0, 1, \dots, k$, so an exact separating counterexample is found without an SMT solver. If the coefficient constraints become inconsistent, raise the degree bound.

Proposition 5.5 (Fragment completeness). *For a nonzero f of degree k , the finite-difference map $q \mapsto q(X + 1) - q(X)$ sends a leading term aX^{k+1} to $(k + 1)aX^k$ plus lower terms. Over $\mathbb{Q}$ one may therefore solve for the leading coefficient, subtract its difference, and continue downward: a degree- $(k + 1)$ solution exists and is unique once $q(0)$ is fixed. The zero increment gives a constant solution. Hence exact linear solving with a degree budget of at least $k + 1$ terminates on this fragment.*

Equivalently, the forward-difference operator maps polynomials of degree at most $d + 1$ with zero constant term bijectively onto polynomials of degree at most d , since its matrix in the monomial basis is triangular with nonzero diagonal. In the binomial basis $\Delta\binom{n}{k+1} = \binom{n}{k}$, which a future implementation could exploit; the prototypes use ordinary monomials and exact row reduction because that keeps the certificate checker small.

This is a *fragment-level* coverage result. It says nothing about recurrences involving products with S , branching updates, nonlinear state transformations, piecewise definitions, or integer formulas with truncated division, and it does not make the broader induction planner complete.

The overfitting trap

For $f(X) = X + 1$ the discovered result is $q(X) = X(X + 1)/2$. The interpolated expression

$$\tilde{q}(X) = \frac{X(X + 1)}{2} + X(X - 1)(X - 2)(X - 3)$$

agrees with the triangular-number formula on the first four inputs and is rejected by the step checker. This is a small, concrete reason why conjecture generation and proof acceptance must be

separate operations, and the same trap is built into the test suites as a hostile control: a high-degree term that vanishes at every sampled point but violates the recurrence identity elsewhere.

For $P(n) = n^2$ with $I = 0$ the synthesised result is

$$F(n) = \frac{n^3}{3} - \frac{n^2}{2} + \frac{n}{6} = \frac{n(n-1)(2n-1)}{6},$$

and one run recovered the degree-9 invariant for $r(n) = n^8$,

$$F(n, a) = a + \frac{n^9}{9} - \frac{n^8}{2} + \frac{2n^7}{3} - \frac{7n^5}{15} + \frac{2n^3}{9} - \frac{n}{30}.$$

The state-machine generalisation

A more reusable certificate format treats the invariant as a conserved quantity of a transition system: for initial state s_0 and transition T , require

$$I(s_0) = 0, \quad I(T(s)) = I(s).$$

The second identity is a convenient *sufficient* condition — proving $I(s) = 0 \Rightarrow I(T(s)) = 0$ would also suffice but is not the implemented certificate language. For sums of squares, clearing denominators gives $J(n, s) = 6s - n(n+1)(2n+1)$, and

$$J(n+1, s + (n+1)^2) - J(n, s) = 6(n+1)^2 - (n+1)(n+2)(2n+3) + n(n+1)(2n+1) = 0.$$

The next step is a genuine invariant worker for a state transition $s' = F(n, s)$, proving the initial condition and

$$I(n, s) \wedge \text{Guard}(n, s) \implies I(n+1, F(n, s)).$$

Coefficient comparison handles identities and exact nonlinear certificates can handle some inequalities. A warning applies to the multi-invariant version: if the step condition takes the form $I_i(T(\vec{x})) = \sum_j h_{ij}(\vec{x})I_j(\vec{x})$, then jointly discovering the I_j and the multipliers is a *bilinear* search, not linear algebra, and it should not be described as the latter. Separate candidate generation from multiplier certification, or fix one side at a time.

Remark. An invariant does not prove that a loop or a recursive definition terminates. Termination is a separate obligation.

5.5 Negative verdicts: when “not found” may become “does not exist”

Everywhere else in this document a bounded search that finds nothing returns UNKNOWN, and that rule is not negotiable. But UNKNOWN is not the only possible negative answer, and treating it as such gives up a real capability. A tactic that can sometimes report *provably no such term exists* is doing something `exact?`, `tauto`, `Aesop`, `decide` and `omega` do not (§2.1) — and something a user acts on very differently from a timeout.

Exactly one worker in this collection can already do it: integer-lattice elimination emits a divisibility obstruction that *proves* no integer solution exists (Proposition 5.7). The question is what makes that legitimate, and what else could qualify.

Exhausted is not truncated

The first requirement is a distinction the worker contract of §3 does not draw. `Unknown(reason)` conflates two outcomes that are logically different:

- **exhausted** — the search space was covered and contained no solution;
- **truncated** — a budget, depth cap, queue bound, candidate cutoff, or omitted plan stopped the search early.

Only the first can support a negative conclusion, and a worker must therefore report which occurred rather than leaving the caller to guess. A useful shape for a resumable search is a small-step machine whose event type makes the distinction unforgeable:

```
inductive SearchEvent where
| found      (evidence : Certificate) (next : Machine)
| stepped    (next : Machine)          -- administrative progress
| exhausted  (summary : ExhaustionSummary)
| truncated  (reason   : TruncationReason)
```

with the invariants that every transition consumes its documented choice count; that administrative steps cannot be mistaken for exhaustion; that a truncated branch contributes nothing to global negative evidence; and that a plan scheduler shares a global budget rather than resetting a local one per plan. That last invariant is the same one §3 states for nested actions, and it is load-bearing here: a search whose sub-searches each reset their own budget can report **exhausted** having covered very little.

The completeness ledger

Exhaustion of the search is necessary but not sufficient. The *problem* handed to the search must also have been a faithful rendering of the goal, and that is a separate obligation with its own failure modes: an unsupported subterm silently abstracted to an atom, a family description truncated at a recursion bound, an instance binder dropped, a plan omitted from the enumeration.

The mechanism that makes this checkable is a *completeness ledger*: a data structure carried alongside the reified problem whose constructors name exactly which obligation is missing. It is not a Boolean. Any entry in it weakens a no-solution result to UNKNOWN, and because its constructors are named, the diagnostic can say *why* the negative verdict was withheld.

Two implementation properties make this work, and both are cheap.

First, the ledger is computed by the *encoder*, at translation time, before any search runs. It is a property of the rendering, not of the outcome, so it cannot be influenced by how the search went. A reference implementation carries it as one bit per translated goal, set at each site that loses information: a quantifier in negative position sealed as an opaque atom, a recursive datatype unfolded only to a bounded depth, a class context accepted without its dictionary premises.

Second, the bit is carried *per node*, not only per problem. Every atom in the reified goal records whether it is safe for refutation. An atom built purely from universally quantified variables keeps a negative verdict sound; an atom mentioning a concrete constant, whose structure the engine cannot see, poisons it. So does an erased typeclass dictionary — because the dictionary can carry data, and a term inhabiting the goal may exist by way of a method the search never modelled.

The payoff is a diagnostic that names the obstruction. Rather than “no term found within bounds”, the user can be told *which symbols* blocked the stronger claim — which is the difference between an unhelpful timeout and an actionable one.

The combined predicate is then a conjunction, checked in a fixed order:

$$\text{search completed} \wedge \text{translation lossless} \wedge \text{no self-reference available.}$$

The third conjunct is worth keeping. If the goal’s own name was excluded from the premises during search — as it must be, to prevent a circular proof — then a negative result can be sharpened by one extra bounded search that reintroduces it, paid for out of the first search’s unused budget. Succeeding there yields the genuinely useful message “*no non-recursive inhabitant exists; one exists if the definition may call itself*”. The discipline that makes this safe is monotonicity: the sharper claim may degrade to the weaker claim, but exhausting the budget during the diagnostic pass must never degrade an already-decided result to UNKNOWN.

Invariant 13 (Negative evidence requires an intact chain). A “provably no solution” verdict requires *both* that the search reported **exhausted**, *and* that the completeness ledger for the reified problem is empty. Failure of either yields UNKNOWN. The display layer must never upgrade UNKNOWN to a refutation.

For a term-synthesis worker over a dependently typed goal this specialises to a two-axis rule: a negative verdict requires that no quantified occurrence was left opaque or bounded-instantiated, *and* that no atom is hiding dependent structure. Otherwise the answer is “no term found within bounds”.

Portfolio asymmetry

A portfolio that mixes a complete worker with incomplete ones needs an explicit rule, or the incomplete ones will corrupt the complete one’s verdicts. The rule is asymmetric:

- An incomplete worker’s *candidates* improve coverage and ranking.
- An incomplete worker’s *failure* can neither create nor cancel a refutation.
- A complete worker’s negative evidence survives only if its own fragment, translation and search completeness chain is intact (Invariant 13).
- A wrapper that transforms the problem — a classical translation, a strengthened goal — produces a *new* problem, and must drop any evidence that no longer denotes it.

Concretely: a Bernstein worker timing out says nothing about a goal that the Sturm worker of §6.8 decides, and the scheduler must not let the first result suppress the second.

What could qualify

Three further sources of genuine negative evidence are available to this design, and only the first is implemented.

Lattice obstructions (implemented). $g \nmid r$ after unimodular elimination proves that $Ax = b$ has no integer solution. The certificate is replayable and the checker verifies $\det U = \pm 1$ independently.

Checked countermodels (partially implemented). An exact rational point at which the target is negative, *validated independently* for membership in the domain and for sign, refutes a nonnegativity goal. One Bernstein implementation already returns such a point as a three-valued REFUTED outcome with a separate **check_witness**. The discipline that makes this sound is worth stating as a formula: a solver’s output is not semantic evidence, and only

sealed problem + canonical query + bounded observation + independent replay

produces a countermodel receipt. A satisfying assignment to an abstraction remains a **Candidate**.

Intuitionistic propositional inhabitation (not implemented; the largest available addition). A terminating decision procedure for the implicational propositional fragment — Dyckhoff’s contraction-free LJ being the standard choice — decides inhabitation rather than merely searching for it. Within its fragment, exhaustion *is* non-inhabitation.

It is worth being exact about the *strength* of that, because it is not the same as the lattice worker’s. A reference implementation of this design exists and produces, for a negative answer, a single enumeration tag and no artefact: there is no closed-branch tree, no refutation object, nothing a second program can re-check. Its soundness rests on the decision procedure being correctly implemented, plus the translation gate below. So it belongs in a *weaker* slot than lattice elimination, which hands back a certificate whose checker verifies $\det U = \pm 1$ without calling the elimination that produced U .

Remark (An available contribution). That gap is a genuine opportunity rather than a fact of nature. LJT is contraction-free and terminating, so a failed search tree is *finite* and could be reified into a checkable refutation object. Nobody appears to have built that. Doing so would give this design a second *certificate-producing* infeasibility worker, in a fragment — arrows, products, constructor-tagged sums, empty types, over opaque atoms — where users regularly believe something is impossible and get no confirmation.

Two properties make this unusually good value here. First, it operates on the *propositional skeleton* of a goal, so dependent subformulas that only need to be transported rather than analysed can be alpha-normalised to opaque atoms and the goal still decided: $(\forall n : \mathbb{N}, P n) \rightarrow Q \rightarrow (\forall n : \mathbb{N}, P n)$ is closed by $\lambda h _ \Rightarrow h$ without the worker ever looking inside P . Second, it composes with a classical fallback that is complete for the same fragment. By Glivenko’s theorem, a propositional formula is classically provable exactly when its double negation is intuitionistically provable, so re-running the same decision procedure on $\neg\neg G$ decides classical propositional provability, and a resulting term $t : \neg G \rightarrow \perp$ reconstructs as `Classical.byContradiction t`. Peirce’s law is the standard illustration: constructively uninhabited, and the fallback supplies the classical witness. The honesty condition is that *quantified goals skip the fallback*, since Glivenko does not extend past the propositional fragment without double-negation shifts.

5.6 Witness synthesis: construct terms, not satisfying assignments

The logical distinction

To prove $\forall x \exists y P(x, y)$, a satisfying assignment for one value of x is not enough. A useful synthesiser produces a term $y = w(x)$ and proves $\forall x P(x, w(x))$. For

$$\forall x \in \mathbb{R}, \quad \exists y \in \mathbb{R}, \quad x + 1 \leq y \leq x + 2,$$

the witness is $w(x) = x + 1$; no constant works for every real x , and a constant-only template returns no certificate where the affine template succeeds.

Sampling individual values of x and finding unrelated values of y constructs no function and proves nothing. The grammar must represent a well-typed expression $w(x)$. A synthesis oracle may help choose w ; it cannot justify $P(x, w(x))$ by testing.

Witness binders retain their dependencies. A witness for the second existential may refer to an earlier witness but not to a future binder or to a branch-local variable outside its scope. The instructive negative example is $\exists x : \alpha, \forall y : \alpha, x = y$: the witness for x cannot be the later-bound y , and a naive synthesiser that sees all displayed variables will propose exactly that. The existential task’s context must exclude y , and replay must reject any escaped local variable.

Constructing a value of `Fin n` requires a value *and* a bound proof; over $\mathbb{N}$, a construction using a signed intermediate representation carries a nonnegativity obligation. A witness is a program with a proof obligation.

The search grammar should be type-directed and size-bounded: local variables, constructors, projections, selected recursive calls, affine combinations, and guarded division or remainder operations. Piecewise terms are permitted only with exhaustive, proved branch conditions — never on the assumption that finitely many sampled regions cover the domain.

Affine witnesses with Farkas multipliers

Status: Implemented and tested in Python.

Let the input domain be the rational polyhedron $g(x) = Ax + b \geq 0$ and the target constraints $h(x, y) = Cx + Dy + e \geq 0$, all componentwise and non-strict, and seek $y = Mx + c$. A sufficient certificate is a nonnegative matrix Λ and a nonnegative vector s with

$$C + DM = \Lambda A, \tag{5}$$

$$e + Dc = \Lambda b + s. \tag{6}$$

Proposition 5.6 (Affine witness certificate). *If (5)–(6) hold with $\Lambda, s \geq 0$, then for every x with $Ax + b \geq 0$ the term $y = Mx + c$ satisfies $Cx + Dy + e \geq 0$.*

Proof. Substitution gives $Cx + D(Mx + c) + e = \Lambda(Ax + b) + s$. Every entry on the right is a sum of nonnegative multiples of nonnegative quantities plus a nonnegative slack. $\square$

The decisive observation is that (5) and (6) are *linear in the unknowns* M, c, Λ, s , because A, b, C, D, e are fixed problem data. Witness synthesis and proof search are therefore *one* linear feasibility problem rather than an enumeration of candidate affine functions followed by a separate verification pass. With n inputs, m outputs, r domain constraints and q target constraints the encoding has $m(n + 1) + q(r + 1)$ unknowns and $q(n + 1)$ coefficient equations before template restrictions; entries of M, c are unrestricted and entries of Λ, s are constrained nonnegative. A constant-only grammar adds $M = 0$.

For a nonempty rational polyhedron, the affine implication form of Farkas’ lemma gives a completeness route for a *fixed* affine witness: validity of each non-strict affine target inequality has such multipliers [62]. The floating-point proposer is not a complete exact LP algorithm and an empty domain needs separate handling, but neither qualification affects certificate soundness.

A variant fragment solves $Aw = Bx + c$ for $w = Wx + d$ by coefficient matching, giving the exact systems $AW = B$ and $Ad = c$, each solved by rational reduced row-echelon elimination with free variables set to zero, and re-multiplied exactly by the checker with no sampled values of x involved.

Integer witnesses are a different problem

A rational affine solution is not an integer proof. The implemented integer mode merely requires every coefficient of W and d to be integral, and is deliberately incomplete: a different choice of free variables might yield an integral solution when the default rational one does not. The instructive case is $2w = x$, which has the rational witness $x/2$ and no integer witness valid for every integer x ; an integer solver must obtain a parity assumption or fail, and must never silently reinterpret rational division as integer arithmetic.

A practical first integer grammar is

$$w(x) ::= c + \sum_i a_i x_i \mid \left\lfloor \frac{c + \sum_i a_i x_i}{d} \right\rfloor \mid \text{ite}(g, w_1, w_2), \quad d > 0,$$

with every candidate checked in the source integer semantics.

Integer lattices and certificates of impossibility

Status: Implemented and tested in Python. The only worker here that emits a certificate of infeasibility.

A stronger integer fragment solves $Ax = b$ over $\mathbb{Z}$ by row-by-row unimodular elimination, returning either a particular solution with a basis for all homogeneous freedom, or a replayable divisibility obstruction. It does not search a bounded numerical box. For $6x + 10y = 2$ it returns

$$(x, y) = (2, -1) + t(-5, 3), \quad t \in \mathbb{Z},$$

and the trace explains why this family contains *all* solutions — which is what a later impossibility argument needs.

After processing rows $0, \dots, i - 1$, maintain

$$\{x \in \mathbb{Z}^n : A_{<i}x = b_{<i}\} = \{x_0 + Bz : z \in \mathbb{Z}^d\}, \quad (7)$$

initially $x_0 = 0$, $B = I_n$, $d = n$. For the next row a compute $v = aB$ and $r = b_i - ax_0$, and find an integer unimodular U with $vU = (g, 0, \dots, 0)$, $g \geq 0$. If $g = 0$, consistency requires $r = 0$ and the family is unchanged. If $g > 0$, consistency requires $g \mid r$; with $q = r/g$, update

$$x_0 \leftarrow x_0 + (BU)_{:,0} q, \quad B \leftarrow (BU)_{:,1:}, \quad d \leftarrow d - 1.$$

The elementary block, for a current pair (a, b) with $sa + tb = g$, is

$$\begin{pmatrix} s & -b/g \\ t & a/g \end{pmatrix},$$

whose determinant is $(sa + tb)/g = 1$ and which sends the pair to $(g, 0)$; a sign flip makes the final first entry nonnegative.

Proposition 5.7 (Lattice elimination). *Each successful row update preserves (7), and a reported divisibility obstruction proves that no integer solution exists.*

Proof. Because U is unimodular, $z = Uw$ is a bijection of $\mathbb{Z}^d$. The new row equation becomes $gw_0 = r$. For $g > 0$ this has an integer solution precisely when $g \mid r$, in which case $w_0 = r/g$ and all other coordinates remain free; substitution gives the stated update. For $g = 0$ the row is either redundant or contradictory. Induction over processed rows proves both claims. $\square$

The replay format stores, per row, the row number, U , g , r , and either the quotient or an obstruction marker. The independent checker recomputes the current row image and residual, checks that all entries are integers, verifies $\det U = \pm 1$, and performs the exact state update — without calling the extended-gcd construction or the solver. For [6 10] one trace step is

$$U = \begin{pmatrix} 2 & -5 \\ -1 & 3 \end{pmatrix}, \quad [6 \ 10]U = [2 \ 0], \quad \det U = 1,$$

and with right-hand side 1 the same reduction establishes the obstruction $2 \nmid 1$.

A production Lean checker might instead accept elementary column operations or an explicitly checked integer inverse; the elementary-operation format is attractive because it has short local correctness lemmas and avoids determinant expansion. For a positive existential goal, checking a returned witness directly is cheaper than verifying completeness of the whole lattice — the latter is needed only when the family or a negative result is used.

Parameterised right-hand sides extend the same machinery: if $Au_0 = b_0$ and $AU = C$, then $x = u_0 + Up$ is a witness family for $Ax = b_0 + Cp$. This sufficient method misses witnesses that need piecewise divisibility conditions or integer division, so it must not report impossibility when the coefficientwise construction fails. Carrying affine residuals through the same row algorithm produces obligations $g \mid r(p)$ and a checked integer quotient once they are proved — for example, $2x = p$ requires evenness of p . If the goal also includes $Cx \leq d$, substitute the lattice parameterisation to obtain inequalities in the remaining integer parameters; *dropping the inequalities after solving the equalities would be unsound*, and natural-number witnesses additionally require nonnegativity, since solutions over $\mathbb{Z}$ do not automatically inhabit $\mathbb{N}$.

There are at most n dimension-reducing steps, but intermediate coefficient bit lengths rather than matrix dimensions can dominate cost. The prototype is a straightforward exact implementation, not an optimised Smith or Hermite normal-form library.

Finite-residue witnesses

Status: Implemented and tested in Python.

A different existential shape is discharged by finite case analysis on a residue class. Given a modulus m , synthesise offsets d_r , one per residue class r modulo m , such that for every integer n ,

$$n \leq n + d_{n \bmod m} < n + m \quad \text{and} \quad a(r + d_r) \equiv b \pmod{m}.$$

Verification is a finite check over the m residue classes; the universal interpretation is justified by the quotient/remainder theorem, not by the sampled integers. When the congruence $az \equiv b \pmod{m}$ has no solution — detected by a gcd divisibility condition — the worker reports impossibility for that parameter tuple rather than failing silently. The implementation is exercised at inputs such as $10^{20} + 7$ to confirm that the finite table really does instantiate at arbitrary integers.

Polynomial witnesses certified by nonnegativity

Status: Implemented and tested in Python.

The composition pattern the whole architecture argues for appears in miniature here. For

$$\forall x \in \mathbb{R}, \quad \exists y \in \mathbb{R}, \quad x \leq y \wedge -x \leq y,$$

one prototype enumerates $w(x) = a + bx + cx^2$ with $a, b, c \in \{-2, \dots, 2\}$, ordered by coefficient ℓ_1 size and then lexicographically, using five sample values only as a rejection filter. The 23rd candidate is $w(x) = x^2 + 1$, and acceptance requires *two* nonnegativity certificates, for $w - x \geq 0$ and $w + x \geq 0$. Convenient explanatory decompositions are

$$x^2 + 1 - x = \left(x - \frac{1}{2}\right)^2 + \frac{3}{4}, \quad x^2 + 1 + x = \left(x + \frac{1}{2}\right)^2 + \frac{3}{4},$$

but the recorded certificates contain the different rational weighted-square decompositions the LP actually returned; both replay exactly. The artefact does not pretend that a hand-simplified certificate was the optimiser's output.

A production implementation should also try simpler witnesses — an available absolute-value or maximum expression — when the local library makes those cheap. Restricting a prototype to polynomials is an experiment design, not a recommended restriction on all **Forge** witnesses.

6 Nonlinear arithmetic: exact certificates, not floating-point proofs

6.1 The acceptance language

The nonlinear worker targets polynomial statements over $\mathbb{R}$ with exact rational coefficients. Variables may stand for opaque real subexpressions; any further relationship between them must be supplied as a proved constraint. The same representation must not be reused over an arbitrary ring with an assumed real ordering.

Normalise the target to $p(\mathbf{x}) \geq 0$, the hypotheses to $g_i(\mathbf{x}) \geq 0$, and the equalities to $h_j(\mathbf{x}) = 0$. The certificate language is

$$p = \sum_{k=1}^s c_k q_k^2 \prod_{i=1}^m g_i^{a_{ki}} + \sum_{j=1}^r u_j h_j, \quad c_k \in \mathbb{Q}_{\geq 0}, \quad a_{ki} \in \mathbb{N}, \quad (8)$$

with q_k, u_j rational polynomials. The checker is given the original p, g_i, h_j independently of the certificate, reconstructs the right-hand side exactly, and requires polynomial identity.

Proposition 6.1 (Cone-certificate soundness). *If (8) holds, every point satisfying $g_i \geq 0$ and $h_j = 0$ satisfies $p \geq 0$.*

Proof. Each square is nonnegative, each hypothesis product is nonnegative, and each coefficient c_k is nonnegative. Every equality-multiplier term vanishes. Summing proves the claim. $\square$

This elementary proof is exactly why search can be far more complicated than checking — the observation Harrison made foundational for SOS reconstruction [56]. The language is a bounded preordering-style cone with equality-ideal terms. Restricting products to individual g_i gives a smaller family; fixing the possible q_k gives a finite cone. None of these should be called an unrestricted positivity decision procedure.

Two details of (8) matter in implementation. The equality multipliers u_j carry *no sign condition*: an LP formulation with nonnegative variables therefore needs $\pm$ columns for each ideal generator. The reason is not pedantry — under $y = x$,

$$y^2 - 2x + 1 = (x - 1)^2 + (x + y)(y - x),$$

and the second term need not be nonnegative. It vanishes because of the equality. Treating every multiplier as nonnegative would discard that flexibility. The second detail is naming: in the serialised formats the field called **square** stores q_k , not q_k^2 .

Strict inequalities. Strictness needs explicit information. One sufficient certificate is an identity of the same form for $p - \varepsilon$ with rational $\varepsilon > 0$. It is not necessary in general: a polynomial can be strictly positive on an unbounded domain while having infimum zero, so $x > 0$ implies $x > 0$ without a uniform rational lower bound. Another route is a term whose coefficient and factors are provably strictly positive at every admissible point. The proof layer must preserve the strict inequality rather than silently weakening it to nonnegativity.

Refutation. A certificate for -1 in the nonnegative cone plus the equality ideal gives a contradiction. If $q > 0$ is a strict hypothesis, a checked identity expressing $-q^m$ as a cone element plus vanishing ideal terms, with $m \geq 1$, also gives one.

6.2 Search is untrusted; repair, do not round

Every implemented search here follows the same shape, and the shape is the point.

```
searchCone(problem, degree, budgets):
  columns := boundedSquaresTimesConstraintProducts(problem)
            ++ signedIdealColumns(problem.equalities)
  support := floatingPointLP(columns, target)      -- untrusted
  if no support: return UNKNOWN
  coefficients := exactRationalSolve(support, target)
  if unavailable: return UNKNOWN
  certificate := assemble(coefficients)
  if exactCheck(problem, certificate): return certificate
  return UNKNOWN
```

A floating result is not a certificate. The prototypes take the support of the proposed coefficient vector, re-solve the corresponding coefficient equations over exact rationals, require the repaired coefficients to be nonnegative, and expand the certificate to check the original identity. Rationalisation uses a denominator cap; tiny weights may be dropped during *search reconstruction*, which never weakens the final check.

This separation handles the most important failure mode: a numerical solver can treat a small nonzero residual as acceptable, whereas an identity proof cannot. A rational coefficient merely close to a valid one is still wrong, and a residual of 10^{-12} is not a proof. The test suites include a targeted attack on this — a weight of $(10^{20} + 1)/10^{20}$ offered where 1 is required — which is why the checkers assert the weight’s exact rational type before comparing it.

Numerical infeasibility is likewise not a proof that the goal is false, nor even a formally established proof that the finite cone contains no solution. Indeed the implementations are *not complete even for every feasible instance of their own finite LP cone*, because the support and free-parameter heuristics can miss an exact feasible solution. And even a mathematically exact LP infeasibility certificate would refute *that coefficient representation problem*, not the original theorem. The repair procedures are deliberately modest: they take coefficients above a fixed support threshold and set free parameters of the rational system to zero, which can miss valid certificates with small coefficients or a different feasible choice of free parameters. The correct result is UNKNOWN, not a weakened acceptance test. A better oracle can explore nearby supports and solve exact feasibility in the nullspace without changing the checker.

6.3 Building the dictionary

For each square-free subset or multiset I of the nonnegative premises with $|I| \leq s$, form the product g_I and enumerate monomials of degree at most $\lfloor (D - \deg g_I)/2 \rfloor$. Take squares of those monomials and of their pairwise sums and differences. Add signed monomial multiples of each equality as ideal columns.

Two refinements repay their cost. The first is *target-driven* square proposal: if the target contains $au^2 + cuv + bv^2$ with $a > 0$, propose $q = u + (c/2a)v$, and when $b > 0$ also the opposite orientation; likewise propose binomial roots from exact rational square-root ratios of

positive even-degree terms. One implementation’s initial heuristic recognised $(x - 2/3)^2$ but was defeated by $(x - 2/3)^2 + 1/97$, where a positive constant obscured the coefficient ratio; adding the cross-coefficient construction recovered the intended square, and the case is now a regression test.

The second is *variable-support partitioning*: partition the constraints by variable support and request cross-component products only when the target connects those components, instead of expanding a global Cartesian product. Recorded generator budgets run to a few thousand columns; with M monomials an ungaurded binomial dictionary already has up to $M + 6\binom{M}{2}$ entries before deduplication.

Four honest failures of a finite dictionary

The prototypes each kept a control showing a true statement their own search misses. They are different arguments and all four are worth keeping.

- **Motzkin.** $M(x, y) = x^4y^2 + x^2y^4 + 1 - 3x^2y^2$ is nonnegative by arithmetic–geometric mean on its three nonnegative terms, and is famously not a sum of squares of polynomials. Bounded square dictionaries return UNKNOWN.
- $(2x - 3y)^2$, **with a counting argument.** A dictionary restricted to m , $m + n$ and $m - n$ cannot represent a square with unequal coefficients: covering the $-12xy$ cross term using $(x - y)^2$ requires weight 6, already exceeding the target’s x^2 coefficient 4, and adding other positive dictionary atoms cannot repair the deficit.
- $(x + y + 1)^2$, **with a rank argument.** A rank-one quadratic Gram matrix for the coefficient vector $(1, 1, 1)$ cannot be decomposed as a positive sum of rank-one vectors with only two nonzero coordinates unless the summands lie in its one-dimensional range, and none of the allowed binomial vectors does.
- $(2x + y)^2$ **at degree cap 2.** A restricted coefficient-ratio dictionary misses it outright — a goal that the exact quadratic decomposition of §6.4 decides immediately.

The last of these is the most useful, because it is a *cross-engine* argument rather than a limitation. One prototype’s dictionary obstruction and another’s quadratic completeness theorem together justify the multi-engine design better than either does alone. A failure of one search is a reason to route, not a reason to stop.

Remark. A nonnegative polynomial need not be a sum of polynomial squares at all. Failure of a bounded search proves only that this search found nothing.

6.4 Exact rational quadratic decomposition

Status: Implemented, tested, and used to generate Lean replay candidates.

For a rational quadratic polynomial in n variables let $z = (1, x_1, \dots, x_n)^T$ and $p(\mathbf{x}) = z^T Q z$, where Q is the unique symmetric matrix obtained by placing constant and square coefficients on the diagonal and half of each mixed or linear coefficient in the corresponding symmetric off-diagonal entries.

Theorem 6.2 (Quadratic fragment). *A rational quadratic polynomial is nonnegative on all of $\mathbb{R}^n$ if and only if its homogenised symmetric matrix Q is positive semidefinite. In that case it admits a certificate $p = \sum_k d_k \ell_k^2$ with nonnegative rational d_k and rational affine linear forms ℓ_k .*

Proof. If Q is positive semidefinite the first assertion is immediate. Conversely, for a vector $(t, \mathbf{y})$ with $t \neq 0$,

$$(t, \mathbf{y})^T Q (t, \mathbf{y}) = t^2 p(\mathbf{y}/t) \geq 0,$$

and the assertion for $t = 0$ follows by continuity; thus Q is positive semidefinite.

For the rational decomposition, choose a positive diagonal pivot $d = Q_{kk}$. Completing the square gives

$$z^T Q z = d \left(z_k + \sum_{j \neq k} \frac{Q_{kj}}{d} z_j \right)^2 + z_R^T \left(Q_{RR} - \frac{Q_{Rk} Q_{kR}}{d} \right) z_R.$$

The residual Schur complement is positive semidefinite and every entry remains rational. Repeat. If all remaining diagonal entries vanish in a positive semidefinite matrix, all remaining off-diagonal entries also vanish: otherwise a suitable vector supported on two coordinates gives a negative value. The process therefore terminates with the claimed rational weighted squares. $\square$

```
quadratic_certificate(p):
  if total_degree(p) > 2: return Unknown
  Q := exact_homogenized_matrix(p)
  active := all coordinates
  terms := []
  while active is not empty:
    if any active diagonal is negative: return Unknown
    choose a positive diagonal pivot d
    if no pivot exists:
      if the active submatrix is zero: break
      else: return Unknown
    emit d * (pivot_variable + rational_linear_tail)^2
    Q := exact Schur complement on the remaining coordinates
  certificate := sum of emitted terms
  return certificate only if expand(certificate) == p
```

The algorithm rejects a negative diagonal pivot or a nonzero zero-diagonal residual, and the returned certificate is checked again by polynomial expansion through the *general* cone checker. No numerical eigenvalue threshold is used: a matrix with a tiny nonzero rational off-diagonal entry is treated exactly, not rounded into positive semidefiniteness.

The matrix arithmetic uses $O(n^3)$ rational operations, which is not an $O(n^3)$ bit-complexity claim — numerators and denominators can grow substantially, and sparse pivots, fraction-free variants and explicit bit-length budgets are natural production improvements. A fraction-preserving LDL^T decomposition is the same algorithm in matrix form.

A failed global quadratic test does not imply the constrained goal is false: x is not globally nonnegative but is nonnegative under the hypothesis $x \geq 0$. Such a result routes to a constraint-aware worker; it does not terminate the tactic.

Expanding a nonnegative quadratic hides the linear forms whose squares establish its sign. Existing arithmetic preprocessing may discover the right consequences, but it need not start from those forms. This worker reconstructs the certificate systematically rather than hoping a useful square already appears syntactically.

6.5 Discovered certificates worth recording

The implemented searches returned, among others:

$$\begin{aligned}
(a^2 + b^2)(c^2 + d^2) - (ac + bd)^2 &= (ad - bc)^2, \\
x^2 + y^2 + z^2 - xy - yz - zx &= \frac{1}{2}(x - y)^2 + \frac{1}{2}(y - z)^2 + \frac{1}{2}(z - x)^2, \\
x^4 + y^4 - 2x^2y^2 &= (x^2 - y^2)^2, \\
x^4 + y^4 + 1 - x^2y^2 - x^2 - y^2 &= \frac{1}{2}(x^2 - y^2)^2 + \frac{1}{2}(x^2 - 1)^2 + \frac{1}{2}(y^2 - 1)^2, \\
x^4 + y^4 + z^4 - x^2y^2 - y^2z^2 - z^2x^2 &= \frac{1}{2}(x^2 - y^2)^2 + \frac{1}{2}(y^2 - z^2)^2 + \frac{1}{2}(z^2 - x^2)^2, \\
(x^2 + y^2 + z^2)^2 - 3(x^2y^2 + y^2z^2 + z^2x^2) &= \frac{1}{2}((x^2 - y^2)^2 + (x^2 - z^2)^2 + (y^2 - z^2)^2), \\
x - x^3 &= x^2(1 - x) + x(1 - x), \\
x^2 + y^2 - 2 &= (x - y)^2 + 2(xy - 1), \\
x^3 + y^3 - xy(x + y) &= (x + y)(x - y)^2 \quad (x, y \geq 0),
\end{aligned}$$

together with the manually supplied discriminant identity

$$b^2 - 4ac = (2ad + b)^2 - 4a(ad^2 + bd + c), \quad (9)$$

which is accepted by the checker although it lies outside every implemented search dictionary — a deliberate demonstration that the checker’s language is larger than the searcher’s.

A domain-dependent example shows why hypothesis products matter. Under $0 \leq x \leq 1$ and $0 \leq y \leq 1$,

$$2x(1 - x) + 3y(1 - y) + xy + \frac{1}{5} \geq 0.$$

Supplied in expanded form, with negative x^2 and y^2 coefficients, the useful certificate consists of products of the four bound polynomials $x, 1 - x, y, 1 - y$. Global SOS search alone cannot establish this, because the assertion is domain-dependent. Removing or changing a hypothesis makes the exact checker reject the certificate, so hypothesis provenance is part of the tested interface.

6.6 A concrete path to general SOS search

A later worker can replace the fixed square list with Gram matrices: choose a monomial vector v_D and seek $Q_\alpha \succeq 0$ satisfying coefficient identities for

$$p = v_D^T Q_0 v_D + \sum_{\alpha} v_{D_\alpha}^T Q_\alpha v_{D_\alpha} g^\alpha + \sum_j u_j h_j.$$

CVXPY or CSDP can serve as modelling front ends [65, 66]. Their output remains untrusted: recover rational matrices inside the exact affine coefficient-constraint space, perform an exact positive-semidefiniteness check, and convert each matrix to rational weighted squares. Entrywise rounding followed by approximate eigenvalue testing is not acceptable. Singular Gram matrices are especially delicate, since perturbation can destroy either positivity or the coefficient identity; exact nullspace handling, support changes, and facial reduction are worthwhile *search* techniques, but acceptance must still reduce to (8) [58]. No implementation here contains an SDP solver and no SDP performance claim is made.

Remark (Reuse the existing Lean backend). This point deserves emphasis because it materially narrows what *Forge* should claim. A Lean *sos* project already implements an end-to-end nonlinear-real-arithmetic tactic after Harrison’s decision procedure: reification, search, rational reconstruction, and certificate-based proof construction, with a witness-oriented replay interface and a configuration that already includes degree deepening, constraint-product bounds, and optional refutation powers [31, 32, 33]. Consequently “add an SOS tactic to Lean” is *not* a novel component of this design. Similarly, a Lean *lp* project supplies verified linear programming and a supported quantified rational-affine fragment [36, 37], covering much of the affine-witness machinery.

The production arrangement should therefore be staged: run the cheap in-house certificate searches first, and escalate to the existing libraries rather than building a second unaudited wrapper. *Forge*’s contribution in this area is *when* to request a certificate, *how* to bind it to the source problem, and *what* to do with the proved consequence afterwards — not the certificate search itself. The version caveat of §2 applies: compatibility between those projects and the pinned baseline must be established, not assumed.

6.7 Bernstein certificates on rational boxes

Status: Implemented and tested in Python by seven independent efforts.

Coefficients and the bound

Let p be a polynomial on the rational box $X = \prod_{i=1}^n [l_i, u_i]$ with $l_i < u_i$. Substituting $x_i = l_i + (u_i - l_i)t_i$ gives $q(\mathbf{t}) = \sum_{\alpha} a_{\alpha} \mathbf{t}^{\alpha}$ on the unit box. With d_i a coordinatewise degree bound, the tensor-product Bernstein expansion is

$$q(\mathbf{t}) = \sum_{0 \leq \beta \leq \mathbf{d}} b_{\beta} \prod_i \binom{d_i}{\beta_i} t_i^{\beta_i} (1 - t_i)^{d_i - \beta_i},$$

with exact rational coefficients

$$b_{\beta} = \sum_{\alpha \leq \beta} a_{\alpha} \prod_i \frac{\binom{\beta_i}{\alpha_i}}{\binom{d_i}{\alpha_i}}. \quad (10)$$

Lemma 6.3. For $0 \leq a \leq d$, $t^a = \sum_{b=a}^d \frac{\binom{b}{a}}{\binom{d}{a}} \binom{d}{b} t^b (1 - t)^{d-b}$.

Proof. Use $\binom{d}{b} \binom{b}{a} = \binom{d}{a} \binom{d-a}{b-a}$, factor out t^a , and apply the binomial theorem to the remaining sum, which collapses to $(t + (1 - t))^{d-a} = 1$. $\square$

Applying this in each coordinate proves (10). Each basis term is nonnegative on the unit box and the basis terms sum to one, so

$$\min_{\beta} b_{\beta} \leq p(\mathbf{x}) \leq \max_{\beta} b_{\beta} \quad (\mathbf{x} \in X).$$

In particular nonnegative coefficients certify nonnegativity throughout the box, and strictly positive coefficients certify strict positivity.

Remark. Negative Bernstein coefficients are *not* counterexamples. They mean only that this sufficient condition has not proved the desired sign. A returned negative point is a different kind of result and must be evaluated exactly.

The subdivision tree is part of the proof

If a box does not certify immediately, split a side at an interior rational point. A certificate is a binary tree: an internal node contains an axis, an interior rational split point, and *both* children; a leaf contains its coefficients or a claimed nonnegative rational lower bound.

The checker independently reconstructs every child box from the root, recomputes each leaf’s coefficients, and confirms the claimed bound. Requiring both children proves coverage — there is no way to omit a difficult corner by supplying an incomplete list of favourable boxes. A missing child, an invalid split axis, or a split point at an interval endpoint is rejected, and every basis index must appear exactly once in canonical order. Overlap on a shared face is harmless; missing an entire child is not. Node and depth limits are operational rejection limits, not mathematical assumptions about positivity.

The strongest implementations make the checker algorithmically *opposite* to the producer: rather than recomputing (10), they expand the claimed Bernstein representation — in the original x coordinates, using

$$\binom{d}{k} \frac{(x-a)^k (b-x)^{d-k}}{(b-a)^d},$$

— and compare the result against an independently computed affine transform of the target. Producer and checker then share no conversion formula, though they do still share the sparse rational arithmetic layer, so their independence is not complete.

Search may evaluate the polynomial at a midpoint to abandon a hopeless box early, but it must never return that evaluation as a refutation. Some implementations do return a checked refutation: an exact point inside the box at which the polynomial is negative, validated independently for membership and sign. That is a genuine three-valued outcome — PROVED/REFUTED/UNKNOWN — and it is the only sound negative route any of the prototypes implements.

Split-axis policy matters for the completeness argument below. Cycling axes as depth mod n , or choosing the largest *relative* width, both ensure every coordinate shrinks along a branch; choosing the largest absolute width introduces a coordinate-unit bias.

A qualified completeness result, and a sharp obstruction

Proposition 6.4 (Strictly positive polynomials). *For a polynomial strictly positive on a compact box, a fair subdivision search that shrinks every coordinate along each branch eventually makes all Bernstein coefficients positive.*

Justification. The transformed higher-degree coefficients tend uniformly to zero as box widths shrink, while the constant terms stay above a positive minimum: each local Bernstein coefficient is $p(a) + O(\max_i(b_i - a_i))$ with a bound uniform over the original box. Compactness supplies the positive minimum. $\square$

This does not give a useful bound for every input and does not justify ignoring the configured caps.

Zeros behave completely differently, and the difference is a theorem rather than a matter of search depth.

Theorem 6.5 (Interior-zero obstruction). *Let p be a nonzero polynomial with a zero in the interior of a box B . Then p has no all-nonnegative Bernstein expansion on B .*

Proof. Every Bernstein basis function is strictly positive at an interior point. If all coefficients were nonnegative, a zero value at an interior point would force every coefficient to vanish, hence $p \equiv 0$ on B and so $p \equiv 0$. $\square$

Consequently a finite dyadic subdivision cannot certify $(x - 1/3)^2$ on $[-1, 1]$ by nonnegative leaf coefficients alone: some leaf must contain $1/3$ in its interior. The obstruction is quantitative. On $[0, 1]$, the dyadic cell containing $1/3$ always has the root in its interior, and if its endpoints are $\ell < u$ then the middle quadratic Bernstein coefficient is exactly

$$(\ell - \text{frac}13)(u - \text{frac}13) < 0.$$

No finite midpoint-only subdivision can certify that cell by nonnegative degree-two coefficients, whereas the exact quadratic engine of §fsec:quadratic proves the same inequality immediately. This is a concrete reason to cooperate across representations rather than escalate one representation’s resource cap indefinitely. The same obstruction applies with an *irrational* zero, and there it rules out any rational-box tree whatever: $(x^2 - 2)^2$ on $[1, 2]$ has its zero at $\sqrt{2}$, which no rational split point can ever fall on. The prototypes record both as expected UNKNOWN outcomes, at depth 12 and at their depth limits respectively.

A second, milder limitation is worth recording: $(x - y)^2$ on $[0, 1]^2$ returns UNKNOWN at depth four even though a single square certificate proves it immediately. Again this routes rather than stops.

Worked instances

For $p(x) = (x - 1/2)^2 + 1/100$ on $[0, 1]$, the degree-two coefficients at the root are $13/50, -6/25, 13/50$, so the root check cannot certify positivity. Splitting at $1/2$ gives

$$(13/50, 1/100, 1/100) \quad \text{and} \quad (1/100, 1/100, 13/50),$$

so two leaves suffice for strict positivity. For $p(x) = x^2 - x + 1/3$ on $[0, 1]$ the root coefficients are $(1/3, -1/6, 1/3)$, and splitting at $1/2$ gives $(1/3, 1/12, 1/12)$ and $(1/12, 1/12, 1/3)$. For $p(x) = x^2 - x + 3/10$ the split yields two leaves each with exact lower bound $1/20 > 0$ and a three-node certificate. These are exact calculations, not numerical plots.

A multivariate instance uses the bounded Motzkin polynomial $M(x, y) = x^4y^2 + x^2y^4 - 3x^2y^2 + 1$. The search certifies $M + 1/16$ on three domains: on $[-1, 1]^2$ a single leaf suffices, with minimum coefficient exactly $1/16$; on $[-2, 2]^2$ it constructs 32 leaves at maximum depth six; on $[-3, 3]^2$, 56 leaves at maximum depth nine.

Remark (An instructive correction). In one development run the first test wrongly expected the smallest domain to require subdivision. The exact coefficient calculation showed the test assumption was wrong. The corruption test was moved to the larger domain and independent symbolic reconstruction tests were added. No positivity condition was weakened to make the test pass. That is the correct response to a failing test in this setting, and it is recorded here because the opposite response is the standard way such a project becomes unsound.

Cost

The tensor has $\prod_i (d_i + 1)$ coefficients and a binary tree can have exponentially many leaves in its depth bound; exact coefficient bit sizes also grow under repeated substitution. The method suits moderate dimension, low coordinatewise degree, and meaningful compact bounds. Implementations cap tensor size before expansion precisely because an adversarial certificate could otherwise request an enormous one. This is attractive research code, not a complete defence against hostile memory-exhaustion input, and not a universal substitute for nonlinear arithmetic.

6.8 Univariate certificates that preserve zeros

Status: Implemented and tested in Python by one effort. No generic Lean Sturm replayer exists.

Theorem 6.5 is not merely a caveat; it is a capability gap with a constructive answer. This worker is the answer, and it is the only mechanism in the whole collection that can certify a polynomial with an interior zero.

Square-factor reduction

For a nonzero rational univariate polynomial, extract even multiplicities:

$$p(x) = q(x)^2 r(x),$$

where r is square-free and its rational scalar coefficient retains the original sign. Full factorisation is not necessary in principle — square-free decomposition suffices. The prototype uses a rational factorisation interface as an *oracle* and then verifies the resulting identity with its own exact representation [64].

A certificate on a rational interval $[a, b]$, $a < b$, consists of q , r , and a Sturm chain for r . The checker requires:

1. the exact identity $p = q^2r$, and a valid terminating signed-remainder chain for r ;
2. no roots of r in the *open* interval (a, b) ;
3. $r((a + b)/2) > 0$ and $p(a), p(b) \geq 0$.

The zero polynomial is handled by $q = 0, r = 1$.

The checked Sturm chain

The chain starts with $r_0 = r, r_1 = r'$, and continues with $r_{i+1} = -\text{rem}(r_{i-1}, r_i)$ until the next remainder is zero; a constant r has a one-element chain. The checker recomputes the derivatives and rational remainders, requires exact equality with the supplied chain, and requires the last element to be a nonzero constant — which establishes that r is square-free. Crucially it *recomputes the entire chain* rather than trusting a stored root count.

Let $V(t)$ denote the number of sign changes in the chain. Endpoint roots are the easy thing to get wrong here, so the implementation computes *one-sided* signs: at $a+$, the sign of a chain polynomial is the sign of its first nonzero derivative at a ; at $b-$, that sign is multiplied by $(-1)^k$ when the first nonzero derivative has order k . The open-interval root count is then

$$N_{(a,b)}(r) = V(a+) - V(b-).$$

Independent tests compare root counts against a symbolic oracle and explicitly exercise rational roots at both endpoints.

Theorem 6.6 (Univariate interval certificate). *A certificate satisfying the three conditions above proves $p(x) \geq 0$ for every $x \in [a, b]$.*

Proof. The Sturm count establishes that r has no root in (a, b) . By continuity its sign is constant there, and the positive midpoint value fixes that sign as positive. Hence $q^2r \geq 0$ on the interior. The exact identity and the two endpoint checks complete the proof on the closed interval. $\square$

Proposition 6.7 (Fragment completeness). *Ignoring degree and resource caps, exact square-free decomposition together with this check is complete for nonnegativity of rational univariate polynomials on a rational closed interval.*

Proof. If a nonzero $p = q^2r$ is nonnegative, a simple interior root of the square-free r would change the sign of p , which is impossible. Thus r has no interior root and is positive there, and the certificate theorem applies. $\square$

This argument uses the mathematical Sturm theorem. Its use is justified in the algorithm specification and exercised by tests, but it is *not* formalised in Lean anywhere in this collection. A production Lean implementation must prove or integrate that theorem and check the instantiated certificate before this worker can be trusted; that is a substantial verification task and must not be hidden behind an unchecked foreign call.

Why it belongs alongside Bernstein search

$(x - 1/3)^2$ is handled without aligning a dyadic split with its zero, and so is $(x^2 - 2)^2$, whose real zeros are irrational. A constructed test family multiplies rational and irrational even-root factors by a positive quadratic and the endpoint factors $(x + 2)(2 - x)$ on $[-2, 2]$. All 30 instances are certified by the square-factor/Sturm worker; *none* is certified by the depth-12 Bernstein-only ablation.

That is a genuine component-level distinction. It does not imply these examples are difficult for an existing Lean tactic once a factorisation is supplied. The contribution is an explicit search-and-check route for *finding* and *validating* the relevant algebraic structure.

In several variables, candidate factorisation, algebraic equality reasoning, and constraint-relative certificates can play an analogous role. A generic multivariate zero-set treatment is not part of any prototype.

6.9 Cheap facts for other engines: certified product envelopes

Status: Specified; not separately implemented.

Bound propagation can generate inexpensive facts before any larger nonlinear search runs. If $l_x \leq x \leq u_x$, $l_y \leq y \leq u_y$, and $z = xy$, then the four products of nonnegative bound differences give

$$\begin{aligned} z &\geq l_x y + l_y x - l_x l_y, & z &\geq u_x y + u_y x - u_x u_y, \\ z &\leq u_x y + l_y x - u_x l_y, & z &\leq l_x y + u_y x - l_x u_y. \end{aligned}$$

The first, for instance, is the expansion of $(x - l_x)(y - l_y) \geq 0$. These facts are *linear* in x, y, z and can be consumed immediately by the existing arithmetic engine; their derivations, bounds, and the equation $z = xy$ must accompany them.

This worker should be prioritised because its certificates are small and its output is useful across theories, and it has a natural trigger: run only when the bounds of a product operand improve. Conversely, an extracted alternative form such as $(x - y)^2 + 2(xy - 1)$ can become a positivity candidate for $x^2 + y^2 - 2$, and a cone certificate can request a particular difference term $x - y$ as a guide for other matching tasks. What is shared between views is a *proved relation*, never a mutable polynomial whose meaning changes during search.

6.10 A complete cross-theory proof shape

Suppose $p = x^2 - 2xy + y^2$ with hypothesis $p \leq 0$, and the target is $f(p) = f(0)$ for an otherwise uninterpreted $f : \mathbb{R} \rightarrow \mathbb{R}$. The nonlinear worker discovers $p = (x - y)^2$, proves $0 \leq p$, and exports that inequality; antisymmetry gives $p = 0$; congruence closure gives the target. No worker needs a theory of f , and no single worker solves the whole problem.

This is the central integration pattern of the whole design. It is not an assertion that stock `grind` fails on this small example.

6.11 Domains, casts, and division

The controller must establish that the source assumptions imply the box bounds before applying a Bernstein theorem, and polynomial reification must preserve variable identity, source operations, and casts. A real-field certificate cannot be applied to an arbitrary ring or a field of finite characteristic.

For natural or integer inequalities, prove the embedding into an ordered field explicitly and translate the relevant operations. Integer division, truncated subtraction, and modular arithmetic are not field operations. Translating `(a - b : Nat)` to a field difference requires a guard such as $b \leq a$ and an appropriate transport theorem; otherwise the operation stays opaque or the relevant cases are split.

Unknown real-valued applications can be polynomial atoms without being uninterpreted everywhere. One engine may know a theorem about $f(x)$ while the polynomial engine treats its value as a single coordinate; once that theorem yields a bound or equality, the bound is proved in the original context and can then be translated into the polynomial representation. No global erasure of dependent types is necessary.

Finally, if congruence closure identifies two source terms, a worker may share one polynomial variable only after preserving the equality proof that justifies the identification. It must not merge different variables because their printed names agree. Treating equal terms as distinct opaque variables is a loss of proving power rather than an unsound strengthening — which is precisely why the shared-state integration is worth the engineering.

7 Boolean search, equality exploration, and external engines

7.1 A separate lane before a core-engine rewrite

Large combinatorial Boolean search is not what a congruence-closure and theory-propagation engine is designed for, and the Lean reference says as much: it distinguishes `grind`'s branching

strategy from bulk Boolean solving and recommends `bv_decide` for the latter [1]. These are complementary roles. Nothing here is evidence that `grind` is a weak simplifier, and the existing shared-state `grind => bv_decide` route already encodes equivalence classes and theory-learned facts into the SAT problem [8].

The proposal is therefore a *separate lane* with an explicit proof contract, scheduled after structural steps have made the relevant facts available, and implemented late. Every roadmap in this collection places the Boolean adapter last, and that ordering should be kept even though it is the component with the most complete prototype.

7.2 A proof-producing SAT/theory loop

The coarse-grained version needs no changes to any existing engine:

```
while budget remains:
    answer := SAT.solve(currentClauses)
    if answer is UNSAT:
        replay the propositional certificate and all clause origins
        lift the contradiction to the original Lean goal
        return PROVED only if replay and the axiom audit succeed
    if answer is not a complete Boolean assignment:
        return UNKNOWN
    open a temporary scope with the assignment's signed atoms
    ask the relevant Lean theory engines for a contradiction
    if a contradiction is proved:
        discharge its used decisions into a learned clause
        add the proved clause; close the temporary scope
    else:
        refine the supported abstraction or return UNKNOWN
```

Incremental CDCL(T) is the later optimisation. When a theory implies a literal L from current literals $A_1, \dots, A_k$, it must return a proof of the clause $\neg A_1 \vee \dots \vee \neg A_k \vee L$. A stale explanation referring to a withdrawn equality is a correctness bug even when the Boolean clause still looks plausible.

7.3 An implemented CDCL(T) with checkable learning

Status: Implemented and tested in Python by two independent efforts, with two different proof formats.

The implemented theory is integer difference logic: atoms $x - y \leq c$ over integer variables, with Boolean structure in CNF. The solver performs unit propagation to a Boolean fixed point, then tests the implied difference constraints for a negative cycle; a negative cycle yields a theory lemma. First-UIP conflict analysis and non-chronological backjumping are implemented, and tautological resolvents are detected during analysis.

The integer-negation trap. The negation of $x - y \leq c$ over $\mathbb{Z}$ is $x - y \geq c + 1$, that is $y - x \leq -c - 1$ — *not* $y - x \leq -c$. Getting this wrong produces a solver that is subtly unsound on exactly the instances it is meant to decide. Two defences are worth copying. The consistency oracle carries a dedicated regression assertion at the complement boundary, tested at magnitudes such as 10^{60} to confirm that arbitrary-precision bounds survive; and the certificate checker *duplicates* the negation rule rather than calling the solver's shared edge constructor, so that a bug in one copy cannot cancel itself out in the other. For the same reason the all-pairs consistency check uses explicit `None` sentinels for absent edges instead of a finite infinity.

Two certificate formats. One implementation emits a *resolution DAG*: nodes tagged `input`, `theory`, or `resolution`, each resolution node carrying parent indices and a pivot. Replay is purely structural and cheap — it re-does the solver's own steps, which makes it fast but means the checker trusts the solver's choice of derivation.

The other emits *RUP additions plus signed negative-cycle lemmas*, and its checker re-derives each added clause by *independent* unit propagation. It never trusts the solver’s resolution chain. That independence is the stronger property, and it is why a merged implementation should offer the resolution DAG as an optional second mode — strictly cheaper to check when available — while keeping RUP as the default.

A theory lemma is validated by exact coefficient summation: the cycle’s coefficients telescope, and the resulting constant bound must be strictly negative. A satisfying assignment is validated separately: the Boolean assignment must satisfy every original clause, *and* each atom’s truth value must match the integer model.

Proposition 7.1 (Decision procedure for the implemented fragment). *For a fixed finite set of integer difference atoms and a CNF over them, exhaustive Boolean search together with a complete difference-logic consistency check is a decision procedure.*

This does not extend automatically to the other theories or the quantified goals Forge targets, and it is stated only to fix what the implemented fragment does.

Production requirements. A Lean port needs the clause origins bound to actual Lean propositions, a supported certificate format, and a declared trust profile. Existing work is directly relevant: `bv_check` already demonstrates the useful separation of a stored SAT certificate from an installed solver, and PBLearn shows that existing Lean SAT integrations make an explicit compiled-reflection choice [23, 38]. The classical DPLL(T) framework supplies the architecture; the contribution proposed here is an integration contract with native Lean terms, not the algorithm [57].

Remark (A benchmark fairness rule). A comparison must not measure strict kernel replay in one tactic against native replay in another and report them as identical trust configurations. Caching an LRAT file for `bv_check` does not by itself establish a kernel-only replay profile.

7.4 Bounded equational exploration

Equality saturation should be used *selectively*, because `grind` already supplies congruence machinery. The proposed extension maintains small collections of alternative forms for terms relevant to a blocked obligation, prioritising forms that expose recursive calls, match a theorem conclusion, or reduce an arithmetic certificate’s dimension. Rewrites are justified by equality proofs; conditional rewrites create explicit guards, and if a conditional rule depends on an unproved proposition, its output belongs in the obligation graph rather than in an unconditional equality class.

Do not saturate associativity, commutativity, and distributivity indiscriminately: existing normalisers already handle much of that structure more efficiently. Use algebraic normal forms for certificate checking while retaining a few factored or recursive views for search, and limit term count, expansion degree, and rewrite generations.

Extraction deserves more care than it usually receives. At each fuel boundary, extract *several* candidate representatives rather than a single syntactically cheapest term: one may have the fewest nodes, another the simplest arithmetic degree, another the best overlap with a desired lemma. A practical cost vector is lexicographic — unsupported constructs; unsolved side conditions; maximum polynomial degree or recursive mismatch count; expression size; estimated replay size — and different obligation kinds lead with different coordinates. Cyclic equivalence classes require a well-founded extraction procedure, because a cycle is not a finite Lean term.

An e-class identifier is meaningful only within a state lineage and revision. A proof node must not depend on a mutable representative number that does not survive rollback. The binder-sensitivity and dependent-transport obligations involved in doing this over real Lean expressions are documented elsewhere [46, 47], and the historical `lean-egg` package is deprecated in favour of `grind` [45]; an archived experimental project should not become a critical dependency because its name sounds aligned with the design.

7.5 Higher-order structure and external provers

Before translating a goal to a first-order backend, expose its constructive structure. For a function equality, attempt the relevant extensionality principle and introduce a fresh argument. For a universally quantified goal, introduce its binder. For a conjunction or constructor target, create the corresponding subgoals. For an existential, construct a scoped witness and then prove its specification.

A bounded type-directed term synthesiser handles witness goals beyond the affine fragment. Its grammar includes local terms, lambda abstraction, application of retrieved lemmas and functions, constructors, and selected recursor or fold schemas, with every candidate elaborated before semantic checking. Higher-order pattern unification handles tractable application patterns; unrestricted higher-order unification is not assumed to be a cheap or complete subroutine.

To synthesise $h : \text{List}(A) \rightarrow \text{List}(B)$ with $h([]) = []$ and $h(x :: xs) = f(x) :: h(xs)$, a list-fold schema already fixes the recursion structure, so search needs only the empty case and the constructor step; a library index may retrieve `map`, and otherwise a recursor term provides an equivalent witness whose equations hold by reduction. Canonical is the relevant prior work on inhabitation and unification in dependent type theory, and it should be integrated through its proof-term interface rather than by flattening away the dependencies that make its task meaningful [54, 55].

Instance obligations during synthesis

§8 takes the position that instance arguments should be left implicit and resolved by the elaborator during verification, and for a first implementation that is right: the elaborator is a better evidence resolver than anything one would port. But a synthesiser that defers *everything* prunes nothing, and one that prunes on instances needs its own account of them. Four disciplines make that account safe, and they generalise well beyond typeclasses.

One function, two policies. The routine that prunes a branch during search and the routine that validates the final term must be the *same* code, parameterised by what to do with an obligation that is not yet ground. During search, a constraint containing free variables means *defer, keep going*; during final checking it means *reject*. Writing these as two functions is the classic way a prover admits something its own checker would refuse.

Do the termination work at admission, not at query time. Seal the inventory of instances *once*, and at that point reject superclass cycles, reject overlapping heads, materialise superclass closures into each instance’s prerequisites, and enforce a structural decrease condition on prerequisites — a prerequisite is admissible if it mentions a variable absent from the head, or if it is no larger than the head and no variable occurs more often. Then query-time resolution needs only a path-local cycle cut and a node budget, and termination is a property of the admitted environment rather than a hope about the search. *Forge* can impose such a gate over the subset of instances it admits into a synthesis inventory without touching the ambient environment.

Budget exhausted is not “no instance”. The result type should be an error-or-optional, not a plain optional: malformed input and exhausted budget on one side, genuine absence of evidence on the other. Collapsing them means an instance obligation that timed out looks like a refutation of the candidate — the same category error as §5.5’s `exhausted` versus `truncated`, one level down.

A resolution receipt, bound to its environment. Where a receipt is wanted, the natural object is the derivation tree: goal, the instance head selected, and one sub-proof per discharged prerequisite. The discipline that makes it safe is that the receipt carries the sealed environment it

was derived against, and authority comes from *replaying* it against that environment and that goal — environment identity checked before the goal is even inspected — rather than from possessing the tree. This matters precisely because a synthesised term’s instance arguments must ultimately be re-elaborated in the user’s local context, not the synthesiser’s snapshot.

One further idea is worth stealing outright, because it is a single line with real consequences: make a class obligation a proposition that is *structurally disjoint* from every ordinary proposition in the search language. A dictionary requirement is then not confusable with a value whose type happens to print the same way, an inhabitant of $C\ a \Rightarrow \text{Unit}$ cannot discharge $C\ a$, and the incompleteness of the dictionary-free fragment becomes mechanically detectable rather than a matter of care.

External provers

After structural exposure, a saturation-based prover may find theorem combinations a local tactic misses. `Duper` is an in-Lean proof-producing option [26]; `lean-auto` supplies translation and monomorphization infrastructure [27]; `lean-smt` and `QuerySMT` supply SMT search with Lean reconstruction [28, 29]. Two operational points: `duper` `[*]` requests local facts, so a bare `duper` should not be assumed to consume them automatically — pass an explicit recorded premise slice. And `lean-auto` distinguishes reconstruction-capable paths from standalone invocation modes *without* reconstruction, so an external `unsat` is not a uniform proof interface.

The controller must not send every declaration in the local environment. Select the dependency-relevant facts, instantiate polymorphic schemas only at the types the current demand requires, and record which source theorem and which universe/type instantiation each exported formula came from, so that reconstruction applies the actual theorem.

A useful ordering for reconstruction, from most to least preferred: an existing Lean tactic on the original expressions; a restricted, proved translation with a known replay path; externally proposed hints replayed on the original expressions; and finally a full external proof import with explicit supported rules. `lean-smt`’s documented behaviour includes returning proof-reconstruction holes as Lean goals; those holes enter the obligation graph as children of the current AND node, and an external `unsat` response does not bypass them. Partial progress is useful; it is not completed verification.

7.6 Solver islands and the limits of theory combination

A *solver island* is a selected subproblem whose translation and proof reconstruction are supported end to end. The island may prove a lemma over shared interface terms rather than the complete original goal. For an island under assumptions $A_1, \dots, A_k$, reconstruct a proof of $A_1 \rightarrow \dots \rightarrow A_k \rightarrow L$ and apply it to the appropriate scoped proofs to insert L into the main graph.

Every export carries a *translation ledger*: the Lean expressions, their solver symbols, their types and operations, and the proof obligations connecting the encoding to the original proposition. Universe instantiation, monomorphization, typeclass resolution, and auxiliary assertions must all have justified transformations.

Abstraction has a direction, and both directions must be stated. Replacing a difficult subterm by a fresh atom is sound for proving a *universal consequence* of the abstraction: a proof valid for every assignment of those atoms is valid for the actual expressions. But an abstract satisfying assignment can be spurious and is not a counterexample to the Lean goal. Conversely, erasing dependencies in an indexed datatype is *not* automatically a sound abstraction; use an explicit bridge or leave that expression in the native layer.

Remark (No blanket combination theorem). Do not assume a generic Nelson–Oppen completeness result merely because solvers exchange equalities. Finite datatypes and fixed-width bit-vectors do not satisfy the usual stably-infinite assumptions. `Forge` avoids relying on a blanket combination theorem: plugins exchange *proved interface facts*. An arithmetic atom abbreviating xy must carry

a definition or equality proof; treating it as an unrelated real variable yields a relaxation, not a bidirectional equivalence of problems.

7.7 Candidate generation can be untrusted without being unconstrained

An optional model, learned ranker, or language model may propose a lemma, an induction variable, a proof sketch, or a term. It cannot change the accepted logical rules, and it cannot submit arbitrary Lean commands that introduce axioms or alter the environment under the guise of a proof term. Restrict the import channel to a typed certificate schema or to a term elaborated inside a fixed environment, with resource and size limits protecting the checker from pathological proposals. Never trust a textual theorem statement supplied by a model.

Neural ranking is optional and is never proof evidence. The design does not require an online model service: the executed prototypes use deterministic enumeration, exact algebra, and local LP proposals, and their algorithms remain useful in a fully offline implementation.

Finally, isolation is a separate concern from soundness. External search should run in bounded child processes without network access — not because a wrong answer would be accepted, but because a foreign-function boundary is a memory-safety surface and an unbounded search is a denial-of-service surface. Neither is a soundness failure; both make a tactic unusable.

8 Lean integration: a conservative bridge before an invasive one

Status: Integration specification. The cited core names were inspected at tag v4.34.0; every adapter interface below is proposed and none has been compiled.

8.1 Begin at a stable boundary

The first implementation should invoke `grind` on ordinary Lean goals and supply newly proved local facts explicitly. This is less efficient than an ideal native integration, but it lets the structural planner and the certificate pipeline be tested without depending immediately on every internal representation.

There are concrete lower-level entry points to investigate next. In the inspected v4.34.0 source, `Lean.Meta.Grind.Params` carries extensions, extra theorems, extra facts, simplification data, and configuration; `assertExtra` introduces supplied proof facts through `addNewRawFact`; `mkGoalCore` initialises a goal and its solver states; and the result structure carries failure information and counters [12]. `Lean/Meta/Tactic/Grind/Types.lean` exposes `GrindM`, `GoalM`, opaque solver-extension state, and a `SavedState` containing both Meta and grind state, whose restore operation restores both components [11]. There is also a registration path for solver extensions, whose methods cover internalisation, new-equality and new-disequality notification, an action, consistency checks, and a model-combination hook [17].

A second inspection of the same release found a more specific surface: a registration function `registerSolverExtension` in the `Lean.Meta.Grind` namespace, together with `SolverExtension.setMethods`, whose callbacks are `internalize`, `newEq`, `newDiseq`, `mbtc`, `action`, `check` and `checkInv`; registration happens at initialisation time and extension state is per goal. The API also distinguishes `Solvers.mkActionCore` from the wrapper that drains pending facts [17]. A polynomial adapter would mark relevant expressions during internalisation, mark a component dirty after an equality, perform bounded certificate search on the expensive-check path, and publish its accepted output as a proved fact through the ordinary fact machinery. Note that the internal-invariant check in that interface is a check of data-structure bookkeeping; it is not the mathematical soundness argument for a certificate.

These are implementation hooks, not a promise of a stable public integration contract, and two qualifications matter.

First, feeding `Params.extraFacts` into a *fresh* run is not the same as resuming an earlier saturation state. A true incremental adapter must preserve the goal state and all relevant solver and context state, and must add facts through the normal internalisation and propagation path. Otherwise an equality can exist in one table while being invisible to another solver.

Second, restoring a Lean metavariable snapshot alone is not enough. *Forge* needs its own rollback layer for the additional clause-database views, demand queues, proof dependencies, and branch-local plugin state that it introduces. `SavedState` is necessary, not sufficient.

A proposed versioned adapter surface:

<code>GrindAdapter.open</code>	<code>(snapshot, options)</code>	<code>-> session</code>
<code>GrindAdapter.seed</code>	<code>(session, checkedProofs)</code>	<code>-> session</code>
<code>GrindAdapter.advance</code>	<code>(session, workQuota)</code>	<code>-> localResult</code>
<code>GrindAdapter.exportRelevant</code>	<code>(session, demand)</code>	<code>-> checkedSummary</code>
<code>GrindAdapter.snapshot</code>	<code>(session)</code>	<code>-> resumableState</code>
<code>GrindAdapter.restore</code>	<code>(resumableState)</code>	<code>-> session</code>

All version-specific internals live in one compatibility module per pinned Lean version — for example `Forge.Bridge.GrindV434`. Theory adapters and the outer scheduler target *Forge*’s own records rather than importing dozens of evolving internal modules. Unit-test seeded facts, case-split rollback, proof extraction, metavariable changes, and equality propagation through every enabled local theory. Until that adapter is reliable, restarting a local tactic with explicit checked facts is preferable to a fast but incoherent shared state.

8.2 The tactic is the interface; it need not be the executor

§8 so far has assumed one axis of choice: how deeply the tactic reaches into *grind*’s internals. There is a second axis, and it is arguably more consequential — *where the search runs*.

The obvious arrangement is that a native tactic performs the search in the Lean server process. That has real advantages: exact access to the local context, open namespaces, options and reducibility settings; direct return of an `Expr` with no serialisation; natural integration with the language server, tactic traces and source positions; automatic compilation against the project’s own toolchain; and invocation at an actual proof hole rather than at a separate prompt. Any usable front end wants all of that.

But running the search there carries costs that a long-running planner feels much more than an ordinary tactic does:

- a long or allocating search competes with elaboration and editor responsiveness;
- cooperative cancellation only works if the search polls often enough;
- a crash, an FFI failure, or a memory blow-up takes down the Lean server process;
- keeping expensive search sessions alive *across commands* requires environment extensions or external state whose lifecycle has to cooperate with incremental recompilation.

The last one deserves attention, because a persistent obligation graph spanning a whole proof is exactly the thing this design wants and exactly the thing incremental recompilation is entitled to discard.

The resolution is to keep the native tactic as the *interface* while moving execution into a project-local worker:

- a **controller** — the tactic front end, or a standalone session — owns user intent, settings, history, protocol framing, deadlines, process supervision and presentation. It holds *no authority to declare a candidate well-typed, a goal unprovable, or a solver observation valid*;
- a **worker**, compiled under the target project’s Lake environment, owns the `Environment`, elaboration, exact identities, reification, search, lowering to `Expr`, and kernel checking;
- an optional **solver process** receives only a canonical query and returns bounded observations.

The tactic then prepares the exact goal, hands a *pure prepared problem* to the worker, and kernel-checks the returned expression in the server. A worker crash loses process-local state only; the controller rebuilds from an import root and a replay log.

This also answers a question §4 raised and did not settle: where the sandboxing boundary should be. It is not primarily about soundness — an exact certificate rejects a bad proposal wherever it was produced — but about not letting a numerical search take the editor with it.

8.3 Three levels of identity

§3 required scope keys to carry expression, type, universe levels, instance identity, context fingerprint and scope, and warned that printed text is not a safe key. That is right but under-specified: it treats identity as one thing with several fields. It is better understood as three distinct kinds of identity with different lifetimes and different authority.

1. **Lean identity** — exact `Name`, local declaration identity, environment generation, source `Expr`. Valid only inside one worker, one toolchain, one session generation.
2. **Neutral semantic identity** — stable variable, global, constructor and provider IDs in a checked inventory. Serialisable, fingerprintable, independent of elaborator allocation.
3. **Presentation identity** — delaborated syntax, qualified spelling, candidate ordinal, rendering route. *Never sufficient to recover an authority.*

A mapping from Lean identity to neutral identity is built during preparation and kept in a private origin table; lowering uses the inverse mapping only *after* the neutral candidate has passed its checker; presentation is derived last. The rule that falls out is sharper than anything in §4’s protocol discussion:

Invariant 14 (The origin table never leaves the worker). An external process receives a sealed neutral problem and returns a neutral candidate. No raw `Expr`, environment pointer, free-variable identifier, process-local handle or certificate carrier crosses that boundary. Origin resolution and kernel checking remain with the worker.

Environment generations make rollback safe in a way cache invalidation does not. Attach a monotonically fresh generation token to every committed environment state and index every cache, prepared origin, candidate token and inventory by it. Undo then restores an older environment *object* while issuing a *new* generation token, so stale capabilities cannot be reused by accident — which is a better answer to the stale-state failure mode of §4 than “invalidate the affected caches”, because it fails closed rather than depending on the invalidation being complete.

Durable snapshots, correspondingly, retain project and toolchain identity, root imports, schema fingerprints, replay history and settings — and retain no raw `Expr`, no `FVarId`, no environment pointer, no process handle, no candidate token, and no certificate carrier.

8.4 Do not search in `Expr`

A related temptation is to drop the typed view entirely and search directly in Lean’s expression and metavariable machinery, on the grounds that this buys definitional equality and typeclass synthesis for free. For a *tactic* step that is right. For a bounded, accounted, certificate-producing *search* it costs several properties this design depends on:

- a small, auditable fragment with an explicit completeness ledger (§5.5);
- stable alpha-normal fingerprints independent of elaborator allocation identifiers;
- deterministic serialisation, and differential testing against an independent implementation;
- independent checking, rather than replaying the search’s own mutations;

- exact accounting of steps, choices, pruning and plans — without which **exhausted** cannot be distinguished from **truncated**;
- the ability to keep an unsupported dependent region opaque without it being normalised away under an aggressive reducibility setting;
- decoupling from changes in Lean’s metavariable implementation.

The metavariable context is excellent for final elaboration and for local tactic work, and awkward as an authoritative branch representation: it is optimised for elaborator workflows and is hard to serialise, fingerprint, compare, replay and branch under an exact resource policy. The recommendation is a Lean-oriented *neutral* IR that may reference **Names**, binder kinds and exact declaration handles at the worker boundary, while owning its own recursive structure and substitution rules — and a pure branch-state model for search, with **Expr** entering only at lowering.

Remark (Deduplication can discard the authority). One consequence is worth spelling out because it is a silent failure. If candidates are deduplicated by a lossy key — pretty-printed text, alpha-equivalence, eta, reducible unfolding, proof irrelevance — a container keyed by that equality can discard the representative that carries the checked evidence and keep a look-alike that does not. Finish ranking and display grouping over explicit keys, keep a separate pointer to the exact origin, and do not define equality or ordering on the opaque authority merely for convenience. Exact normalised neutral-tree identity is the safe default; text remains useful as the user’s observed grouping key and nothing more.

8.5 A typed semantic boundary for each worker

A reifier does more than turn Lean expressions into strings. For a polynomial fragment it produces a variable table, an exact polynomial, and a proof that the polynomial’s denotation under that table is the source expression:

$$\text{reify}(e) = (\text{environment } \rho, \text{ IR } t, \pi : e = \text{eval}_\rho(t)).$$

For an inequality or a formula the contract is an equivalence or a suitable one-way implication, and the direction must match how the result will be used. Unsupported subexpressions may become opaque atoms when that abstraction is sound for the requested procedure.

An algebraically correct certificate for the wrong denotation is not a proof of the original statement. Equality classes must be homogeneous up to justified type compatibility; heterogeneous equality is not an excuse to ignore type indices, so keep **HEq** reasoning and transport explicit and use elaboration to validate casts. A proof of $i = j$ does not make **Vector A i** and **Vector A j** definitionally interchangeable, and they must not be identified because their indices were merged in an untyped side representation. Typeclass dictionaries and universe instantiations are part of the elaborated expression, not metadata that can always be erased.

An initial external term language should remain first-order and homogeneous rather than implementing an ad hoc untyped approximation of all of Lean.

8.6 Three levels of proof reconstruction

1. **Ordinary theorem applications and tactic steps.** Appropriate for synthesised witnesses, explicit induction motives, small nonnegative linear combinations, and recurrence identities. The search can be complicated while the replay stays short.
2. **A typed certificate to a reflected checker**, with a theorem

$$\text{check}(P, C) = \text{true} \implies \text{Denotes}(P).$$

The problem P comes from the source reifier. The external solver may choose only C ; it may never replace P . In a conservative configuration, reduce the checker inside the trusted proof-checking path rather than accepting an unchecked foreign result.

3. External proof import with explicit supported rules, every unsupported step becoming an obligation.

For larger algebraic results the second level becomes worthwhile once its soundness theorem exists; below that threshold, reconstructing small certificates with `ring`, `linarith`, `positivity`, `norm_num` and cast normalisation avoids introducing a second independent arithmetic proof language prematurely [43, 40, 42, 44].

The mixed bit-vector case should use the existing shared-state `grind => bv_decide` route where applicable rather than duplicating its encoding to fit a uniform architecture [8].

8.7 A proposed module layout

The following paths are an implementation plan. No file claimed here exists in this repository.

```
Forge/Frontend.lean          -- syntax, configuration, diagnostics
Forge/Core/GoalSnapshot.lean -- restorable state identity
Forge/Core/Obligation.lean   -- AND/OR graph and proof assembly
Forge/Core/CheckedFact.lean  -- scoped propositions, dependencies
Forge/Core/Scheduler.lean    -- deterministic quotas and budgets
Forge/Core/Replay.lean       -- minimization and axiom audit

Forge/Bridge/GrindV434.lean   -- ALL version-pinned internals
Forge/Reify/Polynomial.lean   -- typed views and denotation proofs

Forge/Structural/CallAnalysis.lean
Forge/Structural/Generalize.lean
Forge/Structural/LemmaFrontier.lean
Forge/Structural/Witness.lean

Forge/Arithmetic/Cone.lean    -- cone semantics and exact replay
Forge/Arithmetic/Quadratic.lean
Forge/Arithmetic/Bernstein.lean
Forge/Arithmetic/Sturm.lean   -- needs a proved root-count theorem
Forge/Arithmetic/Lattice.lean

Forge/Instantiation/ConstrainedMatch.lean
Forge/Instantiation/ModelGuided.lean

Forge/Backend/SOS.lean        -- adapter to the existing sos project
Forge/Backend/LP.lean         -- adapter to the existing lp project
Forge/Backend/SMT.lean
Forge/Oracle/Protocol.lean    -- bounded certificate data, never code
Forge/Tactic.lean
```

The core modules depend only on Lean and a small shared representation. Mathlib-specific reifiers and tactics sit behind separate imports. External processes consume a versioned certificate format and return only untrusted proposals, and the package remains useful when those processes are absent.

That last property is worth stating as a design goal rather than an accident: the separation between core, checking, and external search permits *replay-only builds*. A numerical solver can be absent on the machine that checks a finished proof.

Module	Responsibility	Acceptance contract
<code>Forge.Core</code>	Scoped expression, fact, and obligation graph	No unproved facts; valid dependency scopes
<code>Forge.Bridge</code>	Version-specific extension hooks	Restorable state; exact expression and proof transport

Module	Responsibility	Acceptance contract
Forge.Demand	Retrieval, instantiation, tabling	Every application elaborates; acyclic proof parents
Forge.Reify	Typed polynomial representation	Denotation equality for each reified term
Forge.Arithmetic	Cone, box, lattice certificate checking	Proved checker soundness, or explicit proof reconstruction
Forge.Structural	Recursor selection and generalisation	Goal refinement through an actual induction theorem
Forge.Synth	Invariants and existential candidates	Base/step identities or witness specification discharged
Forge.Backend	SAT, SMT, SOS, LP, superposition adapters	Supported translation and proof rules; no raw oracle acceptance
Forge.Replay	Proof minimisation and audit	Exact original goal, no remaining metavariables, approved axioms

Proposition 8.1 (Compositional soundness is not inherited). *Soundness of the assembled system requires soundness at every layer simultaneously. An unsound term translator is not repaired by an exact polynomial checker. A sound checker is not repaired by a replay path that assumes its result through an oracle axiom. An acyclic certificate DAG is not sufficient if a proof parent was imported from the wrong branch.*

8.8 A proposed user interface

Proposed syntax only; no such tactic exists.

```
forge
forge? [selectedLemma, recursiveDefinition]
forge (config := {
  mode           := .compatibility
  induction      := true
  inductionDepth := 2
  synthesizeLemmas := true
  witnesses      := .affine
  witnessSize    := 6
  polynomialDegree := 4
  externalSearch := false
  trust          := .kernelOnly
})
```

A successful explanatory invocation emits a *smaller replay plan*: the chosen induction scheme, the generalised variables, the witness expressions, the library lemmas actually needed, and the concrete certificate data. It must not require the original broad search again, and it must not depend on an external service still being available. A proof that succeeds only while a particular model service is reachable is an unsatisfactory end product when an explicit certificate can be retained instead.

Parameters must describe real implementation limits. A polynomial-degree bound and a certificate monomial bound control different sources of explosion and should not be conflated. A trust option must not change silently because a faster worker happens to be available.

A failure report should identify the first substantive blocked obligation rather than advising a larger timeout, and should distinguish at least:

- no induction motive found;
- candidate lemma refuted by an executable example;

- nonlinear certificate search exhausted;
- unsupported source operation;
- external proof reconstruction incomplete;
- resource limit reached.

A mathematical failure and a resource failure must not produce the same diagnostic. Useful concrete examples of the intended register: “induction step requires a lemma about the changing accumulator; 128 candidate expressions rejected by tests; 3 universal proofs remain open,” and “SOS candidate rejected by exact coefficient check; no theorem added.”

8.9 The deployment pattern

The intended end state for a user’s file is not a permanently expensive tactic call. It is: expensive search once, then a stable, readable proof for ordinary recompilation. Concretely, the accumulator example should end as a separately proved general lemma plus a short specialisation, with the final theorem not invoking the synthesiser at all:

```
theorem revAcc_spec (xs acc : List a) :
  revAcc xs acc = app (rev xs) acc := by
  induction xs generalizing acc with
  | nil => rfl
  | cons x xs ih =>
    simp only [revAcc, rev, ih, app_assoc, app]

theorem revAcc_correct (xs : List a) :
  revAcc xs [] = rev xs := by
  rw [revAcc_spec, app_right_nil]
```

The important algorithmic decision there is the motive — **generalizing acc** — not the final normalisation tactic. A syntactically valid **induction** call with a fixed accumulator gives an induction hypothesis too weak for the recursive call, which is exactly the obstruction of §5.

8.10 Implementation sequence and acceptance gates

The nine contributing efforts proposed between four and six milestones each, with substantial agreement on the ordering. The merged sequence is:

Gate 1 — trustworthy orchestration. Frontend, deterministic compatibility mode, full state rollback, worker contracts, replay records, and the axiom-policy audit. Exit criteria: the restricted mode reproduces a pinned baseline suite without introducing new axioms or losing source assumptions; a deliberately malformed proof or admitted theorem is rejected; a failed branch provably cannot leak a local equality into its sibling. Only existing proof-producing leaf tactics need run at this stage.

Gate 2 — exact algebraic replay. Typed reification and cone replay in Lean; the quadratic and finite-cone oracles called through a bounded data protocol; the affine/Farkas and recurrence certificate reconstructions. Exit criteria: successful Lean checking of every claimed end-to-end example, mutation rejection *at the data boundary*, and recorded axiom dependencies. This stage already adds useful nonlinear facts to a stock **grind** leaf; it need not wait for an induction planner or SDP search.

Gate 3 — structural planning and witnesses. Explicit theorem-application obligations, bounded induction with dependency-directed generalisation, and typed witness grammars. Exit criteria: unseen variations of accumulator correctness are solved *with the correctness theorem absent from the environment*; false-statement and circular-lemma rejection tests pass; an induction success includes the selected principle and reconstructed motive in its replay trace; a witness success includes the witness term and every bound, integrality, and typeclass obligation it needs.

Gate 4 — domain-aware positivity. Bernstein coefficient and coverage rules formalised; compact-box certificates; and, before any univariate certificate is accepted in Lean, a proved or integrated Sturm/root-count result. That verification task is substantial and must not be hidden behind an unchecked foreign call. An explicit multi-leaf replay can validate the search-to-source path first, but production acceptance should prefer a reusable checked certificate theorem to a large amount of duplicated tactic text.

Gate 5 — efficient cooperation. Incremental fact subscriptions, proof-DAG sharing, bound-derived product envelopes, target-driven certificate candidates, and the version-pinned adapter — introduced only where measurement shows that repeated leaf setup dominates. Keep a terminal-tactic fallback so that a Lean upgrade cannot disable the entire system.

Gate 6 — optional broader search. Premise search, SDP proposals with exact reconstruction, the CDCL(T) lane, and optionally a learned ranking policy. Exit criterion for external backends: zero unproved hints in any counted success. Learned components may choose which candidate to try; they never decide which statements are accepted. Native-trust workers remain opt-in, and every optional dependency carries its own toolchain and feature-compatibility check.

What not to implement first. A complete new SMT solver, a rewrite of `grind`, a general SDP backend, a dependent-type-erasing universal intermediate language, or a learned scheduler. The most valuable first release is not “every solver behind one keyword”.

8.11 Where the risk actually is

The highest-risk engineering tasks are not mathematical. They are preserving dependent contexts through generalisation; maintaining valid state across local solver restarts and snapshots; producing compact proof reconstruction; and keeping candidate lemmas out of the proved state. The numerical search is comparatively easy to sandbox, precisely because an exact certificate can reject a bad proposal.

A restricted, correct implementation that exports a small set of ordinary Lean proofs is preferable to a wide solver bridge that silently leaves obligations or enlarges the trusted base.

9 Executed experiments

9.1 Nine runs, reported separately

This section reports nine independent prototype runs, labelled [p1] to [p9] after the source proposals listed in Appendix B. They are reported separately and they are *not* pooled, for a reason stronger than caution.

All nine ran on the same day in the same kind of environment — Python 3.13.5 on Linux x86-64, NumPy 2.3.5, SciPy 1.17.0, SymPy 1.14.0, and where pytest was used, pytest 9.0.2 — and five of them recorded the same seed value, 20260914. That coincidence makes the runs look comparable. They are not. The same seed drives entirely different generators over entirely different case sets: one run generates 30 cone and 60 quadratic cases, another 1,080 witness tables and 300 CDCL(T) instances, another 100 cone and 49 invariant cases, another 1,500 differential SAT cases, another 26 curated certificates. Three runs ([p1], [p8], [p9]) record no seed at all.

Remark (The non-pooling rule). The counts below are not commensurable. “Cases”, “checks”, “assertions”, “tests”, “certificates” and “bundles” are five different units, and several runs use more than one. Summing them would produce a meaningless total, and any such total would describe the prototypes’ own constructed distributions rather than any theorem corpus. Every number in this section carries its run label for that reason.

Two properties do hold uniformly, and they are the ones worth carrying forward. First, no coefficient anywhere in the certificate corpus is stored as a floating-point number: rationals are exact throughout, serialised as strings such as “125/18” or as integer numerator/denominator pairs. A scan of every certificate payload in all nine runs found exactly one float, and it is a timing field. Second, every run’s stored certificates replay under a standard-library-only checker, verified with site packages disabled.

9.2 The runs

[p1] — six certificate families, one uniform protocol

275 pytest instances passed. A deterministic benchmark of 198 constructed instances produced certificates in every case, and a separate standard-library-only decoder accepted 198/198. Each case was searched three times after numerical warm-up; the table reports medians of per-case medians.

Component	Certified	Ablation	Search (ms)	Recheck (ms)
Quadratic SOS	60/60	1	0.864	0.701
Finite cone LP	17/17	0	1.699	0.295
Bernstein box	18/18	1	0.685	0.389
Polynomial recurrence	33/33	—	2.513	0.403
Integral affine witness	40/40	—	0.074	0.027
Univariate with zeros	30/30	0	1.930	0.505

Table 7: [p1]. Ablations are restricted versions of these same prototype mechanisms, **not grind** and not any Lean tactic. A dash means no ablation was run. Search time includes checking performed internally by the search routine, so the recheck column is not the amount of checking hidden inside the search column.

The ablations: the quadratic one recognises only expanded nonnegative combinations of even monomials; the cone one removes richer square candidates or, for the domain example, the hypothesis products; the Bernstein one disallows subdivision; the univariate one runs dyadic Bernstein search to depth 12 without factorisation. The last row is the sharpest result in this run — 30/30 against 0/30 — and it is the experimental face of Theorem 6.5.

Mutation tests alter targets, weights, hypotheses, recurrence base terms, affine offsets, Bernstein split points, child nodes, claimed bounds, and Sturm chains, and require rejection. Independent symbolic tests reconstruct Bernstein expansions, expand quadratic certificates, and compare univariate root counts. Limitations are preserved as regression cases: a degree-limited recurrence search failing on a valid higher-degree answer, integer affine synthesis rejecting a merely rational witness, Bernstein-only search failing at a non-dyadic interior zero.

[p2] — five families, per-family seeds

31 unittest methods passed; several contain multiple seeded trials. 215 certificates were produced and all 215 replayed from storage.

The recurrence family comprises five named power sums and 60 seeded random polynomial increments; it required 312 candidates in total, and the exact step checks generated 247 counterexamples before the 65 certificates were accepted. The 41 Bernstein problems used 82 leaves in total at maximum split depth four; root-only checking certifies 25 of them and subdivision certifies all 41. The six list proof traces contain 6, 3, 8, 10, 9 and 5 rewrites.

Family	Cases	Exact replays	Time (s)
List induction and generalisation	6	6	0.005
Polynomial additive recurrences	65	65	0.241
Affine witnesses with Farkas certificates	63	63	0.352
Finite-dictionary square certificates	40	40	0.648
Bernstein box certificates	41	41	0.067
Total	215	215	

Table 8: [p2]. Seeds are per family: 420 recurrences, 421 witnesses, 422 squares, 423 Bernstein. Times are one local run and are not hardware-normalised.

The run’s own caveat is the important one: *the 40 square-certificate problems are constructed from the same finite dictionary the search uses*, so their 100% success rate tests implementation and rational reconstruction on a designed distribution and must not be read as nonlinear proving coverage. Recorded negative outcomes include the Motzkin polynomial finding no certificate in the plain-square dictionary, the constant witness template failing on the unbounded strip, and $(x - y)^2$ remaining UNKNOWN at depth four.

[p3] — assertion-counted mechanism ablations

1,680 assertions passed; 24 certificate bundles were saved and all 24 replayed under a standard-library-only checker. Seed 20260914; total 2.27 s.

Mechanism	Restricted	Enabled
Cone certificates, curated goals	0/10 diagonal-only	9/10 full dictionary
Cone certificates, constructed	—	30/30
Exact quadratic, constructed	—	60/60
Exact quadratic, curated	—	4/4
Bernstein box, 10 true inequalities	3/10 unsplit	8/10 adaptive
Ground Horn, 12 goals	3/12 unsliced	12/12 demand-sliced
List induction, 8 goals	6/8 fixed accumulator	8/8 generalised

Table 9: [p3]. Every “restricted” column is a restricted version of this run’s own implementation. None is **grind**.

Four negative controls were rejected by the quadratic path, seven corruptions by the Bernstein checker, and eighteen by the induction checker. 250 random Horn graphs and 500 comparisons confirmed that the sliced and unsliced modes agree with a naive least-fixed-point computation. Both Horn modes still pay the full index-construction cost, and the examples deliberately place decoys first.

The 60 generated quadratic examples are constructed from positive square combinations and therefore carry an explicit selection bias; the ten cone goals are deliberately selected mechanism tests, not an unbiased corpus.

[p4] — the largest artefact corpus

64 pytest tests passed. The deterministic experiment produced 1,494 rows and 1,238 saved successful artefacts, all replayed by checking modules that import no numerical or symbolic search library. Seed 20260914. This run is also the only one that re-verified its corpus from an extracted archive copy and confirmed that every family and status count matched the recorded run.

The structural ablation is this run’s sharpest finding, and it is a genuine 2×2 design rather than a single comparison:

Family	Observed result
Residue witness parameters	1,080 checked: 831 proved, 249 impossible by gcd divisibility
Random CNF + integer difference logic	300 oracle agreements: 189 UNSAT, 111 SAT
Sparse cone / equality-ideal certificates	47 of 49; two deliberate UNKNOWN
Exact rational nonnegative quadratics	40/40 manufactured examples
Bernstein covering certificates	12 of 13; the non-dyadic zero case UNKNOWN
Pigeonhole CNF	5 replayed UNSAT proofs
Structural equations	3 synthesised or proved lemmas, 2 rejected false candidates
Induction ablation	4 rows: 1 proved, 3 UNKNOWN

Table 10: [p4]. These families are heterogeneous and must not be combined into a general accuracy score.

Proved append lemmas	Generalise accumulator	Outcome
No	No	UNKNOWN
No	Yes	UNKNOWN
Yes	No	UNKNOWN
Yes	Yes	PROVED

Table 11: [p4]. Only generalisation *and* proved auxiliary lemmas together close the accumulator reversal. The same goal and the same rewriting engine are used in all four configurations, which is stronger evidence of mechanism than a single all-features-enabled demonstration, and it shows why a blanket “try induction” addition would miss part of the design.

Two further details behind the aggregate counts. The 1,080 residue cases sweep all $1 \leq m \leq 20$, $-3 \leq a \leq 5$ and $0 \leq b \leq 5$; the 249 impossible ones fail the exact gcd divisibility condition. And for $p(x) = (x - 1/3)^2 + 10^{-4}$ on $[0, 1]$ — the interior-zero example given a positive margin — the prototype returns a tree with seven leaves, thirteen nodes and maximum depth six, whereas $(x - 1/3)^2$ itself reaches the depth limit, exactly as Theorem 6.5 requires.

The run’s own caveats are worth preserving: 64 is a test-runner count, not a count of mathematical instances, since parametrised tests contain inner-loop cases; forty of the 49 cone cases are manufactured from the search’s own certificate basis; the recorded timing is one in-process pass rather than a repeated performance study; and the field named `certificate_bytes` includes the accompanying statement, not a proof payload alone. The run states the non-pooling rule for itself in the sharpest available form: *do not divide 1,238 by 1,494 and advertise an overall accuracy*, because the denominator mixes proof searches, model searches, a lemma batch, deliberately false formulas, and overlapping ablation runs.

The pigeonhole family is recorded per instance, including the datum that is least flattering to the method:

Pigeons / holes	Decisions	Conflicts	Resolution nodes	Nonchronological jumps
2 / 1	0	1	2	0
3 / 2	1	2	10	0
4 / 3	8	9	45	0
5 / 4	30	31	160	0
6 / 5	89	89	513	1

Table 12: [p4]. The small number of non-chronological jumps is reported rather than hidden. It is not evidence that this branch heuristic is competitive on industrial SAT instances. Note also that this encoding — at least one hole per pigeon and at most one pigeon per hole — differs from [p6]’s, so Tables 15 and this one are not comparable.

[p5] — expected-outcome benchmarking with negative controls

73 pytest tests passed in 1.11 s. Seed 20260914.

Experiment	Outcome	Interpretation
Named nonlinear cases	9 accepted / 12	Two true cases unresolved; one deliberately false case unresolved
Generated product-cone cases	100 / 100	Constructed inside the covered cone; not representative sampling
False-polynomial controls	0 accepted / 24	All UNKNOWN; exact negative witnesses checked separately
Bernstein cases	43 / 45	One true non-dyadic-zero case and one false case unresolved
Additive recurrence invariants	49 / 49	Base and universal step identities checked
Horn growth family	5 depths	Both strategies prove all goals; search work differs sharply
Actual Lean comparison	Not run	No measured superiority over grind

Table 13: [p5].

The run states its own most important qualification: the **109** accepted nonlinear certificates are the nine named successes plus the 100 generated cases, and they are *not* 109 previously unsolved Lean theorems. The 24 negative controls are of the form $(x - a)^2 - \varepsilon$ with rational $\varepsilon > 0$; the benchmark separately checks the exact negative value at a , while the cone routine itself returns UNKNOWN rather than a certified refutation. Median recorded cone discovery time was about 21.5 ms with a maximum of about 103.6 ms; median invariant synthesis about 4.1 ms, including interpolation and validation but excluding any Lean work. 58 exported certificates — nine named cone plus all 49 invariants — were reloaded and re-accepted by a separate validator.

[p6] — proof-logging CDCL(T)

This run’s core is a differential test against an exhaustive reference.

Check	Result
Random pure CNF cases	1,000
Random CNF + integer-difference cases	500
Satisfiable results	1,278
Unsatisfiable results	222
Differential mismatches	0
UNSAT certificates independently replayed	222
Truncated certificates rejected	222
Additional structural edge cases	5
Deliberately mutated theory certificates rejected	2

Table 14: [p6]. Formulas have three to eight Boolean variables, clauses of three distinct variables with independent signs, and one to six times the variable count in clauses; difference constraints use four integer variables and constants in $[-3, 3]$. The reference enumerates all Boolean assignments and uses an all-pairs shortest-path consistency check; the solver under test uses CDCL with negative-cycle theory conflicts. These are small exhaustive tests, not an industrial benchmark.

The five edge cases are an empty conjunction, an empty clause, a tautology, contradictory units, and repeated literals; the integer-complement boundary has its own regression assertion.

The run attaches the caveats itself, and they should be kept. The padded-core family is an artificial diagnostic: it places unused Boolean variables before a two-variable inconsistent core, which a production preprocessor would delete. Its decision-count gap illustrates non-chronological backjumping; it is not credible evidence of practical speedup. More pointedly, *runtime gives a soberer picture than decision counts*: on the largest pigeonhole case both solvers took about 26 ms despite their different decision counts, with replay adding about 21 ms. Operation counts are treated as diagnostic and no general speed claim is made.

Instance	Variables	Clauses	DPLL decisions	CDCL decisions
Padded core, 0 unused	2	4	1	1
Padded core, 4 unused	6	4	31	5
Padded core, 8 unused	10	4	511	9
Padded core, 12 unused	14	4	8,191	13
Pigeonhole $4 \rightarrow 3$	12	22	8	8
Pigeonhole $5 \rightarrow 4$	20	45	51	39
Pigeonhole $6 \rightarrow 5$	30	81	374	148

Table 15: [p6]. The baseline is *this run’s own* chronological unit-propagating DPLL with the same static variable order and phase. It is not *grind* and not a production SAT solver.

Ten polynomial search cases yielded seven certificates and three deliberate UNKNOWNs — a false target, Motzkin at degree cap 6, and $(2x + y)^2$ at degree cap 2. A further 100 randomly planted rational-square certificates exercised the checker and were cross-checked symbolically; those are *checker tests, not 100 additional search successes*, and the manually supplied discriminant identity (9) is likewise separate from the seven discoveries. Six invariant powers $p = 0, \dots, 5$ yielded degrees 1 to 6 with final sample counts 3, 4, 5, 6, 7, 8, and each rejected the alterations $Q + 1$ and $Q + n$, giving 12 mutation rejections on the synthesis side and 12 more under independent replay.

The saved corpus is 8 SAT/theory proof logs, 8 polynomial certificates (seven discovered, one manually supplied), and 6 invariant certificates, all replayed with site packages disabled. Three polynomial search files carry no certificate and are explicitly skipped as UNKNOWN. The replay program fails on a missing corpus rather than reporting vacuous success.

[p7] — component cases with corruption controls

833 principal component test cases produced 891 accepted certificate checks; 330 deliberately corrupted certificates were rejected; 150 exact polynomial cross-checks were run against a symbolic oracle; and 897 synthesised accumulator formulas were compared against recursive execution. Seed 20260914.

The run is explicit that *cases and checks are different units*: some Horn cases compare two successful derivations while others have no derivation, and timing repetitions and archived demonstrations are excluded from the aggregate check count. Five cone-search misses and two invariant-template misses are expected outcomes, not wrong proofs.

26 representative certificates — 9 cone identities, 5 lattice traces, 11 accumulator invariants, 1 Horn derivation — all passed separate standard-library-only replay. The lattice worker was additionally validated by an exhaustive 539-case single-row grid checked against integer gcd, and by 120 random feasible systems whose free-basis samples were re-checked against every row.

[p8] — the cleanest dictionary ablation

260 pytest tests passed, the highest count of the nine. No seed recorded.

Experiment	Restricted baseline	Extended mechanism
Polynomial cones, 60 constructed instances	17 certificates	60 certificates
Positive rational-box instances, 21	0 unsplit	21 subdivided
Affine accumulator templates, 121	not measured	121 synthesised and replayed
Out-of-template recurrences, 5	not applicable	5 UNKNOWN outcomes

Table 16: [p8]. The cone row varies only the dictionary and holds the LP backend fixed, which makes it the most cleanly isolated ablation in the collection. It is still an ablation of this run’s own implementation.

The Horn workload is the largest: a useful chain of length 12 buried in 1,000 unrelated chains of length 24, each with an initial fact. Full closure fires all 24,012 rules; an indexed baseline that stops as soon as the goal is known still fires 11,012 under the specified interleaved queue order; backward slicing selects just the 12 useful rules.

The accompanying observation matters more than the ratio: *proof minimisation reduces even the full-closure derivation to the same 12 proof steps*, so proof size alone would conceal the entire search-work difference. Conversely the cold reverse index still scans all input rules and the demand implementation still loads the initial fact set, so no sublinear end-to-end cold performance is claimed. The run explicitly declines to infer a speedup factor, noting a *slower* early-goal run despite fewer firings.

Supporting evidence: 121 invariant cases each cross-checked against runtime execution with four deterministic random-list checks per pair (484 concrete recurrence checks); 30 seeded sparse-polynomial tests against a symbolic oracle; 40 randomised Horn programs comparing full and sliced success and replaying their derivations; and monomial-enumeration counts checked against $\binom{n+d}{d}$. Only two certificates were exported to files, the fewest of the nine.

[p9] — small, heavily mutation-tested

69 pytest tests passed. 13 persisted bundles replayed with site initialisation disabled: one ordered bank of five inductively proved lemmas with a specialisation proof, four Horn proofs, six cone certificates, one box certificate, and one witness with its two side-condition proofs. These are bundles, not 13 independent Lean theorems. No seed recorded.

Depth	Forward facts	Backward proof nodes	Forward cap	Forward status
4	16	5	5,000	Certified
8	256	9	5,000	Certified
12	4,096	13	5,000	Certified
16	5,000	17	5,000	Cap reached

Table 17: [p9]. Neither column is an implementation of `grind`. The forward baseline stops as soon as the target is produced, which is why its counts are 2^d rather than $2^{d+1} - 1$.

Mechanism outcomes: accumulator-lemma discovery enumerates 41 candidates, filters against 49 sample environments, and the 41st enumerated term $\text{app}(\text{rev}(xs), a)$ is the one that receives an induction certificate; the quartic nonnegativity case builds 280 candidate columns and 35 coefficient equations and returns a three-term certificate; the witness search reaches candidate 23, $w(x) = x^2 + 1$, with three candidates reaching universal certificate search; the Bernstein case $x^2 - x + 3/10$ on $[0, 1]$ needs one split and yields two leaves each with exact lower bound $1/20$.

The deliberate control is $(x + y + 1)^2$: the restricted dictionary finds no certificate, while a manually supplied one-square certificate is accepted. This run therefore has a *checker more general than its searcher*, which it uses explicitly as a guard against an inflated “SOS-complete” claim.

This run has the most systematic mutation testing of the nine, with parametrised bad-input matrices for cone, box, Horn and induction certificates, including rejection of a base-case use of the induction hypothesis, invalid generalisation metadata, rigid-variable intrusion, and reference to an unavailable future lemma.

9.3 Four Horn experiments that must not be merged

Four runs measured demand-directed versus forward Horn search. They used four incompatible setups, and the temptation to combine their tables should be resisted.

That last point is the useful methodological lesson: a factor-of-two difference in a headline growth curve came entirely from a baseline policy decision, not from the mechanism under test.

Run	Setup and headline observation
[p5]	Depth sweep 4/8/10/12/14 on $P(a)$, $P(x) \rightarrow P(g(x))$, $P(x) \rightarrow P(f(x))$; no distractors; no forward cap. Forward generates $2^{d+1} - 1$ facts (31, 511, 2,047, 8,191, 32,767) against $d + 1$ demand atoms; both proofs have $d + 1$ nodes. Timings reported (846.283ms against 0.478ms at depth 14).
[p9]	Same rule family, depths 4/8/12/16, but the forward baseline <i>stops when the target appears</i> , giving 2^d facts; a 5,000-fact cap is reached at depth 16. No timings reported.
[p3]	12 curated goals with decoys placed first; 3/12 unsliced against 12/12 sliced; 250 random graphs and 500 agreement comparisons.
[p8]	Fixed useful chain of length 12 with a distractor sweep to 1,000 chains; 24,012 / 11,012 / 12 firings for full closure, early-goal stop, and demand slicing. Timings explicitly disowned.

Table 18: The same mechanism, four incomparable measurements. [p5] and [p9] differ only in the forward baseline’s stopping rule, and that alone changes the reported growth from $2^{d+1} - 1$ to 2^d .

9.4 Lean status

What the nine runs recorded

None of the nine compiled any Lean source. Each recorded the same reason — no `lean` or `lake` executable in the authoring environment — under five different filenames and in two different formats. Consequently none of them performed a kernel check, an axiom audit, or any comparison against `grind`, `Aesop`, `nlinarith`, `LeanHammer`, or any other tactic. Twenty-one `.lean` files across the nine, totalling about 3,000 lines, were *candidate source*.

What this merge was able to add

The merge environment has `elan`, and installing the pinned `leanprover/lean4:v4.34.0` made a first, narrow compilation check possible. Three of the sixteen files in the merged tree import nothing beyond Lean core and can therefore be elaborated without a built Mathlib:

File	Result	Elapsed (s)
[p3] <code>lean/DesignAPI.lean</code>	elaborated, no errors	211.4
[p6] <code>lean/Contracts.lean</code>	elaborated, no errors	5.3
[p9] <code>lean/Structural.lean</code>	elaborated, no errors	5.1

Table 19: Lean v4.34.0 (commit 293d5d0c), recorded in `results/lean-core-elaboration.json`.

This confirms that the proposed runtime and scheduler contract, the `CertificateSpec` soundness contract, and the Mathlib-free structural list proofs are well-typed Lean 4.34.0 rather than merely plausible-looking source.

It confirms nothing else. It is not a transitive axiom audit. It says nothing about the other thirteen files, which import Mathlib and need a built project. It does not establish that any certificate checker is correct. And it is not a comparison against any Lean tactic — successful elaboration of a hand-written example proves that the example type-checks, not that automation found it.

9.5 What the evidence establishes

It establishes that the implementations actually synthesise certificates in the stated fragments; that certificate replay is separable from the numerical and symbolic search libraries; that the specified mutation and corruption tests are rejected; that several mechanisms solve problems their own restricted precursors cannot; and that three Lean design artefacts type-check under the pinned toolchain.

It does not establish that `Forge` is implemented as a Lean tactic, that the Mathlib-dependent candidate scripts compile, that any certificate is checked by Lean’s kernel, that the methods

outperform existing Lean tactics, or that any success rate transfers to user developments. Those claims require different experiments, described next.

Two structural limitations of the evidence deserve to be named rather than buried. Manufactured examples favour the grammar that generated them — most visibly for [p2]’s 40 square certificates and [p5]’s 100 generated cone cases, both of which are drawn from the very dictionary their search covers. And search and replay share substrate: the same sparse polynomial and term representations appear on both sides in every prototype. Search-free replay is a meaningful independence property, but it is not an independently implemented formal verification of that representation. The strongest partial answers to this are the checkers that run algorithmically *opposite* to their producers — expanding a claimed Bernstein basis rather than recomputing coefficients, re-deriving a clause by independent unit propagation rather than replaying a resolution chain, recomputing a Sturm chain rather than trusting a stored root count, verifying $\det U = \pm 1$ rather than calling the elimination that produced U .

10 The evaluation that would actually decide the question

Nothing in Section 9 measures whether **Forge** would be a better Lean tactic. This section specifies the experiment that would, in enough detail that it could be pre-registered.

10.1 Freeze the comparison, including its theorem knowledge

Use a pinned Lean release, a compatible pinned Mathlib revision, fixed imported modules, and recorded backend versions. Preserve each source goal with its original hypotheses. Run methods in isolated processes so that one theorem’s auxiliary declarations and caches cannot leak into another method’s state, and reset proof state between runs so that a successful trial cannot add a lemma that helps a later baseline run.

A fair test must specify both the theorem statement and the environment in which it is presented. This is the single most common way an automation benchmark misleads, and it has a specific form here: a benchmark extracted from an already compiled library can make the tested theorem itself, a renamed duplicate, or a later theorem depending on it available through retrieval. Renaming a theorem is not enough if its exact type remains indexed as a premise, and attributes that indirectly reintroduce a held-out theorem are also leakage. Build the retrieval index only from the *pre-theorem* environment, or generate each benchmark at the correct declaration point.

Never inspect the target theorem’s existing proof to choose premises unless the experiment is explicitly labelled an *oracle-premise upper bound*.

10.2 Baselines

At minimum: stock **grind**; tuned **grind** with explicit budgets and relevant suggestion settings; a simple public-tactic portfolio; and the full **Forge** system. On applicable subsets add **Aesop**, **nlinarith**, **omega**, **Duper**, the existing **SOS** and **LP** backends, **lean-smt**, and the shared-state bit-vector route.

Two baselines matter more than the rest.

The first is the *equal-knowledge* baseline. Use the same imports, local context, theorem statement, typeclass environment, and premise universe. If **Forge** performs premise retrieval, give the baseline a matched retrieval condition or report the retrieval effect separately. Giving **Forge** an associativity lemma while withholding the same available theorem from a premise-aware baseline measures a data-access difference, not a search improvement. A system that wins only against an artificially weak baseline has demonstrated nothing.

The second is the *non-communicating portfolio*: the same workers, the same total budget, no fact sharing. This is the experiment that distinguishes “integration earns its complexity” from “we added more solvers”, and it is the one most easily omitted.

10.3 Two tracks, three input conditions

Run two tracks. The *cold-knowledge* track allows only the actual pre-theorem environment and tests missing-lemma synthesis. The *equal-premise* track gives all methods the same supplied premises and tests proof search and cooperation.

For structural problems, distinguish three input conditions: definitions only; a common relevant helper-lemma set; and an oracle-assisted set containing the handwritten proof’s helper lemmas. The gap between the first two measures lemma acquisition; the third separates discovery failures from local proof failures. A fourth ablation — supply a hand-written correct induction motive to every eligible method — isolates motive selection from leaf solving.

10.4 Corpus design

Mix existing development goals with newly written examples, and stratify:

- recursive data-structure theorems needing induction or changing accumulators — and not only append and reverse: include trees, mutually recursive syntax, nested recursion, and dependent vectors;
- polynomial cost and invariant proofs;
- semialgebraic inequalities with side conditions — sparse and dense, near-zero and exactly-zero boundary behaviour, rationals with denominator side conditions, integer versus real distinctions;
- quantified statements needing new arithmetic terms;
- constructive existential specifications with scope-sensitive witnesses;
- mixed guarded arithmetic and rewriting;
- Boolean-heavy verification conditions — theory-heavy and theory-light, not only pigeonholes;
- higher-order and indexed-datatype obligations;
- a large control set of goals `grind` already handles well, including its arithmetic and bit-vector strengths.

Include valid but unsupported problems, false conjectures, misleading induction variables, irrelevant assumptions, and goals requiring lemmas outside the first retrieval batch. The control set is not optional: it detects whether the new controller wastes resources on familiar cases.

Split by theorem dependency family or development module, not by random near-duplicate statements. Hold out whole families when tuning heuristic weights — renaming variables or changing constants in a training problem does not create an independent test set for a lemma-discovery heuristic. And do not choose only problems generated from a favoured certificate language; that is precisely the bias Section 9 had to flag repeatedly in the prototype evidence.

10.5 Metrics

Report solved goals at several fixed wall-clock and heartbeat budgets, together with median and tail latency, peak memory, external-process time, proof and certificate size, reconstruction time, and kernel-checking time. Distinguish cold import costs from warm elaboration costs, and count any proof cache or premise index that amortises work explicitly rather than charging it only to the competitor.

Record separately: failed translations, unsupported fragments, timeouts, rejected certificates, and residual reconstruction goals. *Count a proof with one residual goal as unsolved.* A solver verdict with an unclosed proof hole is not a solved benchmark, and a worker’s Boolean result is never the benchmark’s success oracle. Timeouts and reconstruction failures belong in the denominator.

Keep deterministic replay success as its own metric. Report paired wins and losses by problem family, not just total solved counts, using paired confidence intervals or a paired test over an appropriate sampling unit — projects or theorem clusters, since examples within a module are correlated. For times, use budget curves or censored-runtime summaries rather than averaging only the successful cases and discarding timeouts.

Compatibility mode asks a different question and needs a different plot: how much *additional* resource rescues failures without replacing the successful baseline run? Plot rescue coverage against that additional budget, and never present the preservation property of Proposition 1.2 as an equal-cost performance theorem.

One class of outcome is not a metric at all. *Any accepted false theorem or unapproved axiom is a release-blocking failure, not an accuracy regression.*

10.6 Required ablations

Remove one capability at a time: backward obligation production; induction; induction without generalisation; helper-lemma discovery; constrained instantiation; constant-only witnesses; hidden-quadratic certificates; hypothesis-product cones; Bernstein subdivision; zero-aware univariate search; lattice witnesses; the Boolean lane; cross-worker fact exchange; and relevance-based scheduling. Evaluate generalisation separately from helper-lemma discovery so that the contribution of each is visible — [p4]’s 2×2 result (Table 11) is the reason this matters.

And run the stateless-restart versus incremental-adaptor comparison explicitly, since it is the only thing that can justify the invasive integration work of Gate 5.

10.7 Acceptance gates

A defensible acceptance criterion has three parts, and the threshold must be chosen *before* the run:

1. Retain a high fraction of the goals the pinned baseline solves.
2. Solve a substantial share of held-out families requiring combinations of induction, construction, nonlinear inequalities, and Boolean search.
3. Keep every result independently checkable, with proof reconstruction and kernel checking included in the cost.

One contributing proposal offered, purely as an illustration of the *form* such a target takes, “20% more fully checked solves on the selected difficult families and at most 5% regression on the baseline suite”. That is a shape, not a prediction: no such evaluation has been run, no percentage is forecast here, and a 10-point gain in one category would not excuse an undisclosed regression in the control category.

10.8 A soundness regression suite, run continuously

Separately from the performance benchmark, maintain a suite that tries to break the system rather than to score it: adversarial certificates (oversized exponents, duplicate keys, forward proof references, transplanted certificates, omitted subdivision children, negative or float weights, truncated proof logs); branch-leakage tests; circular-lemma tests; wrong-context replay; axiom-policy violations; and compatibility checks against each supported Lean revision.

Mutate a denominator condition; swap two polynomial variables; replace a strict inequality with a weak one; change an exponent; reverse an equality transport; remove a branch from a box cover; alter a typeclass instance; point a proof node to a future node; and replay under a different local context. Each of these has an executable analogue in at least one prototype today, and each should have one in the Lean implementation.

Resource exhaustion is a first-class outcome in this suite. A timeout or out-of-memory event must return a recoverable status — never trigger a fallback that trusts the search output.

11 Assessment and recommended starting point

The most promising route beyond `grind` is not to replace its equality and arithmetic infrastructure. It is to surround that infrastructure with proof-structure search and to feed it new, independently validated mathematical consequences.

Three complementary kinds of progress are available: discovering a certificate, discovering an invariant, and constructing a witness. Each of them changes what is being proved rather than deriving more consequences of a fixed local state, and that is the distinction the whole architecture rests on. A constructor case split and an induction principle are not interchangeable — the former says a list is empty or a cons, the latter licenses a recursive hypothesis about its tail. Recognising an existential proposition is not the same as finding a useful witness for it. Increasing a rewriting budget cannot turn a theorem about one fixed accumulator into a theorem about all accumulators.

The arithmetic design is deliberately broader than a single SOS tactic, and the breadth is justified by an actual obstruction rather than by taste. Exact quadratic decomposition handles hidden global quadratic positivity and is complete on its fragment. Finite cones exploit useful factors and hypotheses, which global SOS search structurally cannot do for a domain-dependent statement. Bernstein trees exploit compact domains. Square-factor and Sturm certificates handle the univariate zeros that Theorem 6.5 proves subdivision can never reach. Each of these fails on problems another one decides immediately — that is why they belong in one system with a router, and why a failure of one search is a reason to route rather than a reason to stop.

11.1 What to build first

Not a new SMT solver, and not a rewrite of `grind`. A narrow Lean controller that combines automatic generalisation and induction with an exact witness and certificate pipeline already changes the *shapes* of proofs that can be found automatically, and it provides clean interfaces for adding stronger nonlinear and quantified reasoning afterwards. The first useful release adds induction with generalisation, typed witness construction, and intermediate certified arithmetic facts. Tightly integrated Boolean search comes after the scope and explanation contracts are reliable — which is worth stating plainly, because the Boolean lane has the most complete prototype here and is still the right thing to do last.

Two findings should narrow the claims a first release makes. A Lean `sos` project already implements an end-to-end nonlinear-real-arithmetic tactic with reification, search, rational reconstruction and certificate-based proof construction; a Lean `lp` project already supplies verified linear programming and a quantified rational-affine fragment. Building either again would be the second unaudited wrapper, not a contribution (Remark 6.6). The contribution is *when* to request a certificate, *how* to bind it to the source problem, and *what* to do with the proved consequence afterwards.

11.2 Where the risk is

The largest soundness risk is mistranslation at the Lean/certificate boundary — local assumptions, casts, dependent binders, and strict inequalities. The largest performance risk is generating candidates whose proofs are expensive to reconstruct. The largest evaluation risk is comparing against an artificially weak premise set, or leaking target theorems into retrieval.

None of these is mathematical, and that is the useful observation. The hard parts of this project are not the algorithms; §8 lists them and they are all engineering. It is worth noticing which way that cuts: a project whose remaining risk is concentrated in state management and proof transport is a project whose mathematics is done, and one that should be judged on whether it ships a narrow correct thing rather than a wide leaky one.

11.3 What this document has established, and what it has not

Established: nine prototype runs synthesise certificates in fragments spanning exact polynomial cones, quadratic decomposition, Bernstein subdivision, univariate Sturm certificates, polynomial and accumulator invariants, affine, lattice, residue and polynomial witnesses, ground Horn slicing,

structural induction with generalisation, and proof-logging CDCL(T); every stored certificate replays without the search libraries; the specified mutation and corruption tests are rejected; several mechanisms solve problems their own restricted precursors cannot; and three Lean design artefacts elaborate under the pinned v4.34.0 toolchain.

Not established: that `Forge` is implemented as a Lean tactic; that the thirteen Mathlib-dependent candidate scripts compile; that any certificate is checked by Lean’s kernel; that any checker is correct rather than merely tested; that the methods outperform any existing Lean tactic; or that any success rate transfers to user developments. No head-to-head comparison against `grind`, `Aesop`, `nlinarith`, `LeanHammer`, or any other tactic exists anywhere in this work.

The gap between those two lists is the project. It is not a gap that more prototype experiments can close: it closes only by writing the Lean implementation described in Section 8 and running the evaluation described in Section 10.

11.4 The intended end state

Worth restating, because it constrains the design: the goal is not a permanently expensive tactic invocation. It is expensive search *once*, then a stable, readable proof for ordinary recompilation — a separately proved general lemma and a short specialisation, with the final theorem not invoking the synthesiser at all. A proof that succeeds only while a particular search process, solver build, or model service remains available is an unsatisfactory end product when an explicit certificate could have been retained instead.

The result is a concrete, falsifiable design for substantially stronger automation, with working algorithmic building blocks and a precise account of what still has to be proved and measured. It is more than a list of tactics to try, and considerably less than a finished Lean prover. Saying which is which, at every point, is the part of this document that matters most.

A Certificate schemas

The nine prototypes serialised the same mathematics in nine mutually incompatible ways. This appendix fixes one canonical shape and records the invariants a checker must enforce. The differences between the original formats were confined to key names and one numeric encoding: no information is lost in either direction, and no float rounding is involved anywhere (§A.10).

A.1 The universal rule

Invariant 15 (Exact rationals only). Every mathematical coefficient is an exact rational, serialised either as a string such as `"125/18"` or as an integer numerator/denominator pair. No mathematical value is ever a JSON floating-point number. A decimal is not parsed into a claim of approximate equality, and there is no epsilon, numerical norm, or solver-status field anywhere in an acceptance condition.

The only floats in the entire corpus are timing diagnostics.

A.2 Sparse polynomials

A polynomial carries a variable count and a list of `[exponentVector, rationalString]` pairs:

```
{
  "nvars": 2,
  "terms": [
    [[0, 2], "1"],
    [[1, 1], "-2"],
    [[2, 0], "1"]
  ]
}
```

Canonical construction rejects malformed exponent vectors, negative exponents, zero coefficients, arity mismatches, and *duplicate or unsorted monomials*. Decoding must also reject duplicate

JSON object keys: two parsers disagreeing about which value wins is a genuine differential hazard, and at least one prototype installs an explicit duplicate-key hook for this reason.

The variable table binding these indices to Lean expressions is a separate production concern and is deliberately absent from the Python formats, which use fixed mathematical variables.

A.3 Cone and ideal certificates

```
{
  "kind": "nonnegative_cone",
  "terms": [
    { "weight": "1/2",           -- exact, nonnegative
      "square": {...polynomial}, -- this stores q, NOT q^2
      "factors": [0, 2] }       -- indices into the supplied hypotheses
  ],
  "equalities": [
    { "index": 1,               -- index into the supplied equations
      "multiplier": {...} }     -- UNRESTRICTED sign
  ]
}
```

Checking, expressed as the loop every implementation runs:

```
acc := zero_polynomial
for term in certificate.terms:
  reject unless term.weight is an exact rational and >= 0
  reject unless term.square has the target's arity
  p := term.weight * term.square^2
  for i in term.factors:
    reject unless i is a valid hypothesis index
    p := p * nonnegative_hypotheses[i]
  acc := acc + p
for (index, multiplier) in certificate.equalities:
  reject unless index is a valid equality index
  acc := acc + multiplier * equalities[index]
accept exactly when acc == target
```

Four points that the prototypes learned the hard way:

- The field named `square` stores q , not q^2 . Mixing the two conventions silently changes the degree of everything.
- The weight must be confirmed to be an exact rational *of the right type* before it is compared to zero. A weight of $(10^{20} + 1)/10^{20}$ offered as 1 is the tested attack.
- Equality multipliers are unrestricted in sign, so an LP over nonnegative variables needs $\pm$ columns for each ideal generator.
- The target, hypotheses and equations are supplied *from outside* the certificate (Invariant 2). The certificate is valid *under* those hypotheses; replay does not establish that they hold at any particular point, and the Lean adapter must map each hypothesis index to an actual proof of the corresponding reified inequality rather than copying a polynomial list into the checker.

A.4 Bernstein trees

```
Tree := { "leaf": { "degrees": [...], "coefficients": [...] } }
        | { "axis": i, "cut": "1/2", "left": Tree, "right": Tree }
```

The root box and the polynomial are separate problem inputs. *Child domains are computed by the checker during replay*, never read from the certificate, so a producer cannot omit a strip of the domain by supplying inconsistent endpoints. Every basis index must appear exactly once in canonical order. A cut at an interval endpoint, a missing child, or an invalid axis is rejected. Tensor size is capped *before* expansion, since an adversarial certificate could otherwise request an enormous one.

A negative Bernstein coefficient is not a negative value of the polynomial; it is failure of a sufficient condition. Conversely a positive sampled value is not enough to accept a leaf. Node and depth limits are operational rejection limits, not mathematical assumptions.

A.5 Induction traces

An induction certificate contains the equation, the induction variable, the *generalised* variables, and four rewrite traces: left and right for the base case and for the step case. A rewrite step is a `(rule_identifier, subterm_path)` pair, optionally with the resulting term.

The checker derives the base and step contexts and the single permitted induction hypothesis itself; these are never claims supplied by the certificate. It reconstructs each substitution and replacement rather than trusting a recorded output term, and the two replayed sides must end syntactically identical. The recursion index is a rigid eigenvariable; only the named generalised parameters may be instantiated. In the base case the induction hypothesis is not available at all.

The strongest variant attaches to each rule an explicit *flexible* variable set and requires the trace to supply exactly that substitution domain with correct sorts.

Certificates are stored and replayed in dependency order, and a lemma is installed only after its own certificate has been checked. A bundle checker must additionally reject forward dependencies, shadowing of a defining equation or of the induction-hypothesis name, and a tampered status field — which is why an accepting run takes an explicit list of the entries it is *required* to have verified, so that an empty or mistyped bundle cannot pass vacuously.

A.6 Horn derivations

A node records its ground conclusion, the rule index or an input-fact marker, a complete ground substitution, and the indices of its premise nodes. Replay checks groundness; that every referenced rule exists; that the substitution domain is exactly the head’s variables; that the recorded conclusion is that rule’s actual instantiated conclusion; that every premise is already derived; that every premise index is strictly smaller than the node’s own; and that the last node is the goal.

Trimming retains only ancestors of the root, preserving topological order. A dependency cycle in the *search* graph may be legitimate; the final certificate may never justify itself through one.

A.7 Lattice traces

Per row: the row number, the unimodular matrix U , the gcd g , the residual r , and either the quotient r/g or an obstruction marker. Replay recomputes the current row image and residual, checks that all entries are integers, verifies $\det U = \pm 1$ by independent rational elimination, and performs the exact state update — *without* calling the extended-gcd construction or the solver that produced the trace.

A.8 SAT and theory proof logs

Two supported formats.

RUP form: a list of added clauses, each re-derived by independent unit propagation, plus signed negative-cycle theory lemmas validated by exact coefficient summation with a strictly negative resulting bound. The checker duplicates the atom-negation rule rather than calling the solver’s shared edge constructor.

Resolution-DAG form: nodes tagged `input`, `theory` or `resolution`, each resolution node carrying parent indices and a pivot. Cheaper to check, but it replays the solver’s own derivation rather than re-deriving it.

A satisfying assignment is a separate record: it must satisfy every original clause *and* each atom's truth value must match the integer model.

A.9 Invariant records

The record carries the invariant's exact coefficients, the initial state, the transition, and — in some prototypes — the search history and diagnostic residual strings.

Remark. Replay must *ignore* the diagnostic residual claims. It independently reconstructs the invariant, substitutes into the base and step identities, and compares coefficient maps against zero. This is what prevents a fabricated "**residual**": 0 field from serving as evidence.

A.10 What the recorded formats actually diverged on

For the record, and as the specification for the translation layer:

Field	Spellings observed across the nine runs
Polynomial arity	n ([p1], [p3], [p4], [p6], [p8]); variables ([p2], [p7]); nvars ([p5]); a term-tree AST with no arity field ([p9]).
Coefficient encoding	"n/d" strings in eight runs; { numerator , denominator } integer objects in [p7]. This is the only numeric conversion any adapter needs.
Cone container	terms ([p1], [p3], [p5]); squares ([p2]); nonnegative ([p4]); cone ([p6]); positive ([p7], [p8]).
Weight	weight (six runs); coefficient ([p4], [p8]).
Hypothesis reference	powers ([p1]); constraint , singular and nullable ([p2]); hypotheses ([p3]); factors ([p4], [p5], [p6], p8); assumptions ([p7]).
Ideal multipliers	equality_multipliers ([p1]); ideal ([p4]–[p7]); equalities as { index , multiplier } ([p8]); absent in [p2], [p3], [p9].
Bernstein node	{ kind , axis , point , left , right } ([p2]); { leaf : { coefficients , degrees }} ([p3]); { axis , cut , left , right , leaf } ([p8]).

The formats are *semantically compatible and syntactically incompatible*. Every one encodes weighted squares times nonnegative-generator products plus ideal multipliers, over exact rationals. A single canonical schema is reachable by key renaming plus the one [p7] numeric conversion. Three families have no common schema and stay separate: the CNF/RUP proof logs, the lattice elimination traces, and the symbolic term-tree theorem bank with its rewrite paths.

A.11 What a production format must add

A schema version; a source-goal identifier; an environment and encoding revision; a variable-map identifier resolved against the Lean side's own registry; the required assumptions; and the claimed result kind. Resource limits must bound integer bit sizes, monomial counts, degrees, tree nodes, and proof depth *before* expensive checking as well as after it.

The Lean process resolves every identifier against its own state. It never accepts an oracle-provided expression as authoritative, and the decoder itself must be checked — a Lean-side decoder with a proved correspondence to the reified problem is part of the trusted path, not a convenience.

B Provenance: what came from where

This document is a merge of nine independently prepared proposals for the same system. All nine are retained verbatim under **proposals/** so that every claim here can be audited against its source. Nothing in this article is new mathematics: each statement is carried over from at least one of the nine, and this appendix records which.

B.1 The nine sources

Label	Directory, and the archive name it arrived under	Distinguishing emphasis
[p1]	p1-structural-search <i>from</i> FORGE_Beyond_Grind(1) (1)/	Six certificate families under one measurement protocol; the only zero-preserving univariate worker; the hardened certificate decoder.
[p2]	p2-obligation-controller <i>from</i> Forge-design-and-prototypes/	The three-store architecture and the demand protocol; Farkas affine witnesses as a single feasibility problem; theory-constrained instantiation.
[p3]	p3-planner-certificate-layer <i>from</i> Forge_Beyond_Grind/	Two graphs rather than one fact bag; compositional acceptance; the quantitative Bernstein obstruction; the Lean runtime design contract.
[p4]	p4-theory-cooperation <i>from</i> Forge_Beyond_Grind(1)/	The largest artefact corpus; residue witnesses; the named grind solver-extension API; the 2×2 structural ablation.
[p5]	p5-certificate-first <i>from</i> Forge_Tactic_Design/	Target-driven square proposal; tabling against cyclic pseudo-proofs; solver islands and translation ledgers; expected-outcome benchmarking.
[p6]	p6-proof-logging-cdcl <i>from</i> forge-beyond-grind/	Proof-logging CDCL(T) with independent RUP replay; the integer-negation trap; the nativeEqTrue trust finding.
[p7]	p7-successor-architecture <i>from</i> forge-grind/	Integer-lattice witnesses and certificates of impossibility; the evidence broker; conclusion indexing; metavariable ownership.
[p8]	p8-obligation-broker <i>from</i> forge-tactic-design-and-prototypes/	The cleanest dictionary ablation; certificate cost as a scheduling term; the largest Horn workload; checkers that take the target externally.
[p9]	p9-proof-planner <i>from</i> forge_beyond_grind (1)/	The executive assessment; variables absent from the conclusion; witness as a program with an obligation; the rank-one dictionary-gap argument.

B.2 Where each section’s material came from

Section	Principal sources
§1 Objective	The two modes and the preservation proposition appear in all nine; the four evidence levels are [p1]’s framing with [p4]’s vocabulary and [p8]’s “implemented / mathematically justified / proposed / kernel checked” distinction.
§2 Baseline	Five proposals built partial capability tables; the bit-vector and conditional-rewriting rows are [p2]’s, the configuration defaults [p9]’s, the dependency map [p2]’s extended with [p6]’s and [p9]’s library findings.

Section	Principal sources
§3 Architecture	Three stores and the demand vocabulary from [p2]; two graphs, hyperedges and scope-as-identity from [p3]; the evidence broker, event subscriptions and metavariable ownership from [p7]; the ledger invariant and certificate-cost-in-benefit from [p8]; the nine-tuple planner state and the six-constructor message vocabulary from [p9]; the scheduler score and reserved quotas from [p1].
§4 Trust	Three layers and the proof-cost budget from [p1]; the five invariants and the review checklist from [p2]; “a type is not a check” from [p3]; axiom auditing against the pinned version from [p4]; the two trust profiles from [p5]; <code>nativeEqTrue</code> and the insufficiency of a single-name blacklist from [p6]; the four-step acceptance procedure from [p9].
§5 Structural search	Conclusion indexing and <code>matchDemand</code> from [p7]; constrained matching from [p2]; tabling and the relative-completeness proposition from [p5]; the slicing propositions from [p3] and [p7]; the accumulator obstruction from [p2] and [p3]; the flexible-variable IH discipline from [p4]; the CEGIS loop and the overfitting trap from [p2]; fragment completeness from [p2] and [p7]; the tag-variable trick from [p7]; the state-machine invariant format from [p5]; Farkas witnesses from [p2]; lattice elimination from [p7]; residue witnesses from [p4]; polynomial witnesses from [p9]; <code>energyAcc</code> from [p3].
§6 Nonlinear	The cone language and its soundness from all nine; the quadratic theorem from [p1] and [p3]; repair-not-rounding from [p1]; target-driven squares from [p5]; the four dictionary-gap arguments from [p2], [p5], [p6], [p7] and [p9]; the Bernstein lemma and the interior-zero theorem from [p1], with [p3]’s quantitative form; the completeness sketch from [p2] and [p3]; square-factor and Sturm certificates from [p1] alone; product envelopes from [p1]; the reuse-the-existing-backend argument from [p9] and [p5].
§7 Boolean and external	The SAT/theory loop from [p7]; CDCL(T), RUP replay and the negation trap from [p6]; the resolution-DAG format, <code>verify_sat</code> and the exhaustive oracle from [p4]; extraction cost vectors from [p7]; solver islands and translation ledgers from [p5]; the reconstruction preference order from [p7]; the no-blanket-combination warning from [p5] and [p9].
§8 Integration	<code>Params/assertExtra/mkGoalCore</code> from [p2] and [p9]; the solver-extension callbacks from [p4]; the rollback-is-not-enough point from [p6]; three reconstruction levels from [p2]; the module layout merged from [p1], [p2], [p3], [p5] and [p9]; the acceptance contract table from [p5]; the deployment pattern from [p9]; the gates merged from all nine.
§9 Experiments	Each run reports itself.
§10 Evaluation	Leakage control from [p1], [p5] and [p9]; the three input conditions from [p2]; the two tracks from [p4]; the non-communicating portfolio from [p1] and [p6]; metric discipline from [p4] and [p9]; the illustrative acceptance target from [p9].

Section	Principal sources
§A Schemas	[p5]’s field names as the base, [p3]’s kind discriminator, [p8]’s <code>{index, multiplier}</code> ideals, [p1]’s bounded decoder, [p3]’s duplicate-key rejection, [p6]’s anti-vacuity guards and ignore-the-residual rule, [p9]’s typed decoding gate.

B.3 Ideas that survive in exactly one source

These were the merge’s first concern, since a careless consolidation loses precisely the material that distinguishes one submission from the rest.

- **Square-factor and Sturm certificates** ([p1]) — the only mechanism anywhere in the collection that can certify a polynomial with an interior zero, and the constructive answer to Theorem 6.5.
- **Integer-lattice elimination with a divisibility obstruction** ([p7]) — the only worker that emits a certificate of *infeasibility*.
- **Finite-residue witnesses** ([p4]) — the only existential-over- $\mathbb{Z}$ -by-case-analysis certificate.
- **Polynomial witnesses discharged by two cone certificates** ([p9]) — the smallest complete instance of the composition pattern the whole architecture argues for.
- **Farkas affine witnesses as one feasibility problem** ([p2]) — the observation that synthesis and proof search are jointly linear.
- **Proof-logging CDCL(T) with independent RUP re-derivation** ([p6]), and the resolution-DAG alternative ([p4]).
- **Theory-constrained quantifier instantiation** ([p2]) — emitting an equality constraint at an interpreted subterm instead of failing.
- **Certificate cost inside the scheduling benefit** ([p8]).
- **The quantitative interior-zero coefficient** $(\ell - 1/3)(u - 1/3) < 0$ ([p3]).
- **The rank-one Gram argument** for $(x+y+1)^2$ ([p9]) and the **weight-6-versus-coefficient-4 argument** for $(2x - 3y)^2$ ([p7]).
- **The Lean design contracts** — the runtime and scheduler contract ([p3]) and the `CertificateSpec` soundness contract ([p6]); these complement rather than duplicate each other, and both now elaborate.

B.4 Disagreements, and how they were resolved

The nine did not agree on everything. Where they conflicted, this document took the more conservative position and says so.

Disagreement	Resolution
Build or reuse the nonlinear backend	[p9] and [p5] identified an existing Lean <code>sos</code> project and a Lean <code>lp</code> project; the other seven proposed building cone and witness machinery from scratch without mentioning either. Resolved in favour of the conservative reading: the in-house searches are a cheap first stage, the existing libraries are the production backend, and “add an SOS tactic to Lean” is not claimed as a contribution (Remark 6.6).
What <code>grind</code> already does	[p6] conceded conclusion-selected backward patterns to the baseline; [p1] listed them as new work. The concession was adopted.
Native-evaluation trust	Three confidence levels were recorded: an RFC-only reading, a tactic-reference reading, and [p6]’s inspection of <code>Lean.Meta.Native</code> and <code>nativeEqTrue</code> . The most specific source was used, with its own instruction that the behaviour must be re-audited against the pinned implementation rather than inherited from a design thread.
Is a stable snapshot API available?	[p4] called the solver-extension callbacks “real integration points, not hypothetical hooks”; [p3] declined to assume any stable public API. Both are reported: the inventory is [p4]’s, the framing is [p3]’s.
Bernstein split policy	Round-robin axis bisection ([p4]) against largest-relative-width ([p3]) against <code>depth mod n</code> ([p2]). All three make every coordinate shrink along a branch, which is what Proposition 6.4 needs; largest- <i>absolute</i> -width does not, and is rejected.
Horn forward-search growth	[p5] reports $2^{d+1} - 1$ facts and [p9] reports 2^d on the same rule family. The difference is entirely the forward baseline’s stopping rule. Both tables are printed, side by side, as a methodological example (Table 18).
Quadratic matrix convention	[p3] places a linear coefficient at $H_{0i} = b_i/2$; [p4] states it as $2Q_{0i}$. These are inverse readings of the same matrix. §6.4 uses one convention throughout.
Accumulator naming	<code>ra/la</code> ([p3]), <code>qrev</code> ([p4]), <code>revAux</code> ([p2]), <code>revAcc</code> ([p9]). One naming is used per derivation; they are never mixed inside one chain of equalities.
Pigeonhole encodings	[p4] and [p6] use different clause sets and report different columns. Both tables are kept, explicitly labelled as non-comparable.

B.5 What the merge changed

Four things in this document are not simply inherited.

First, the nine experiment sections are reported as nine, with the non-pooling rule stated explicitly and the four Horn setups tabulated against each other. Several of the source articles warned against pooling their *own* numbers; none could warn against pooling across articles, because none knew about the others.

Second, the bibliography is deduplicated: roughly 215 `bibitem` entries under about 100 keys became 56 entries under descriptive keys, keeping the richest description of each work and preserving details only one source had recorded — inspected blob SHA-1s, the release timestamp, the blob-id-is-not-a-commit-id caveat, and the `sos` toolchain version mismatch.

Third, the build is plain pdfL^AT_EX with stock packages. The nine used four engines and nine font families between them, and every one of their READMEs had to explain what to do when a font was missing.

Fourth, and the only new evidence: three Lean files were compiled (Table 19). Every source recorded `NOT_RUN`. That is now narrower — three of sixteen files elaborate under Lean v4.34.0 — and the limits of what it shows are stated where the result appears.

B.6 Conventions used in this document

`\status{...}` banners, inherited from several of the sources, mark whether a mechanism was implemented and executed, specified only, or proposed for production. `[pN]` labels attribute a result to the run that produced it. `UNKNOWN` always means a bounded search found nothing; it never means a statement is false.

References

- [1] Lean contributors. *The Lean Language Reference: The `grind` tactic*. Rolling reference; inspected 14 September 2026, when the served version was 4.34.0. Describes `grind` as cooperating proof-producing engines organised around contradiction, congruence closure, constraint propagation, E-matching, guided case analysis, and theory solvers. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [2] Kim Morrison and Leonardo de Moura. *grind: An SMT-Inspired Tactic for Lean 4 (Short Paper — System Description)*. IJCAR 2026, LNCS 16688, pp. 106–114. DOI: 10.1007/978-3-032-32589-1_7. Author’s HTML version: <https://kim-em.github.io/papers/grind/>.
- [3] Lean contributors. *Algebraic Solver (Commutative Rings, Fields)*. The Lean Language Reference. Inspected 14 September 2026. https://lean-lang.org/doc/reference/latest/The--grind--tactic/Algebraic-Solver-_LPAR_Commutative-Rings__Fields_RPAR_/.
- [4] Lean contributors. *Linear Arithmetic Solver*. The Lean Language Reference. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/Linear-Arithmetic-Solver/>.
- [5] Lean contributors. *Linear Integer Arithmetic*. The Lean Language Reference. Documents the integer constraint forms and model-based theory combination that are part of the baseline. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/Linear-Integer-Arithmetic/>.
- [6] Lean contributors. *E-matching*. The Lean Language Reference. https://lean-lang.org/doc/reference/latest/The--grind--tactic/E__matching/.
- [7] Lean contributors. *Case Analysis*. The Lean Language Reference. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/Case-Analysis/>.
- [8] Lean project. *Lean v4.34.0 release*, published 14 September 2026; release metadata inspected through the GitHub API on the same date, with publication timestamp 2026-09-14T14:04:36Z. Release notes include “Bit-blasting Inside `grind` =>”. <https://github.com/leanprover/lean4/releases/tag/v4.34.0>.
- [9] Lean project. *The Lean Language Reference*. Current version observed as 4.34.0 on 14 September 2026. <https://lean-lang.org/doc/reference/latest/>.
- [10] Lean contributors. `src/Init/Grind/Config.lean`, tag v4.34.0. Pinned implementation source for the search-branch, E-matching, generation, and instance limits quoted in this article. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Init/Grind/Config.lean>.
- [11] Lean contributors. `src/Lean/Meta/Tactic/Grind/Types.lean`, tag v4.34.0. Pinned implementation source for the saved-state machinery and proof-bearing types. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Grind/Types.lean>.
- [12] Lean contributors. `src/Lean/Meta/Tactic/Grind/Main.lean`, tag v4.34.0. Source of the `Params`, `assertExtra`, `GrindM.run`, `mkGoalCore` and `Result` entry points discussed in the integration section. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Grind/Main.lean>.
- [13] Leonardo de Moura and Lean contributors. `src/Lean/Meta/Tactic/Grind/Action.lean`, tag v4.34.0. Inspected source describing the continuation-based control interface, backtracking, and script reconstruction. Inspected blob: `ad212d7c8ef8277b4be3650d43738ee98a32cbf5`. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Grind/Action.lean>.
- [14] Lean contributors. `src/Lean/Meta/Tactic/Grind/Finish.lean`, tag v4.34.0. Inspected blob: `6bae01e1c2c4772d0cabe99ff74437492ccebfc5`. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Grind/Finish.lean>.
- [15] Lean contributors. `src/Lean/Meta/Tactic/Grind.lean`, tag v4.34.0. Inspected blob: `db642e7864020148fab27822bf83c445de83d02d`. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Grind.lean>.
- [16] Lean contributors. `src/Lean/Meta/Tactic/Grind/Arith/CommRing/Types.lean`, tag v4.34.0. Commutative-ring constraint types. <https://raw.githubusercontent.com/leanprover/lean4/v4.34.0/src/Lean/Meta/Tactic/Grind/Arith/CommRing/Types.lean>.
- [17] Lean contributors. *Lean.Meta.Tactic.Grind.Types*. Generated API documentation, including solver-extension interfaces. Inspected 14 September 2026. <https://lean-lang.org/doc/api/Lean/Meta/Tactic/Grind/Types.html>.
- [18] Mathlib contributors. Repository revision `1cf325a0cf67aca2b04d76b5380ff6a9e410aefa` and its

- lean-toolchain file, which specifies leanprover/lean4:v4.34.0. <https://github.com/leanprover-community/mathlib4/blob/1cf325a0cf67aca2b04d76b5380ff6a9e410aefa/lean-toolchain>.
- [19] Lean contributors. *The Lean Language Reference: Tactic Reference*. Entries for `native_decide`, `bv_decide`, `decide_cbv`, `omega`, induction, SAT-solver integration, and call-by-value evaluation, with their associated trust behaviour. Inspected 14 September 2026. <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/>.
 - [20] Lean contributors. *Validating a Lean Proof*. The Lean Language Reference, including native-computation changes since Lean 4.29.0. <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>.
 - [21] Lean contributors. *Lean.Meta.Native*, source at tag v4.34.0; in particular `nativeEqTrue` and its generated-axiom path. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Native.lean>.
 - [22] Lean contributors. *RFC: One axiom per native computation*, issue 12216, opened 28 January 2026. Historical design discussion; the exact current behaviour must be audited against the pinned implementation rather than assumed from this thread. <https://github.com/leanprover/lean4/issues/12216>.
 - [23] Lean contributors. *Lean.Elab.Tactic.BVDecide*. Generated source API documentation. Inspected 14 September 2026. https://leanprover-community.github.io/mathlib4_docs/Lean/Elab/Tactic/BVDecide.html.
 - [24] Mario Carneiro. *Lean4Lean: Towards a Verified Typechecker for Lean, in Lean*. arXiv:2403.14064, 2024. <https://arxiv.org/abs/2403.14064>.
 - [25] Aesop contributors. *Aesop: Automated Extensible Search for Obvious Proofs* (white-box automation for Lean 4). Repository documentation, including the search tree, safe/unsafe rule classification, best-first exploration, and induction examples. Observed README blob: f75ec1f8bebcf6d857fc3954cf91cf99f323a758. Inspected 14 September 2026. <https://github.com/leanprover-community/aesop>.
 - [26] Joshua Clune, Yicheng Qian, Alex Bentkamp, and contributors. *Duper*. Repository README: proof-producing saturation, manual premise selection, and a configurable portfolio. Observed README blob: 1859f7fa914fbd8151b274fd34e46f3a09ef1b2d. Inspected 14 September 2026. <https://github.com/leanprover-community/duper>.
 - [27] Yicheng Qian, Joshua Clune, Clark Barrett, and Jeremy Avigad. *Lean-auto: An Interface between Lean 4 and Automated Theorem Provers*. CAV 2025; arXiv:2505.14929. Translation and monomorphization infrastructure. <https://arxiv.org/abs/2505.14929>; <https://github.com/leanprover-community/lean-auto>.
 - [28] Abdalrhman Mohamed, Tomaz Mascarenhas, Harun Khan, Haniel Barbosa, Andrew Reynolds, Yicheng Qian, Cesare Tinelli, and Clark Barrett. *Lean-SMT: An SMT tactic for discharging proof goals in Lean*. CAV 2025; arXiv:2505.15796v1. Documented behaviour includes returning proof-reconstruction holes as goals; its supported theories are not equivalent to the full capability set of cvc5. <https://arxiv.org/abs/2505.15796>; <https://github.com/ufmg-smite/lean-smt>.
 - [29] Joshua Clune, Haniel Barbosa, and Jeremy Avigad. *Hint-Based SMT Proof Reconstruction*. arXiv:2601.14495, January 2026. <https://arxiv.org/abs/2601.14495>; <https://github.com/JOSHCLUNE/QuerySMT>.
 - [30] Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Qiaochu Jiang, and Sean Welleck. *Premise Selection for a Lean Hammer*. arXiv:2506.07477, 2025 (later versions to 25 February 2026). <https://arxiv.org/abs/2506.07477>; <https://github.com/JOSHCLUNE/LeanHammer>.
 - [31] Lean SOS contributors. *SOS: sum-of-squares nonlinear real arithmetic in Lean 4*, after Harrison’s decision procedure. Repository README, architecture, certificate interface, and stated limitations. Inspected 14 September 2026. Its documented architecture separates reification, search, checking, and verification. <https://github.com/leanprover/sos>.
 - [32] Lean SOS project. *SOS/Engine.lean*, observed main source, 14 September 2026. Source of the `SOS.Engine.solve` and `SOS.Engine.Result.check` interfaces named as integration targets. <https://raw.githubusercontent.com/leanprover/sos/main/SOS/Engine.lean>.
 - [33] Lean SOS project. *SOS/Core.lean*, observed main source. <https://raw.githubusercontent.com/leanprover/sos/main/SOS/Core.lean>.
 - [34] Lean SOS project. *lean-toolchain*, retrieved main snapshot specifying v4.32.0-rc1. Recorded because it differs from the v4.34.0 baseline used here; it is not a verified common build manifest.

<https://raw.githubusercontent.com/leanprover/sos/main/lean-toolchain>.

- [35] Lean Hex project. *hex-mv-poly: Mathlib-free executable multivariate polynomials for Lean 4*. The computational polynomial substrate used by the Lean sos project. <https://github.com/leanprover/hex-mv-poly>.
- [36] Lean LP contributors. *Linear programming in Lean 4*. Repository package, backend, tactic, and trust-model documentation. Inspected 14 September 2026. <https://github.com/leanprover/lp>.
- [37] Kim Morrison. *Verified linear programming*, 14 June 2026. Author’s technical overview. <https://kim-em.github.io/blog/2026-6-14-verified-linear-programming/>.
- [38] Stefan Szeider. *PBLean: Pseudo-Boolean Proof Certificates for Lean 4*. arXiv:2602.08692, 2026, revised 2 April 2026. <https://arxiv.org/abs/2602.08692>.
- [39] Evan Wang, Simon Chess, Sophie Szeto, and Theodore Meek. *Learned Interventions in Lean 4 grind*. arXiv:2607.22972v2, 28 July 2026. Cited for its stated approach only; its experiments are not reproduced here, and the failure-triggered cascade it describes is prior art rather than a contribution of this design. <https://arxiv.org/abs/2607.22972>.
- [40] Mathlib contributors. *Mathlib.Tactic.Linarith.Frontend*. Generated documentation and implementation overview: certificate search separated from proof reconstruction, plus the specific nonlinear preprocessing of `nlinarith`. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Linarith/Frontend.html.
- [41] Mathlib contributors. *Mathlib.Tactic.Linarith.Verification*. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Linarith/Verification.html.
- [42] Mathlib contributors. *Mathlib.Tactic.Positivity.Basic* and *Mathlib.Tactic.Positivity.Core*. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Positivity/Basic.html.
- [43] Mathlib contributors. *Mathlib.Tactic.Ring.Basic*. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Ring/Basic.html.
- [44] Mathlib contributors. *Mathlib.Tactic.LinearCombination*. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/LinearCombination.html.
- [45] Marcus Rossel and contributors. *lean-egg*. Repository README, marked deprecated when accessed on 14 September 2026 and directing users toward `grind`. Observed blob: `0dbfa063b5a77c6e892eaa265f068437980812c1`. Also listed on Lean Reservoir as `@marcusrassel/egg`. <https://github.com/marcusrassel/lean-egg>.
- [46] Marcus Rossel and Andrés Goens. *Bridging Syntax and Semantics of Lean Expressions in E-Graphs*. arXiv:2405.10188, 2024. <https://arxiv.org/abs/2405.10188>.
- [47] Marcus Rossel, Rudi Schneider, Thomas Koehler, Michel Steuwer, and Andrés Goens. *Towards Pen-and-Paper-Style Equational Reasoning in Interactive Theorem Provers by Equality Saturation*. Proceedings of the ACM on Programming Languages, 2026. DOI: 10.1145/3776667. <https://doi.org/10.1145/3776667>.
- [48] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. *egg: Fast and Extensible Equality Saturation*. Proceedings of the ACM on Programming Languages 5 (POPL), 2021. DOI: 10.1145/3434304. <https://arxiv.org/abs/2004.03082>.
- [49] The egg contributors. *egg: e-graphs good*. Project and library documentation. <https://egraphs-good.github.io/>; <https://github.com/egraphs-good/egg>.
- [50] Cole Kurashige, Ruyi Ji, Aditya Giridharan, Mark Barbone, Daniel Noor, Shachar Itzhaky, Ranjit Jhala, and Nadia Polikarpova. *CCLemma: E-Graph Guided Lemma Discovery for Inductive Equational Proofs*. Proceedings of the ACM on Programming Languages 8 (ICFP), article 264, 2024. DOI: 10.1145/3674653. <https://icfp24.sigplan.org/details/icfp-2024-papers/29/CCLemma-E-Graph-Guided-Lemma-Discovery-for-Inductive-Equational-Proofs>.
- [51] Alan Bundy, Andrew Stevens, Frank van Harmelen, Andrew Ireland, and Alan Smaill. *Rippling: A heuristic for guiding inductive proofs*. Artificial Intelligence, 62(2):185–253, 1993. <https://www.sciencedirect.com/science/article/pii/000437029390079Q>.
- [52] Yican Sun, Ruyi Ji, Jian Fang, Xuanlin Jiang, Mingshuai Chen, and Yingfei Xiong. *Proving Functional Program Equivalence via Directed Lemma Synthesis*. 2024. <https://arxiv.org/abs/2405.11535>.
- [53] Armando Solar-Lezama. *Program Synthesis by Sketching*. PhD thesis, University of California, Berkeley; Technical Report UCB/EECS-2008-177, 2008. The origin of the counterexample-guided inductive synthesis loop reused by the invariant and witness workers. <https://www2.eecs.berkeley.edu/Pub>

- s/TechRpts/2008/EECS-2008-177.html.
- [54] Chase Norman and Jeremy Avigad. *Canonical for Automated Theorem Proving in Lean*. 2025. <https://arxiv.org/abs/2504.06239>.
 - [55] Chase Norman and Jeremy Avigad. *Implementing Dependent Type Theory Inhabitation and Unification*. 2026. Introduces DTTBench. <https://arxiv.org/abs/2603.01463>.
 - [56] John Harrison. *Verifying nonlinear real formulas via sums of squares*. In *Theorem Proving in Higher Order Logics (TPHOLs 2007)*, LNCS 4732, pp. 102–118, Springer, 2007. The foundational separation of sophisticated certificate search from comparatively straightforward verification. DOI: 10.1007/978-3-540-74591-4_9. <https://www.cl.cam.ac.uk/~jrh13/papers/sos.html>.
 - [57] Robert Nieuwenhuis, Albert Oliveras, and Cesare Tinelli. *Solving SAT and SAT Modulo Theories: From an Abstract Davis–Putnam–Logemann–Loveland Procedure to DPLL(T)*. *Journal of the ACM* 53(6), 937–977, 2006. DOI: 10.1145/1217856.1217859.
 - [58] Victor Magron, Mohab Safey El Din, and Trung-Hieu Vu. *Sum of Squares Decompositions of Polynomials over their Gradient Ideals with Rational Coefficients*. arXiv:2107.11825, 2021. <https://arxiv.org/abs/2107.11825>.
 - [59] NASA Langley Formal Methods Program. *PVS Bernstein Library*. Prior art for formalised Bernstein-basis bounds on boxes. <https://shemesh.larc.nasa.gov/fm/pvs/Bernstein/>.
 - [60] cvc5 developers. *Proof Production*. Documentation of the Cooperating Proof Calculus and external proof formats. <https://cvc5.github.io/docs/latest/proofs/proofs.html>.
 - [61] Z3 project. *Quantifiers*. Z3 Guide, including patterns and model-based quantifier instantiation. <https://microsoft.github.io/z3guide/docs/logic/Quantifiers/>.
 - [62] MOSEK contributors. *MOSEK Modeling Cookbook: linear optimization, infeasibility certificates, and duality*. <https://docs.mosek.com/modeling-cookbook/linear.html>.
 - [63] SciPy contributors. *scipy.optimize.linprog*, including `method='highs'`. Official API documentation. The executed prototypes used SciPy 1.17.0; the online reference may be newer. This is an optimization interface, not a proof system. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>; <https://docs.scipy.org/doc/scipy/reference/optimize.linprog-highs.html>.
 - [64] SymPy contributors. *Polynomials Manipulation Module Reference* and the matrix/linear-algebra modules. Used for rational factorization, Sturm sequences, root counting, and independent symbolic cross-checks. Executed version: SymPy 1.14.0. <https://docs.sympy.org/latest/modules/polys/reference.html>.
 - [65] Steven Diamond and Stephen Boyd. *CVXPY: A Python-Embedded Modeling Language for Convex Optimization*. arXiv:1603.00943, 2016. Proposed optional semidefinite front end; not an installed or tested dependency of any prototype here. <https://arxiv.org/abs/1603.00943>; <https://www.cvxpy.org/tutorial/constraints/index.html>.
 - [66] COIN-OR. *CSDP: A solver for semidefinite programming problems*. Proposed optional backend; not used in any executed experiment. <https://github.com/coin-or/Csdp>.